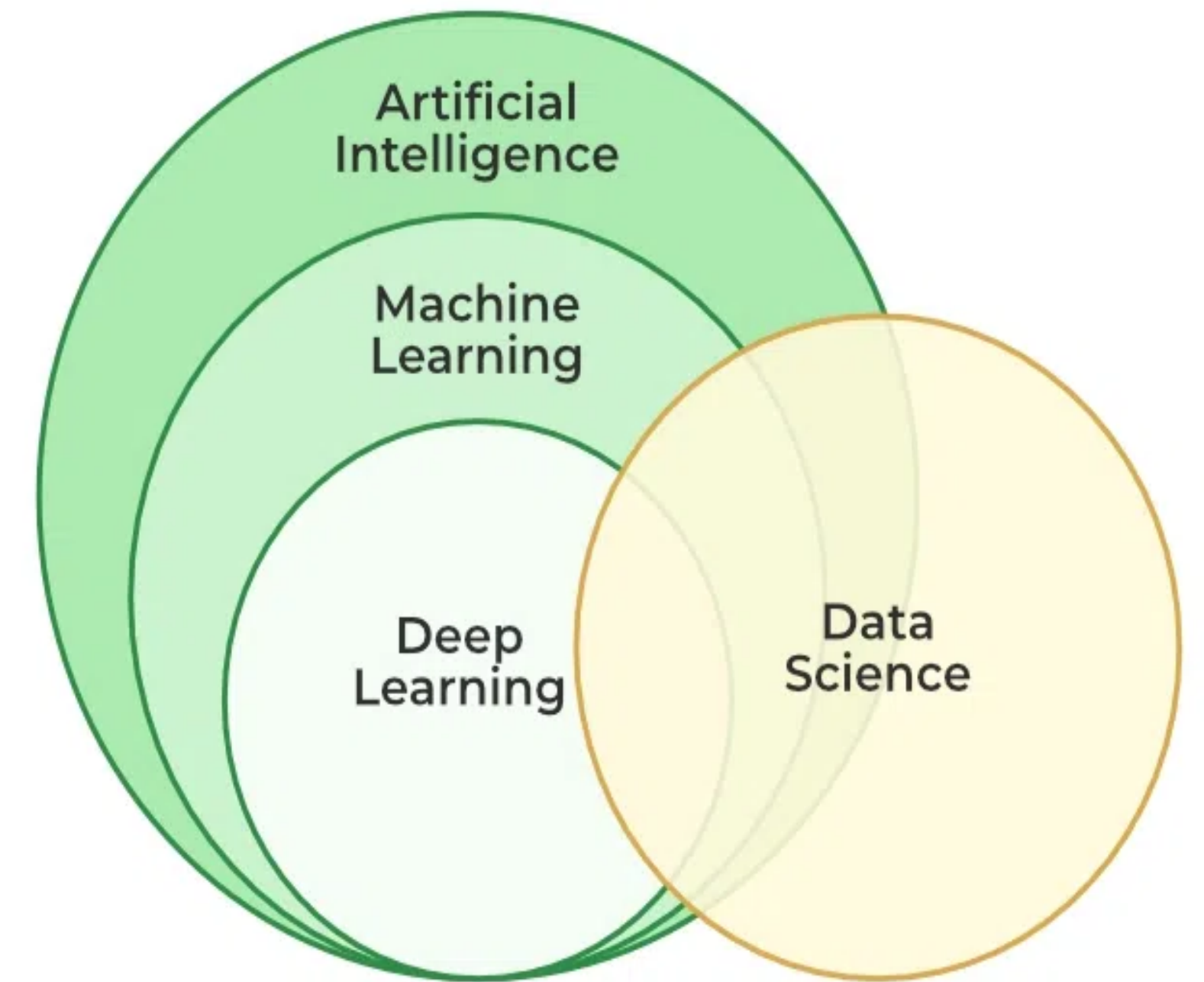


Introduction to Neural Networks and Deep Neural Networks

Dr. Thomas Abraham J V
Associate Professor,
VIT Chennai

How do AI, machine learning, deep learning and neural networks relate to each other?

- Artificial Intelligence
 - Artificial intelligence or AI, the broadest term of the three, is used to classify machines that mimic human intelligence and human cognitive functions like problem-solving and learning. AI uses predictions and automation to optimize and solve complex tasks that humans have historically done, such as facial and speech recognition, decision-making and translation.
- Machine Learning
 - Machine learning is a subset of AI that allows for optimization. When set up correctly, it helps you make predictions that minimize the errors that arise from merely guessing. Human experts determine the hierarchy of features to understand the differences between data inputs, usually requiring more structured data to learn.
- Deep Learning
 - deep learning is a subset of machine learning. The primary difference between machine learning and deep learning is how each algorithm learns and how much data each type of algorithm uses.
 - Deep learning automates much of the feature extraction piece of the process, eliminating some of the manual human intervention required.



What is a neural network?

- Neural networks, also called artificial neural networks or simulated neural networks, are a subset of machine learning and are the backbone of deep learning algorithms. They are called “neural” because they mimic how neurons in the brain signal one another.

Biological vs Artificial Neurons

- **Biological Neural Network:**

Biological Neural Network (BNN) is a structure that consists of Synapse, dendrites, cell body, and axon. In this neural network, the processing is carried out by neurons. Dendrites receive signals from other neurons, Soma sums all the incoming signals and axon transmits the signals to other cells.

- A biological neuron has a cell body or soma to process the impulses, dendrites to receive them, and an axon that transfers them to other neurons.

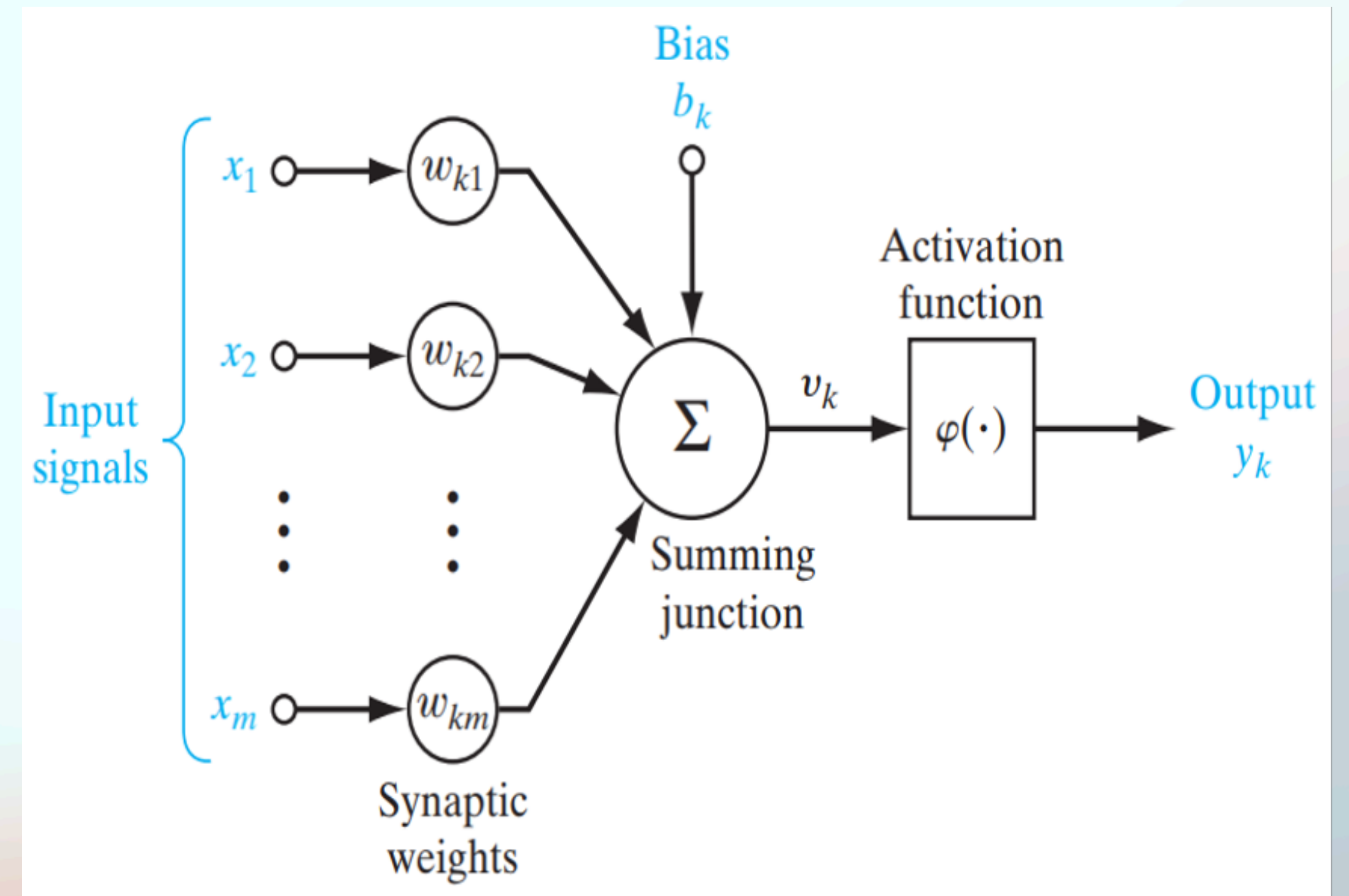
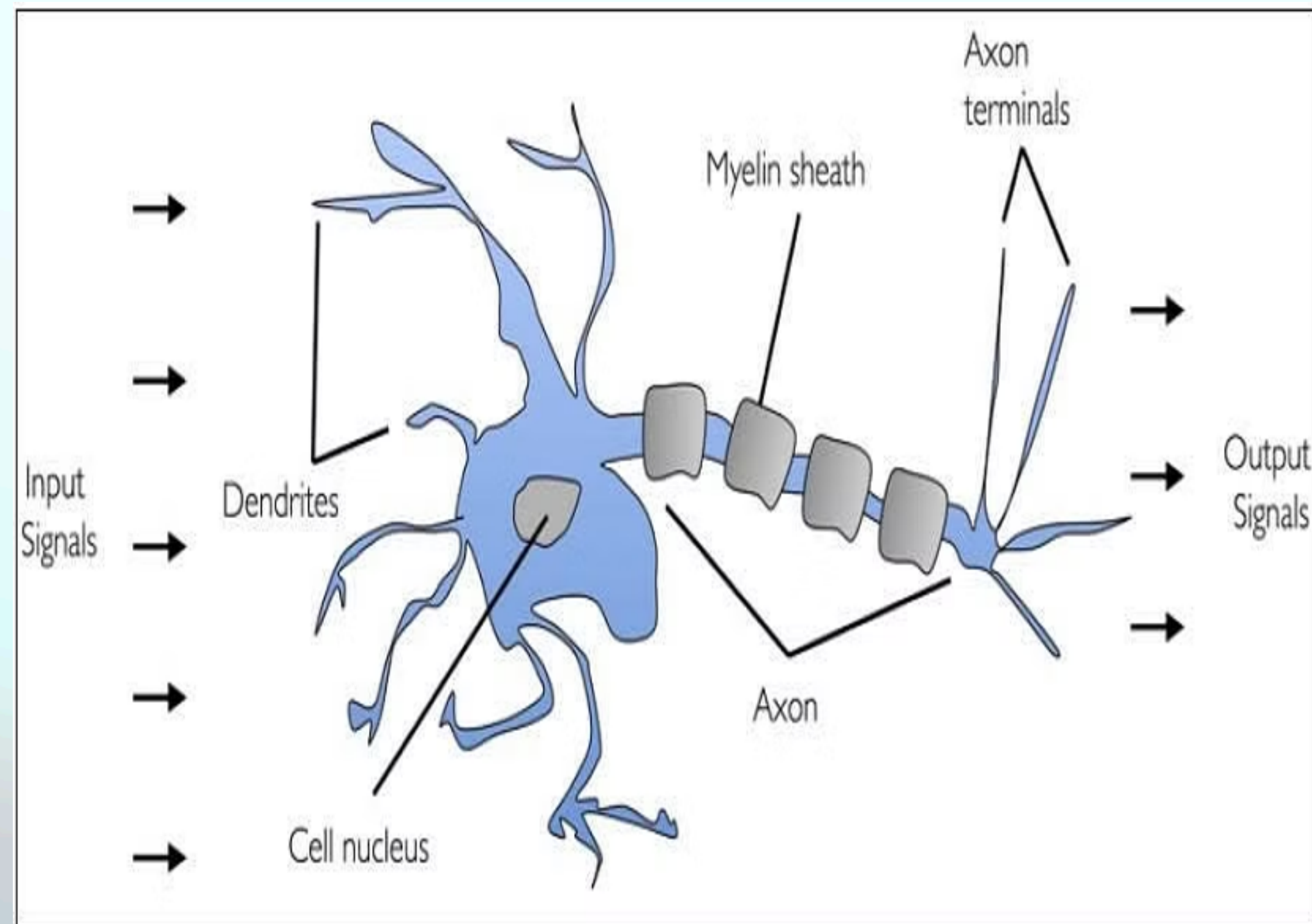
- **Artificial Neural Network:**

Artificial Neural Network (ANN) is a type of neural network that is based on a Feedforward strategy. It is called this because they pass information through the nodes continuously till it reaches the output node. This is also known as the simplest type of neural network.

Artificial Neurons

- Artificial neurons, also known as units, are found in artificial neural networks.
- The whole Artificial Neural Network is composed of these artificial neurons, which are arranged in a series of layers.
- The complexities of neural networks will depend on the complexities of the underlying patterns in the dataset whether a layer has a dozen units or millions of units.
- Commonly, Artificial Neural Network has an input layer, an output layer as well as hidden layers. The input layer receives data from the outside world which the neural network needs to analyze or learn about.

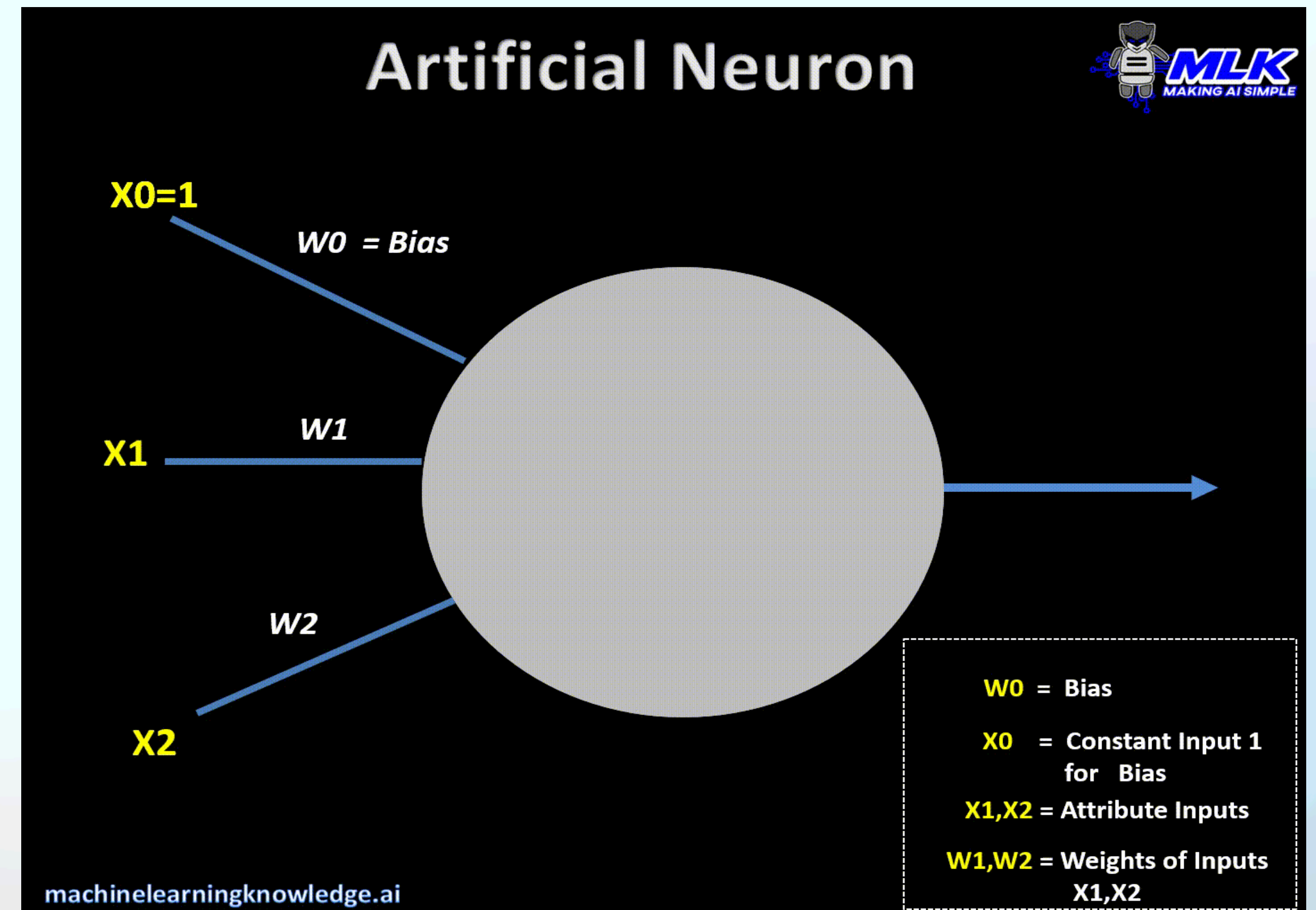
Biological vs Artificial Neurons



Biological vs Artificial Neurons

- Biological neuron has three basic functionality
- Receive signal from outside.
- Process the signal and enhance whether we need to send information or not.
- Communicate the signal to the target cell which can be another neuron or gland.
- A biological neuron has a cell body or soma to process the impulses, dendrites to receive them, and an axon that transfers them to other neurons.

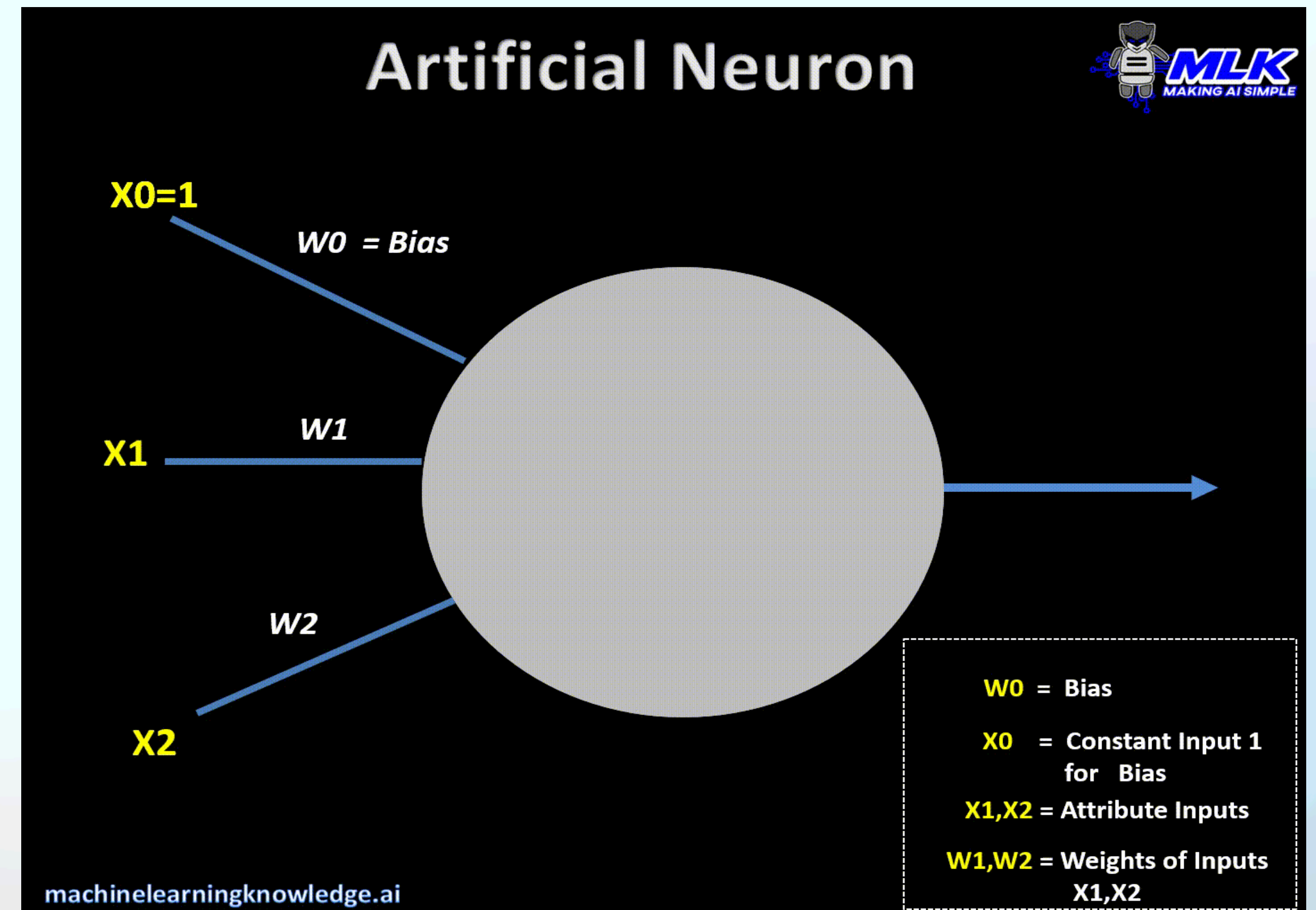
In the same way, neural networks also work.



Biological vs Artificial Neurons

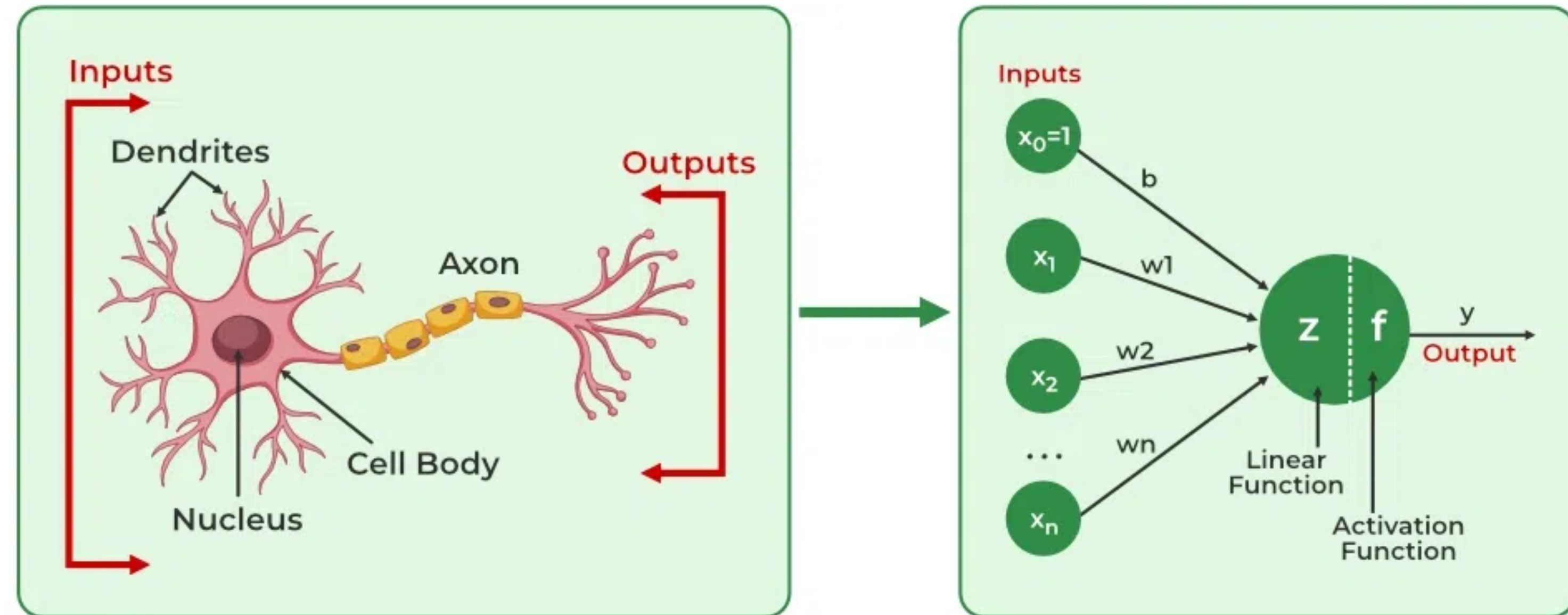
- Biological neuron has three basic functionality
- Receive signal from outside.
- Process the signal and enhance whether we need to send information or not.
- Communicate the signal to the target cell which can be another neuron or gland.
- A biological neuron has a cell body or soma to process the impulses, dendrites to receive them, and an axon that transfers them to other neurons.

In the same way, neural networks also work.



Biological vs Artificial Neurons

Biological Neuron	Artificial Neuron
Dendrites	Inputs
Cell nucleus or Soma	Nodes
Synapses	Weights
Axon	Outputs



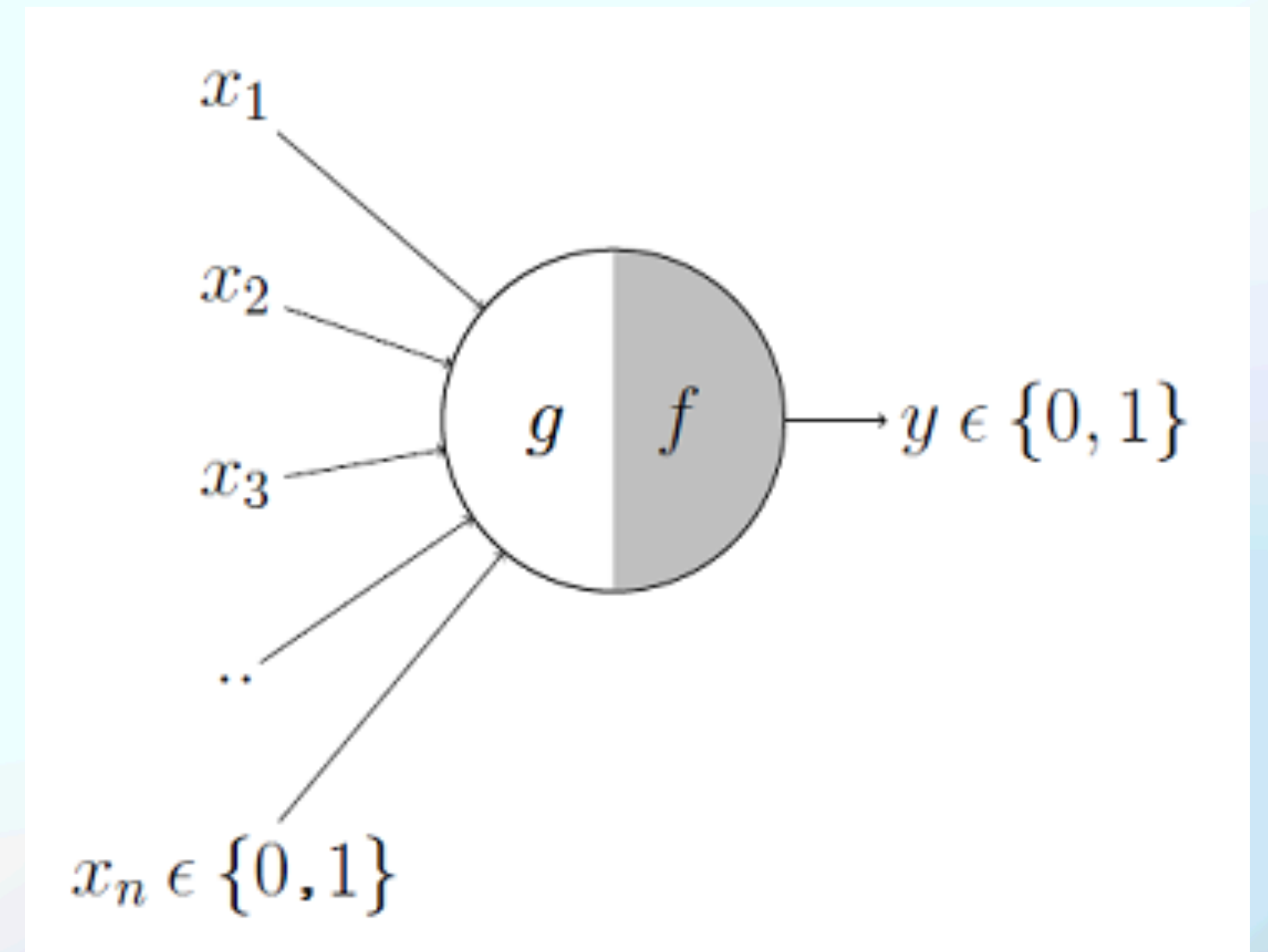
Neurons so far...

- **McCulloch Pitts Neuron (1943)**

This was the first artificial neuron ever conceptualized in 1943 and was actually an attempt to represent biological neurons with a mathematical model.

It used step function as activation function.

The input and output are both binary in nature. The input does not have weights associated with it. It is not a true machine learning model because it did not have any weights to learn. The threshold value of step function is instead calculated manually. It works only with linear data and cannot work with non linear data.



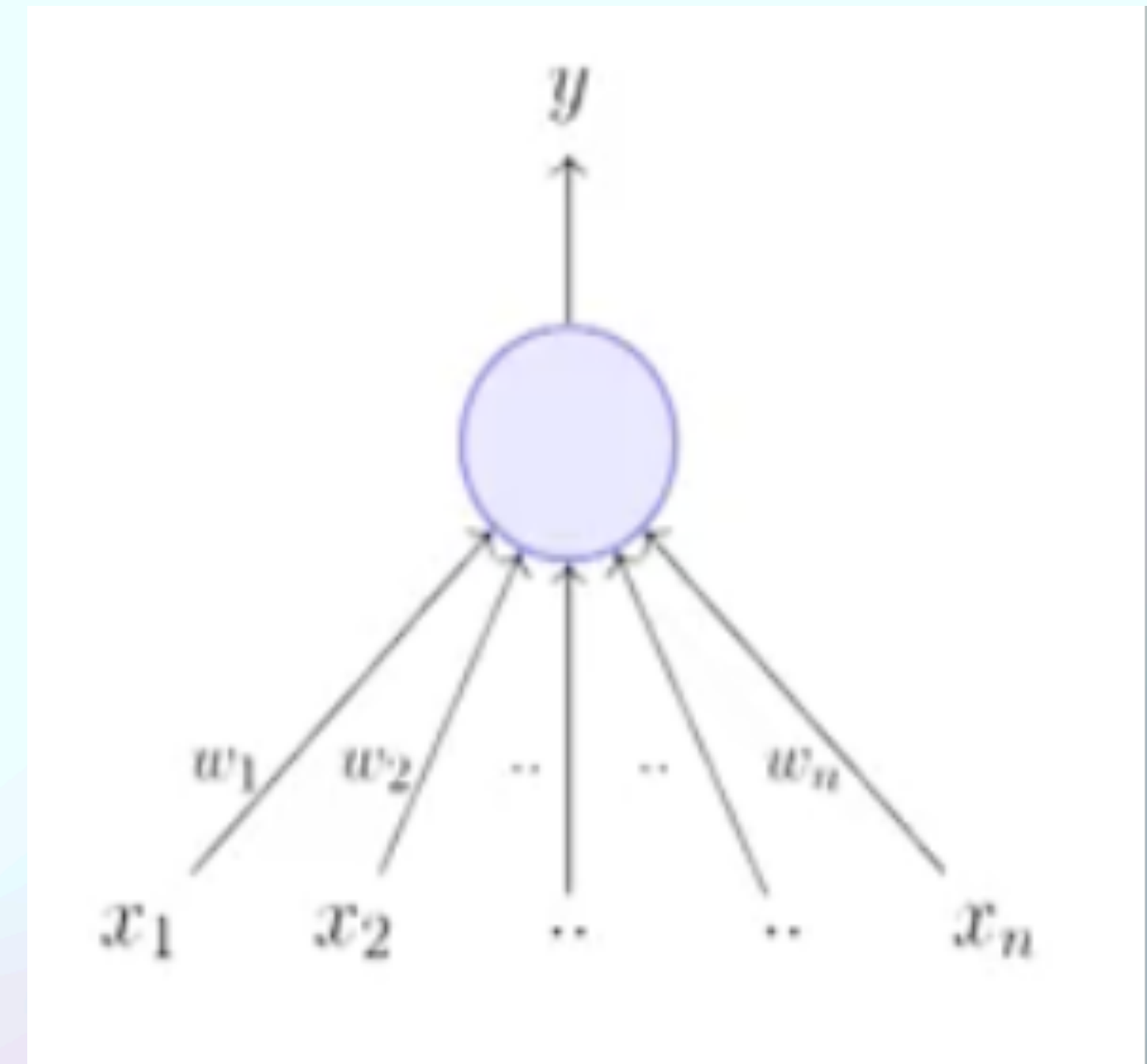
Neurons so far...

- **Perceptron (1957)**

Perceptron also uses a step activation function but has some improvements over McCulloch Pitts Neuron.

The inputs can be any real number and has weights associated with it. The weights value can be learnt with training data and so it is a true machine learning model.

Since it uses step activation function the output is still binary 0 or 1. Also because of step activation function, there is a sudden change in decision from 0 to 1 at threshold value. This sudden change may not be appreciated in real world problem. It still cannot work with non-linear data.



Neurons so far....

- **Sigmoid Neuron**

This neuron uses sigmoid function for activation and is further improvement over Perceptron.

By nature of sigmoid function, there is a smooth transition from 0 to 1. There is no sudden change from 0 to 1 like step function, which was an issue with previous neurons. And because of this sigmoid function, the output is not binary but ranges from 0 to 1.

Sigmoid function is a non linear function so this neuron can work with non linear data up to certain extend.

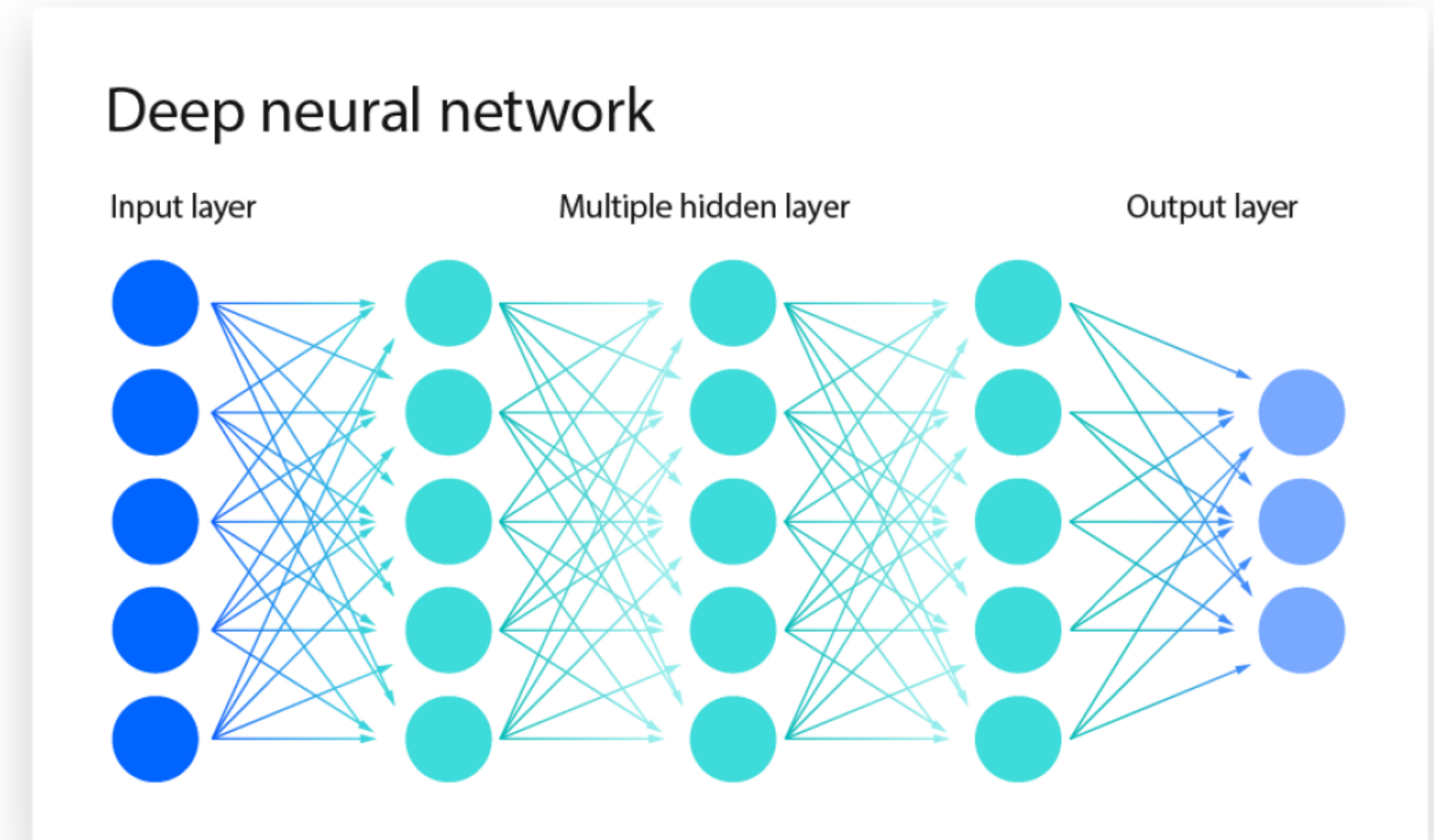
- It is interesting to note here that Sigmoid neuron actually creates a major improvement over Perceptron by just changing the activation function from Step function to Sigmoid function.

Artificial neural networks

- Artificial neural networks are built on the principles of the structure and operation of human neurons. It is also known as neural networks or neural nets.
- An artificial neural network's input layer, which is the first layer, receives input from external sources and passes it on to the hidden layer, which is the second layer.
- Each neuron in the hidden layer gets information from the neurons in the previous layer, computes the weighted total, and then transfers it to the neurons in the next layer. When one neuron's output exceeds the threshold value, it is activated and sends its data to the network's next layer. If it's below the threshold, no data passes along.
- These connections are weighted, which means that the impacts of the inputs from the preceding layer are more or less optimized by giving each input a distinct weight. These weights are then adjusted during the training process to enhance the performance of the model.

What's the difference between deep learning and neural networks?

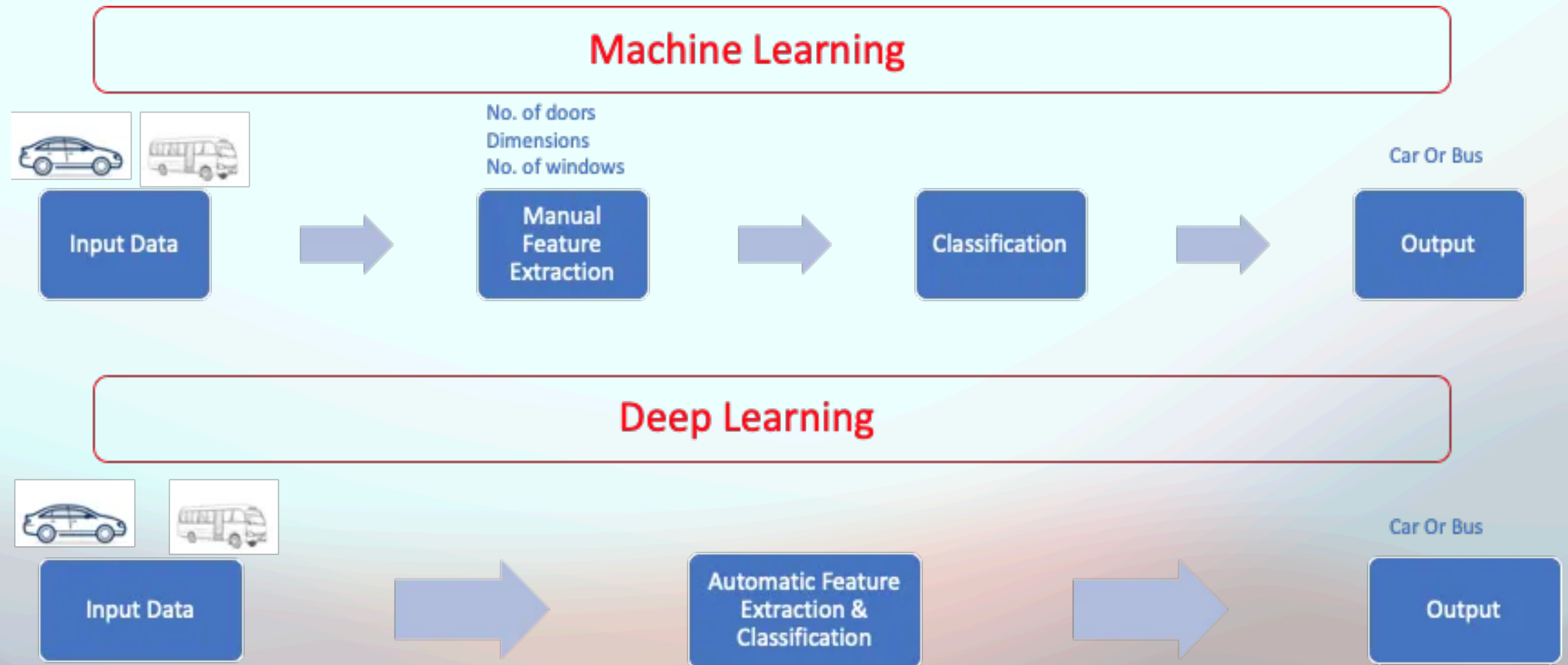
- The “deep” in deep learning refers to the depth of layers in a neural network. A neural network of more than three layers, including the inputs and the output, can be considered a deep-learning algorithm.



Difference between Machine Learning and Deep Learning

Machine Learning	Deep Learning
Apply statistical algorithms to learn the hidden patterns and relationships in the dataset.	Uses artificial neural network architecture to learn the hidden patterns and relationships in the dataset.
Can work on the smaller amount of dataset	Requires the larger volume of dataset compared to machine learning
Better for the low-label task.	Better for complex task like image processing, natural language processing, etc.
Takes less time to train the model.	Takes more time to train the model.
A model is created by relevant features which are manually extracted from images to detect an object in the image.	Relevant features are automatically extracted from images. It is an end-to-end learning process.
Less complex and easy to interpret the result.	More complex, it works like the black box interpretations of the result are not easy.

Difference between Machine Learning and Deep Learning



Deep Learning Frameworks

- Building a deep learning solution is a big challenge because of its complexity
- Frameworks are tools to ease the building of deep learning solutions
- Frameworks offer a higher level of abstraction and simplify potentially difficult programming tasks.

Deep Learning Frameworks



Deep Learning Frameworks

- Tensorflow

Developed by Google, it boasts a comprehensive ecosystem and supports a wide range of applications. TensorFlow's flexibility, scalability, and extensive community support make it a top choice for both research and production-level deployments.

- PyTorch

PyTorch, maintained by Facebook's AI Research lab (FAIR), has gained immense popularity due to its simplicity, flexibility, and dynamic computation graph. Its intuitive interface and Pythonic syntax make it an ideal choice for researchers and developers, allowing for seamless experimentation and rapid prototyping.

- Keras

Keras, now an integral part of TensorFlow, provides a high-level neural networks API that emphasizes ease of use and modularity. It allows for quick iteration and prototyping, making it an excellent choice for beginners while also offering advanced features for building complex models.

Types of deep neural networks

- Deep Learning models can automatically learn features from the data, making them well-suited for tasks such as image recognition, speech recognition, and natural language processing.
- The most widely used architectures in deep learning are:
 - Feedforward neural networks,
 - Convolutional neural networks (CNNs), and
 - Recurrent neural networks (RNNs).

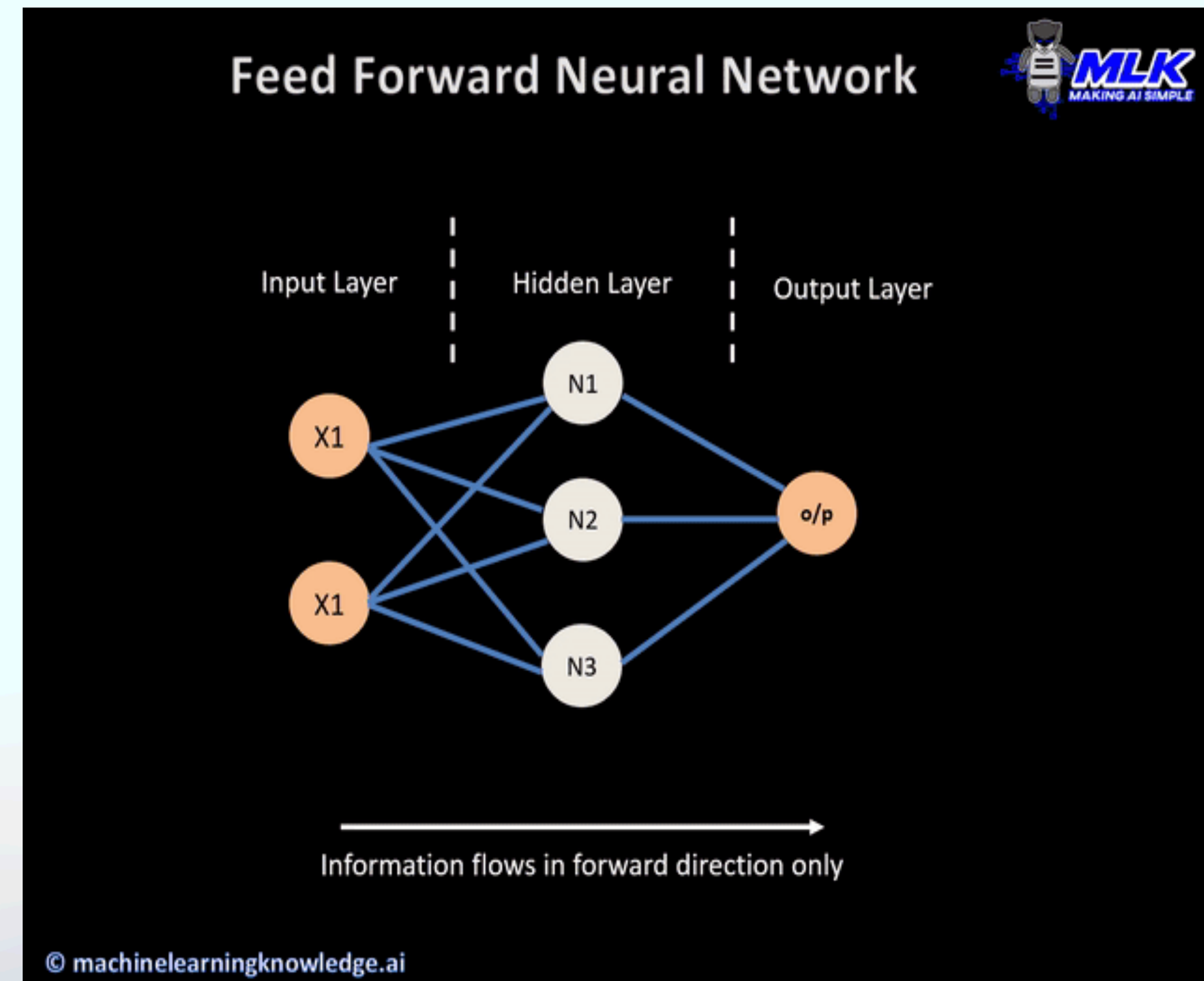
Feedforward Neural Network Architecture

- Input Layer

This layer consists of the input data which is being given to the neural network.

This layer is depicted like neurons only but they are not the actual artificial neuron with computational capabilities that we discussed above.

Each neuron represents a feature of the data. This means if we have a data set with three attributes Age, Salary, City then we will have 3 neurons in the input layer to represent each of them. If we are working with an image of the dimension of 1024×768 pixels then we will have $1024 * 768 = 786432$ neurons in the input layer to represent each of the pixels !!



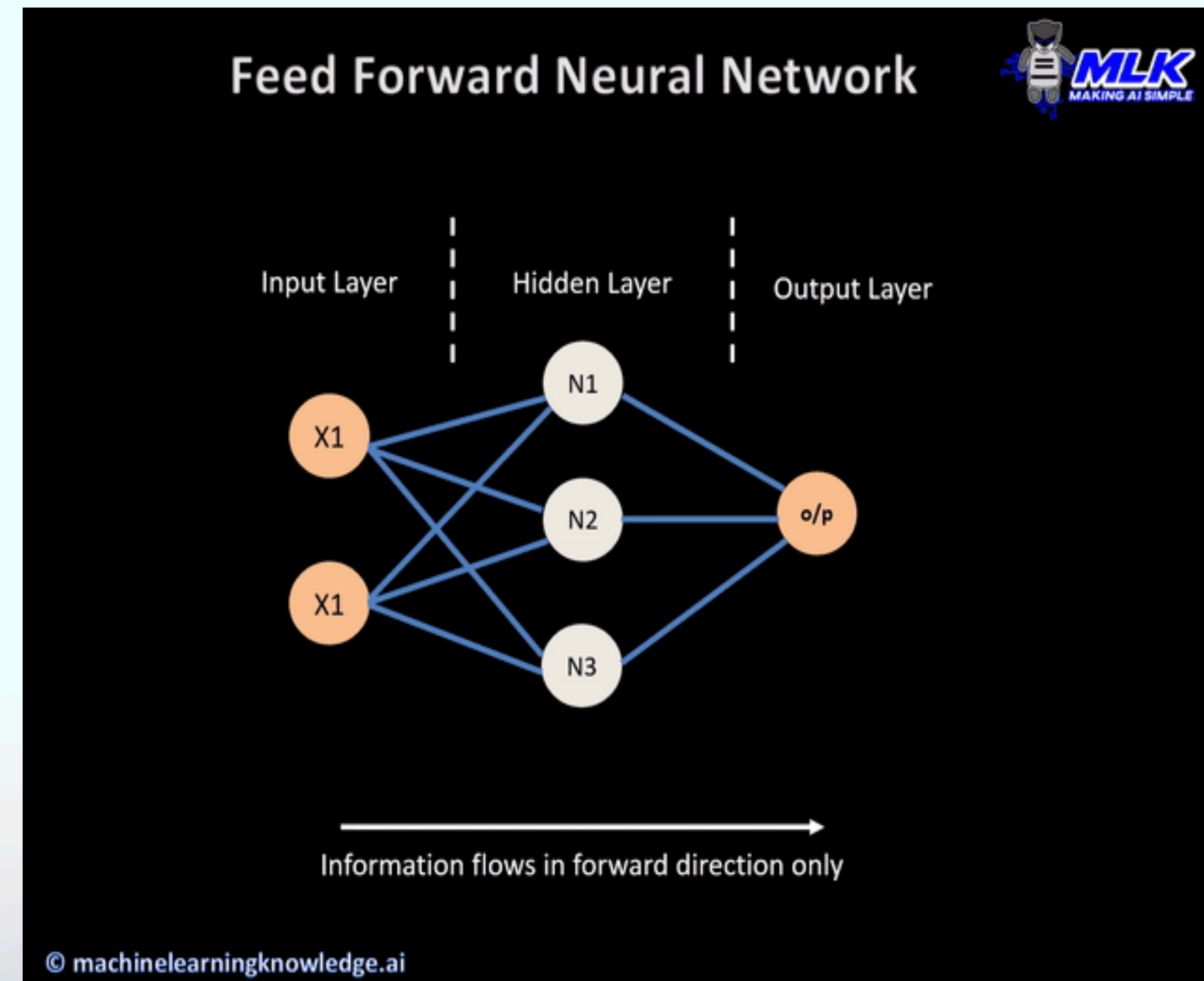
Feedforward Neural Network Architecture

- Input Layer

This layer consists of the input data which is being given to the neural network.

This layer is depicted like neurons only but they are not the actual artificial neuron with computational capabilities that we discussed above.

Each neuron represents a feature of the data. This means if we have a data set with three attributes Age, Salary, City then we will have 3 neurons in the input layer to represent each of them. If we are working with an image of the dimension of 1024×768 pixels then we will have $1024 * 768 = 786432$ neurons in the input layer to represent each of the pixels !!



Feedforward Neural Network Architecture

- Hidden Layer
 - This is the layer that consists of the actual artificial neurons.
 - If the number of hidden layers is one, it is known as a shallow neural network.
 - If the number of hidden layer is more than one then it is known as a deep neural network.
 - In a deep neural network, the output of neurons in one hidden layer is the input to the next hidden layer.
 - In most cases, all the neurons are connected with each other and it is also known as fully connected neural network.
 - In the case of a convolution neural network, however, not all neurons are connected with each other.

Feedforward Neural Network Architecture

- Output Layer

This layer is used to represent the output of the neural network.

The number of output neurons depends on number of output that we are expecting in the problem at hand.

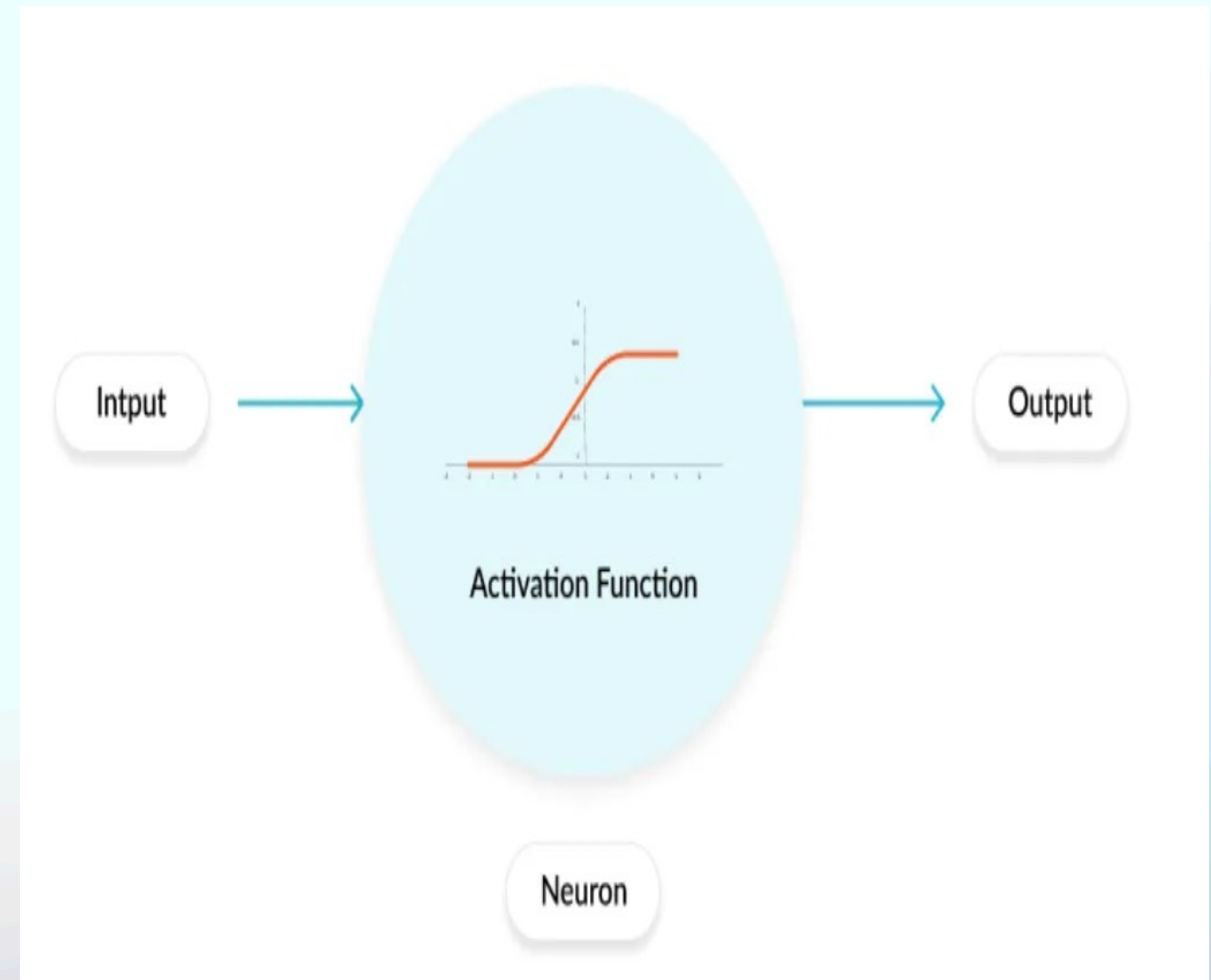
- Weights and Bias

The neurons in the neural network are connected to each other by weights.

Apart from weights, each neuron also has its own bias.

Activation Functions in Neural Network

- An artificial neuron takes the inputs and their respective weights.
- It then applies dot products between input values & its weights and sums them up.
- Finally, it applies the activation function on the above summation and fires the output
- This can be written in a crude way as below –
- $\text{Output} = \text{Activation}(\text{Summation}(\text{Inputs} * \text{Weights} + \text{bias}))$



Activation Functions in Neural Network

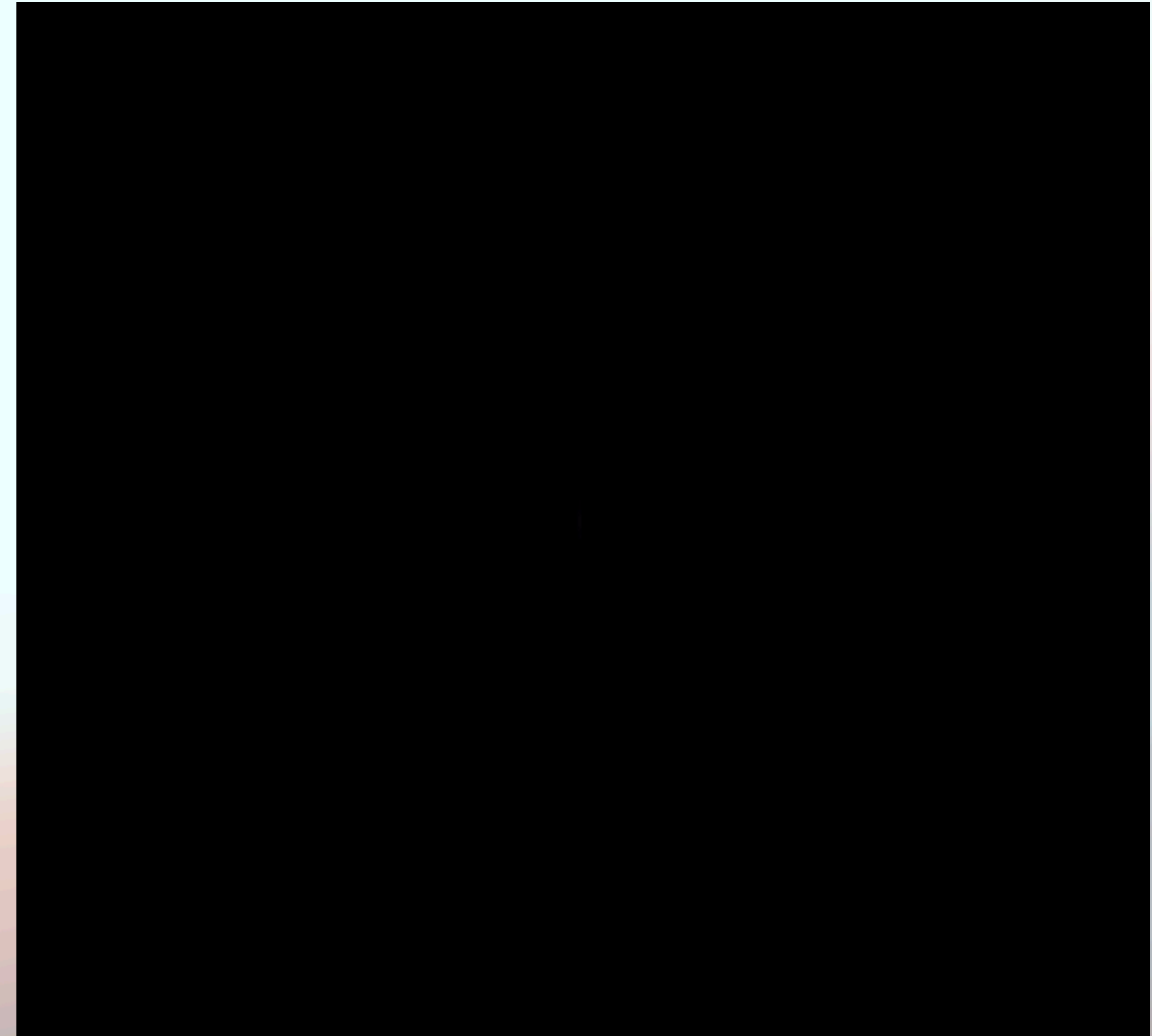
- An artificial neuron without an activation function will just produce the sum of dot products between all inputs and their weights.
- Input : X_0, X_1, X_2 then Output = $W_0X_0 + W_1X_1 + W_2X_2$ [without activation function]
- The linear input has only produced a linear output. No matter how many such artificial neurons stack up in the neural networks, without activation functions the network will only produce a linear output.
- In other words, the neural network would try to map input and output data during the training phase with only linear functions. No matter how hard it tries to learn the relation between input and output data during the training phase at the most neural network can only come up with a linear mapping function.
- **In the real world, hardly any data is so simple that it can be represented with just a linear mapping function.**

Activation Function adds Non-linearity to Neural Network

- So how can neural networks learn the non-linear relationships between data?
 - Using appropriate nonlinear activation functions can help the neural networks understand this nonlinear relationship.
- Given enough training data and training time, the neural network can come up with an approximate function to represent any complex relationship within data.
- Many activation functions can be used in neural networks. Before we take a look at the popular ones let us understand what is an ideal activation function.
 - **Non-Linearity** – Activation function should be able to add nonlinearity in neural network especially in the neurons of hidden layers.
 - **Differentiable** – Activation function should be differentiable. During the training phase, the neural network learns by back-propagating error from the output layer to hidden layers. The backpropagation algorithm uses derivative of activation function of neurons in hidden layers, to adjust their weights so that error in the next training epoch can be reduced.

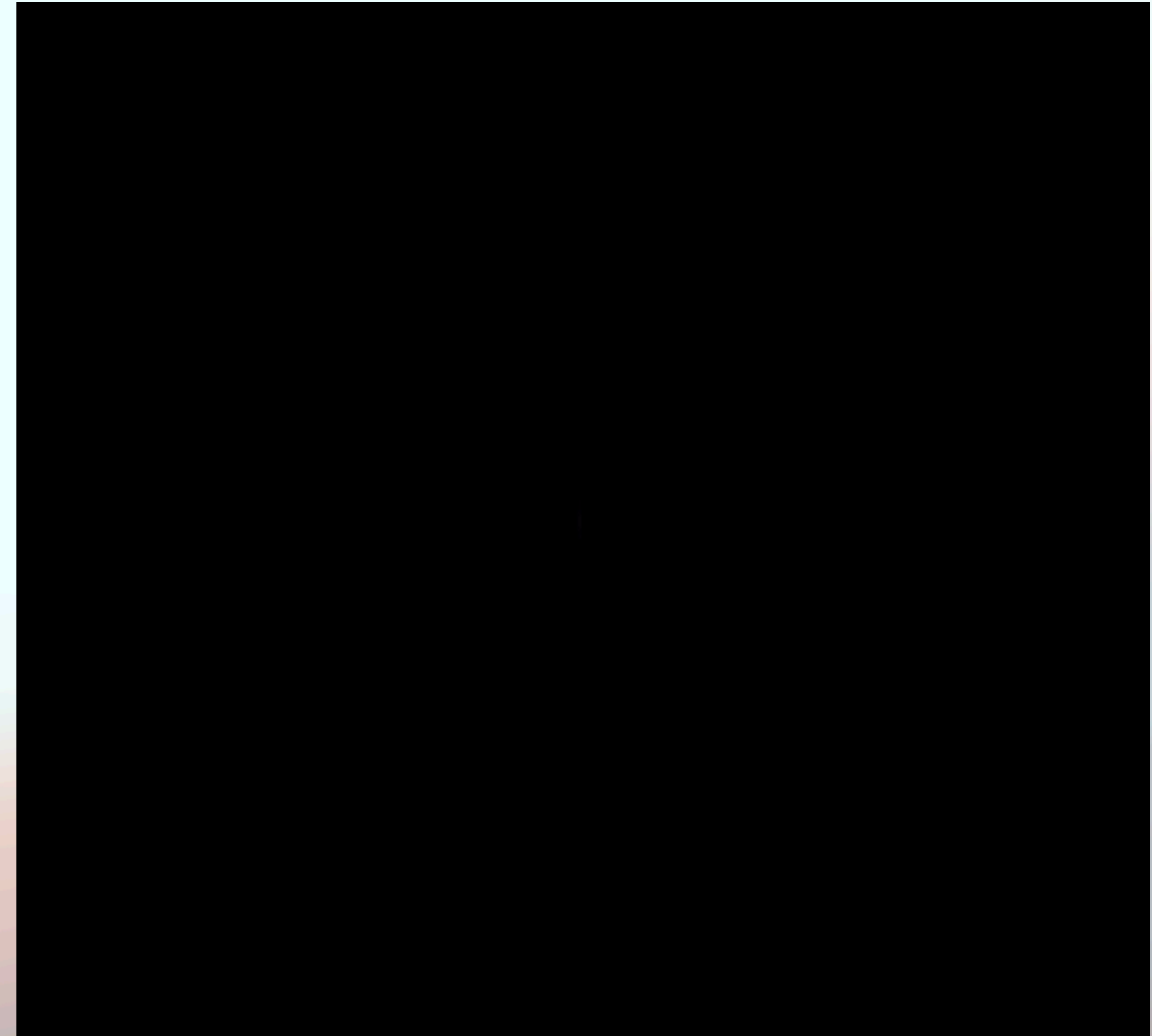
Step Activation Function

- This is a very intuitive activation function which only enables or disables artificial neuron. It can only produce two values 0 or 1. If it is 0 it means neuron has not fired, if it is 1 it means neuron has fired.
- It produces output 1 only when the sum of dot products of inputs & weights is more than a threshold value, else it will be 0.
- The step function is not smooth and has a sudden change at the threshold value.
- It is non-differentiable and it can neither add nonlinearity in the neural network This means it does not comply with good characteristics of activation functions that we discussed above.
- Step function has historical importance as it was used in McCulloch Pitts Neuron & Perceptron that were the earliest models of an artificial neuron.



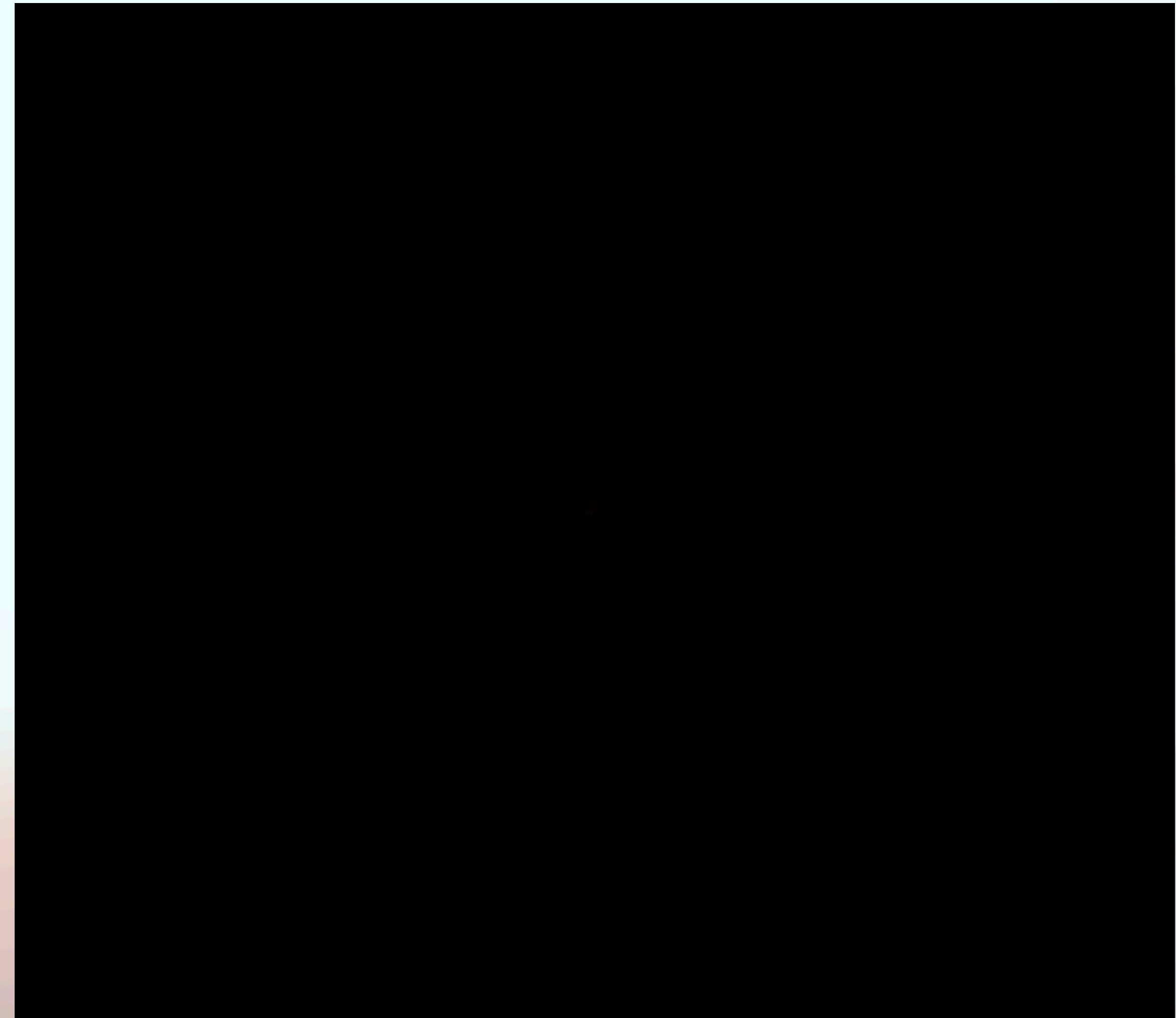
Step Activation Function

- This is a very intuitive activation function which only enables or disables artificial neuron. It can only produce two values 0 or 1. If it is 0 it means neuron has not fired, if it is 1 it means neuron has fired.
- It produces output 1 only when the sum of dot products of inputs & weights is more than a threshold value, else it will be 0.
- The step function is not smooth and has a sudden change at the threshold value.
- It is non-differentiable and it can neither add nonlinearity in the neural network This means it does not comply with good characteristics of activation functions that we discussed above.
- Step function has historical importance as it was used in McCulloch Pitts Neuron & Perceptron that were the earliest models of an artificial neuron.



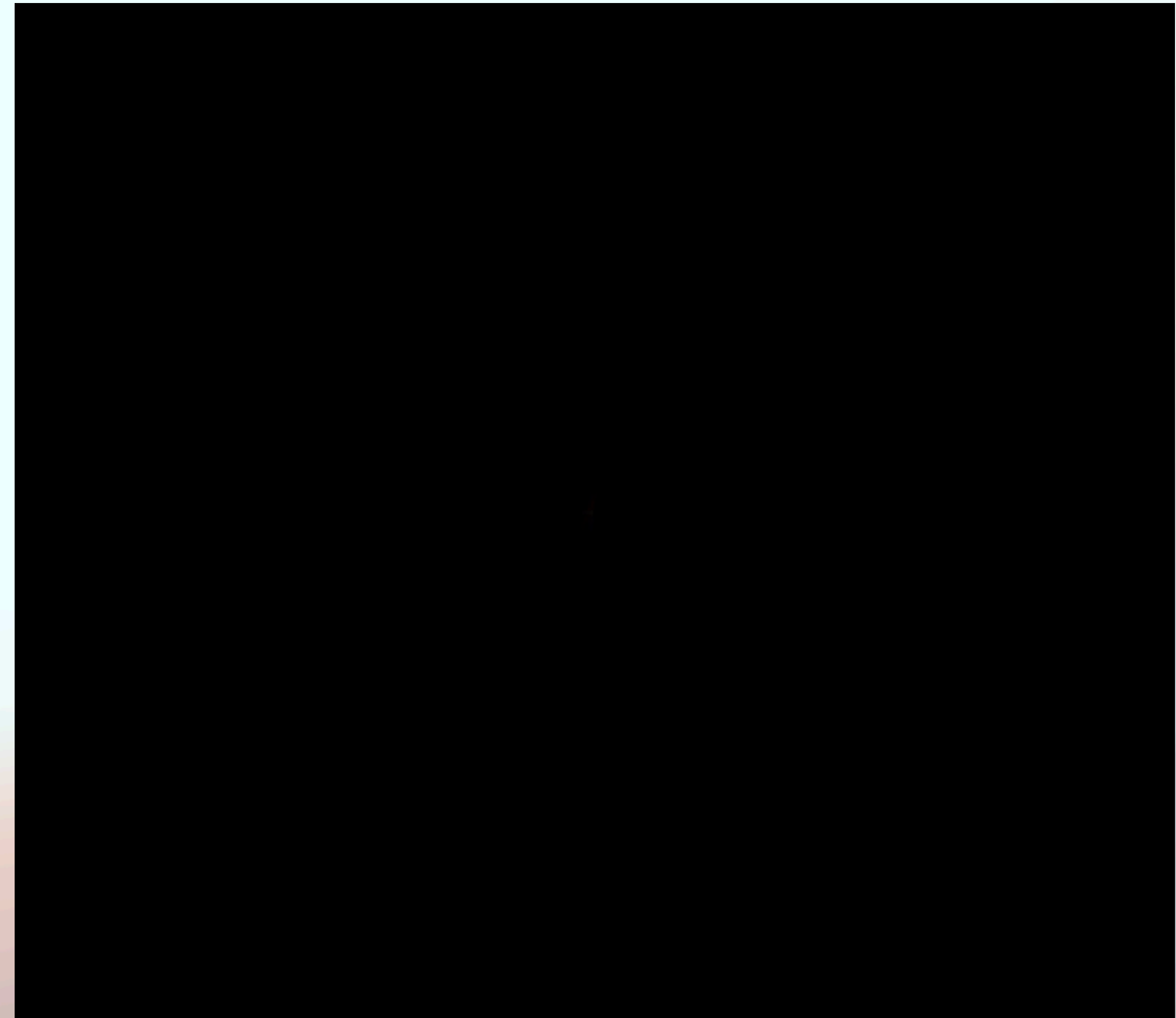
Sigmoid Function

- Unlike the step function, the sigmoid function is S-Shaped and is smooth, there is no sudden spike.
- The output range is between 0 and 1 which is usually interpreted as probability if used in the output layer.
- This function is both non-linear and is differentiable which is a great thing as we discussed in the earlier section.
- This activation is a good candidate for use in neurons of hidden layers of neural networks
- But sigmoid function suffers from phenomena of vanishing gradient which results in slow and poor learning of neural networks during the training phase.



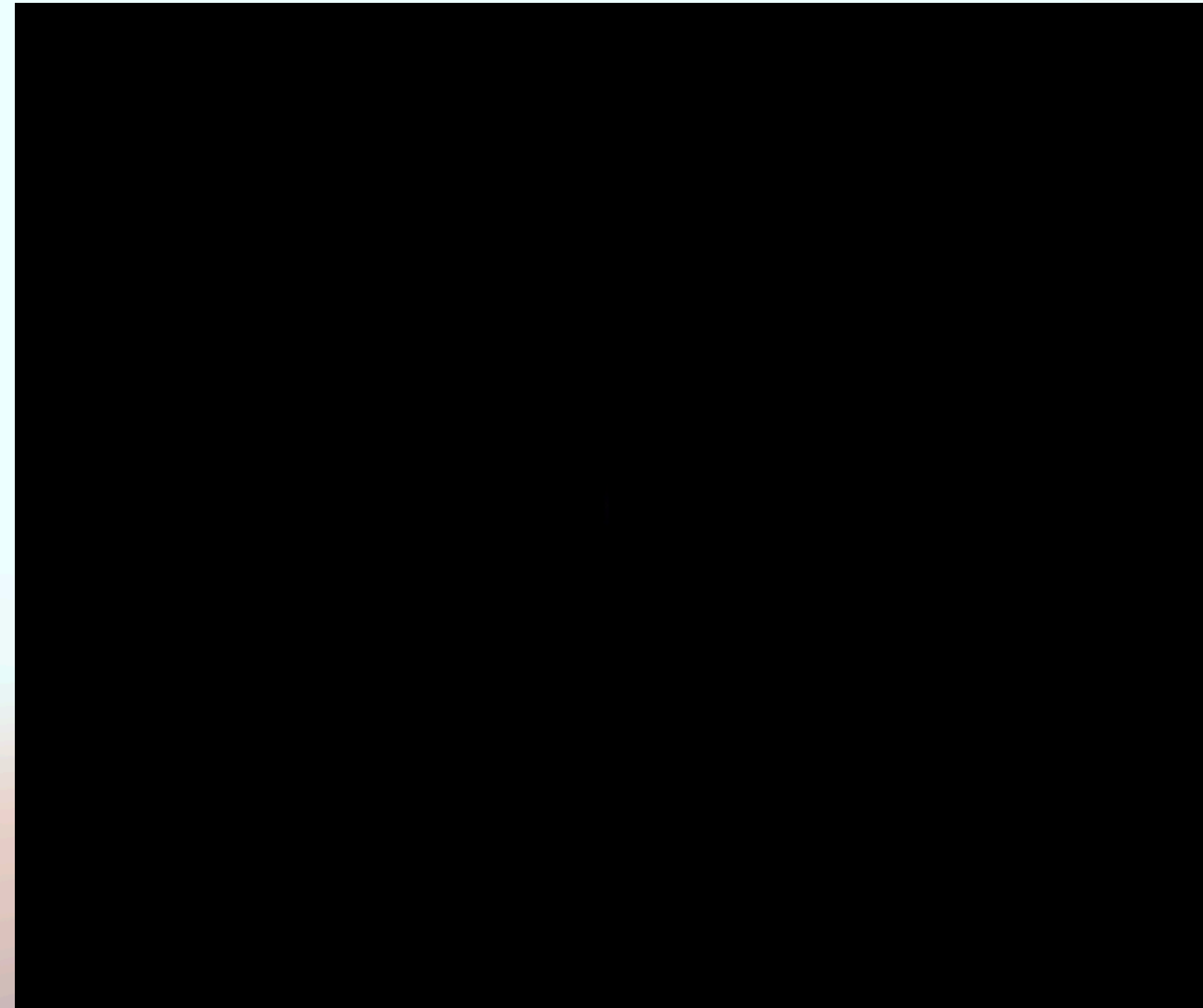
Sigmoid Function

- Unlike the step function, the sigmoid function is S-Shaped and is smooth, there is no sudden spike.
- The output range is between 0 and 1 which is usually interpreted as probability if used in the output layer.
- This function is both non-linear and is differentiable which is a great thing as we discussed in the earlier section.
- This activation is a good candidate for use in neurons of hidden layers of neural networks
- But sigmoid function suffers from phenomena of vanishing gradient which results in slow and poor learning of neural networks during the training phase.



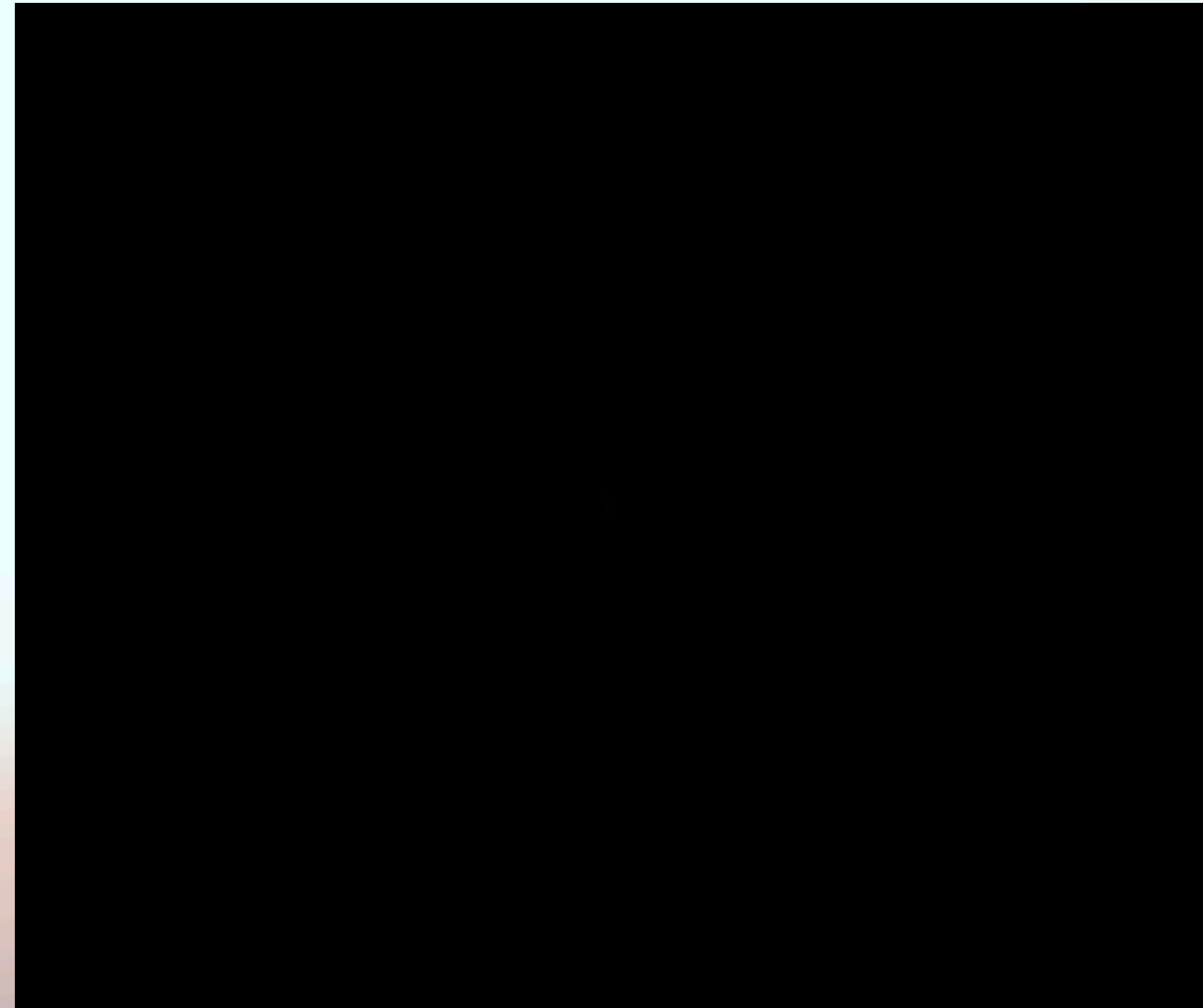
Tanh Function

- Sigmoid function varies from 0 to 1, but there might be scenarios where we would like to introduce a negative sign to the output of the artificial neuron.
- This is where Tanh (hyperbolic tangent function) becomes useful. Tanh function is similar to sigmoid function but its output varies from +1 to -1 and is centered at zero.
- This is non-linear and differentiable but still suffers from phenomena of vanishing gradient similar to a sigmoid function.
- Hence Tanh is also not used much in the neural networks since recent years unless there is a pressing need to use.



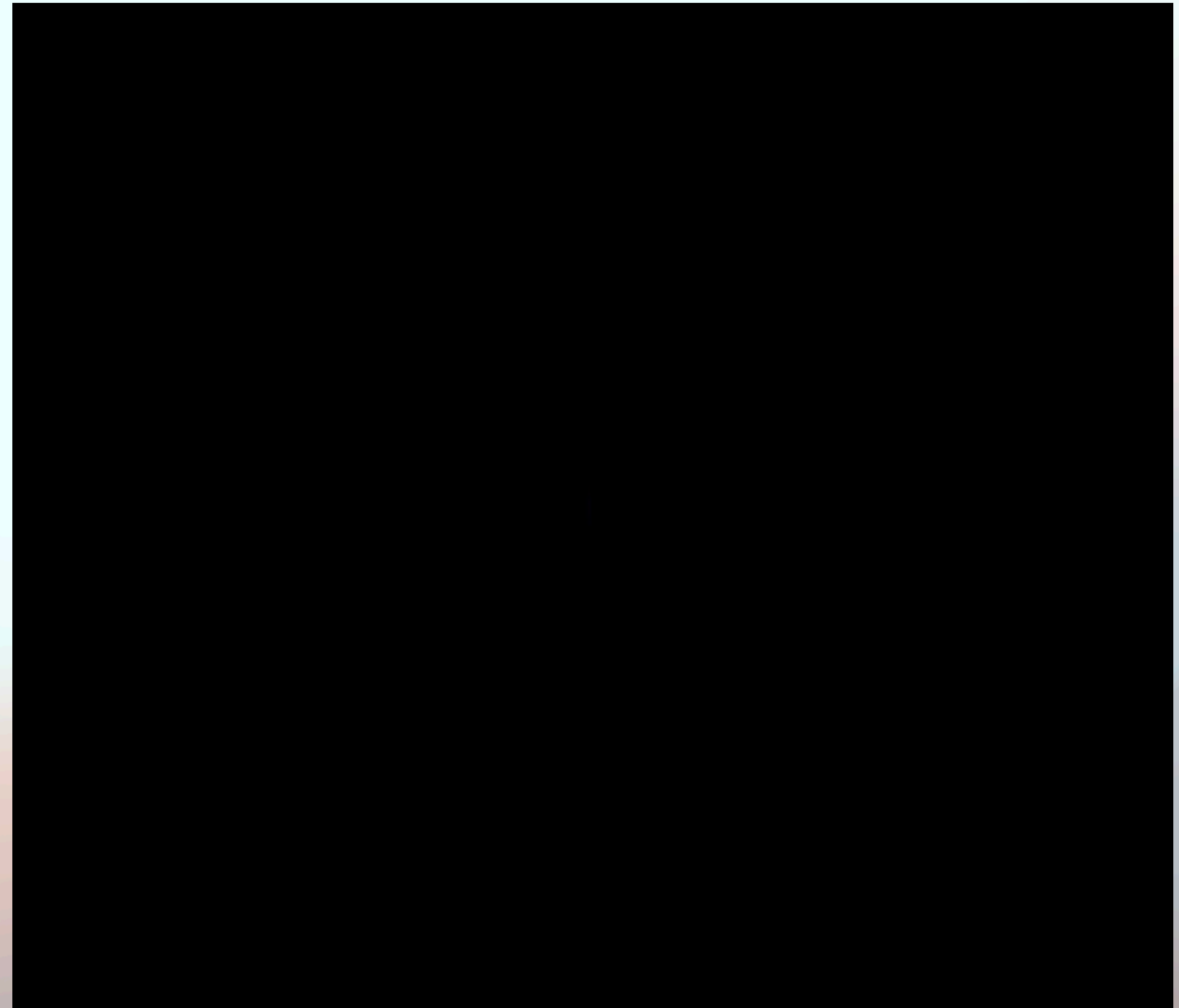
Tanh Function

- Sigmoid function varies from 0 to 1, but there might be scenarios where we would like to introduce a negative sign to the output of the artificial neuron.
- This is where Tanh (hyperbolic tangent function) becomes useful. Tanh function is similar to sigmoid function but its output varies from +1 to -1 and is centered at zero.
- This is non-linear and differentiable but still suffers from phenomena of vanishing gradient similar to a sigmoid function.
- Hence Tanh is also not used much in the neural networks since recent years unless there is a pressing need to use.



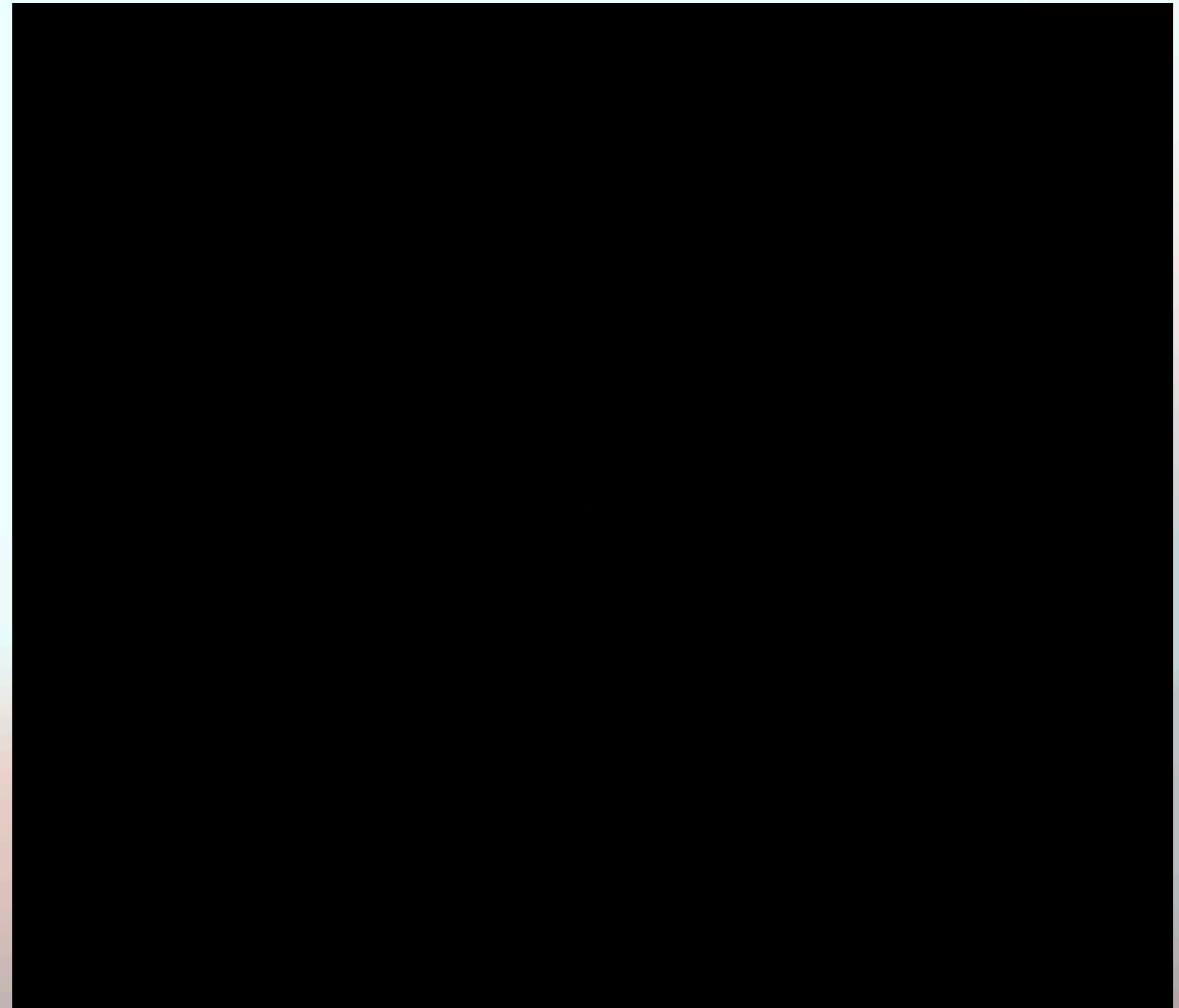
ReLU (Rectified Linear Unit) Function

- ReLU might look deceptive at first look but is actually non-linear and differentiable.
- What makes it stand out is that it does not suffer from phenomena of vanishing gradient. This is really great as this means the neural network will not learn slowly and poorly.
- ReLU, however, suffers from another problem. For any negative value of X , the derivative is 0 and this hinders the learning process so the particular neuron effectively dies out. There might be a scenario where most of the neurons in the neural network start dying and this can become another issue.
- In spite of dying neurons possibility, ReLU is computationally fast and shows great results when used in hidden layers.



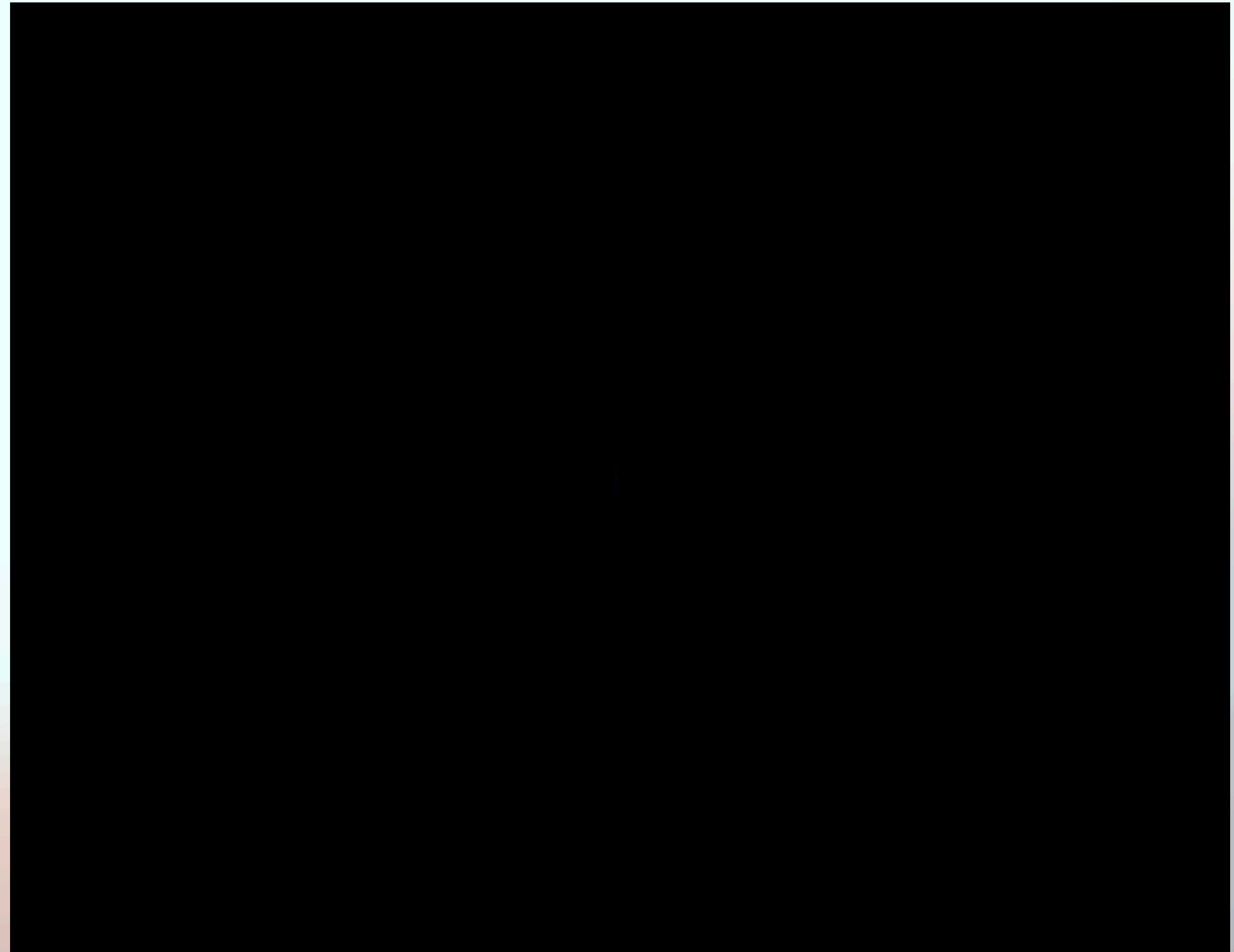
ReLU (Rectified Linear Unit) Function

- ReLU might look deceptive at first look but is actually non-linear and differentiable.
- What makes it stand out is that it does not suffer from phenomena of vanishing gradient. This is really great as this means the neural network will not learn slowly and poorly.
- ReLU, however, suffers from another problem. For any negative value of X , the derivative is 0 and this hinders the learning process so the particular neuron effectively dies out. There might be a scenario where most of the neurons in the neural network start dying and this can become another issue.
- In spite of dying neurons possibility, ReLU is computationally fast and shows great results when used in hidden layers.



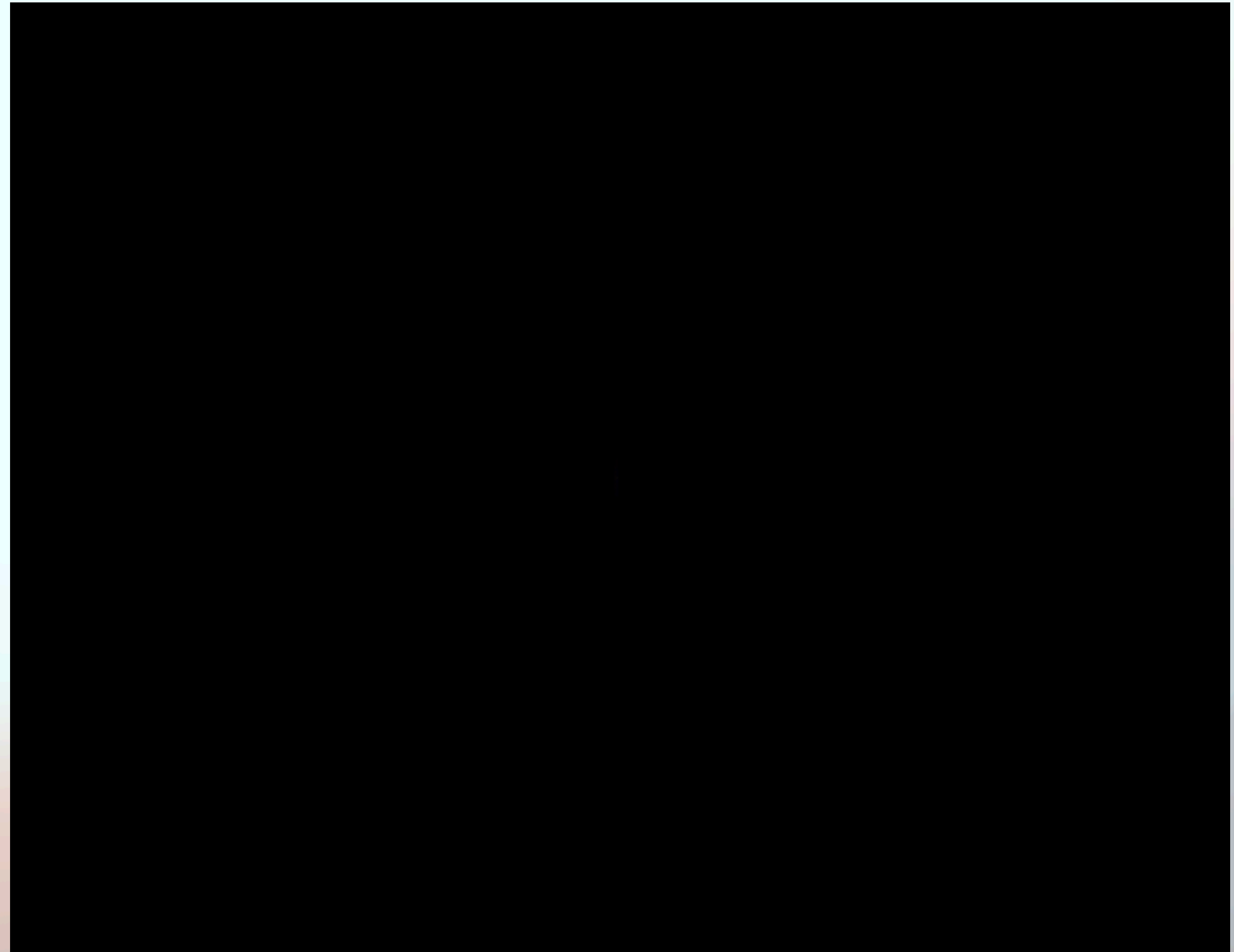
Leaky ReLU

- The name suggests that Leaky ReLU or LReLU has something to do with ReLU that we discussed above. And yes, it does!
- Leaky ReLU tries to address the problem of neurons dying out in the case of the ReLU function.
- As you can see, in the negative part, there is a small tiny bit of extension in the negative area instead of a zero. This means even the derivative would be a very small fraction and not zero. This ensures that the learning of the neuron does not stop during backpropagation.



Leaky ReLU

- The name suggests that Leaky ReLU or LReLU has something to do with ReLU that we discussed above. And yes, it does!
- Leaky ReLU tries to address the problem of neurons dying out in the case of the ReLU function.
- As you can see, in the negative part, there is a small tiny bit of extension in the negative area instead of a zero. This means even the derivative would be a very small fraction and not zero. This ensures that the learning of the neuron does not stop during backpropagation.



Softmax Function

- In the case of binary classification, the Sigmoid function can be used in the output layer to transform incoming signals in the probability range.
- What if it is a case of multi-class classification?
Well, we have Softmax function to help us.
- Softmax function comes from the family of Sigmoid functions only.
- Softmax function squashes the incoming signals for multiple classes in a probability distribution. The sum of this probability distribution would obviously be 1.

Softmax Function

- In the case of binary classification, the Sigmoid function can be used in the output layer to transform incoming signals in the probability range.
- What if it is a case of multi-class classification?
Well, we have Softmax function to help us.
- Softmax function comes from the family of Sigmoid functions only.
- Softmax function squashes the incoming signals for multiple classes in a probability distribution. The sum of this probability distribution would obviously be 1.

Summary

<i>Function Type</i>	<i>Equation</i>	<i>Derivative</i>
Linear	$f(x) = ax + c$	$f'(x) = a$
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x) (1 - f(x))$
TanH	$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ReLU	$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parametric ReLU	$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
ELU	$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

Softmax Function

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

-
- z_i values are the input vector elements and can take any real value.
- K The number of classes in the multi-class classifier

Softmax Function - example

- Given the logits

$$\vec{z} = \begin{bmatrix} 8 \\ 5 \\ 0 \end{bmatrix}$$

-
- Compute softmax output
-

Softmax Function

$$\vec{z} = \begin{bmatrix} 8 \\ 5 \\ 0 \end{bmatrix}$$

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

$$e^{z_1} = e^8 = 2981.0$$

$$e^{z_2} = e^5 = 148.4$$

$$e^{z_3} = e^0 = 1.0$$

$$\sum_{j=1}^K e^{z_j} = e^{z_1} + e^{z_2} + e^{z_3} = 2981.0 + 148.4 + 1.0 = 3130.4$$

$$\sigma(\vec{z})_1 = \frac{2981.0}{3130.4} = 0.9523$$

$$\sigma(\vec{z})_2 = \frac{148.4}{3130.4} = 0.0474$$

$$\sigma(\vec{z})_3 = \frac{1.0}{3130.4} = 0.0003$$

Loss Function

- Loss Functions, also known as error functions, are used for computing the error with the aim that the model should minimize it during training.
- Loss Functions also help in finding out the slope i.e. gradient w.r.t. weights used in the model and then these weights are updated after each epoch with the help of backpropagation.
- During the training process, deep learning algorithms employ optimization techniques such as gradient descent to minimize the loss function. By iteratively adjusting model parameters based on the gradients of the loss function, the algorithm aims to converge to the optimal solution, resulting in a model that accurately captures the underlying patterns in the data.
- Understanding the role of loss functions is essential for effectively training and optimizing machine learning models for various tasks and applications.
- There are many cost functions in deep learning and each has its own use cases depending on whether it is a regression problem or classification problem.
-

Loss Functions in Deep Learning

Intuition about Cost Functions



Model



$$\text{Cost Function} = y - y'$$

Cost functions in machine learning are functions that help machine learning model to determine the offset of their predictions with respect to actual results during the training phase.

Loss Functions in Deep Learning

Intuition about Cost Functions



Model



$$\text{Cost Function} = y - y'$$

Cost functions in machine learning are functions that help machine learning model to determine the offset of their predictions with respect to actual results during the training phase.

Regression Loss Functions

- Mean Absolute Error (MAE) Loss
- The Mean Absolute Error (MAE) Loss is another commonly used loss function for regression. It calculates the average of the absolute differences between the predicted values and the actual values.

$$\mathbf{MAE} = \left(\frac{1}{n}\right) \sum_{i=1}^n |y_i - \hat{y}_i|$$

MAE=Mean Absolute Error

n=total number of data points

y=observed values

$\hat{y}$ =predicted values

•

Regression Loss Functions

- Mean Squared Error/Squared loss/ L2 loss
- The Mean Squared Error (MSE) Loss is one of the most widely used loss functions for regression tasks. It calculates the average of the squared differences between the predicted values and the actual values.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

MSE = mean squared error

n = number of data points

Y_i = observed values

$\hat{Y}_i$ = predicted values

Regression Loss Functions

- Huber Loss
- Huber Loss combines the advantages of MSE and MAE. It is less sensitive to outliers than MSE and differentiable everywhere, unlike MAE.

$$\text{Huber} = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} (y_i - \hat{y}_i)^2 \quad |y_i - \hat{y}_i| \leq \delta$$
$$\text{Huber} = \frac{1}{n} \sum_{i=1}^n \delta \left(|y_i - \hat{y}_i| - \frac{1}{2} \delta \right) \quad |y_i - \hat{y}_i| > \delta$$

- This error calculates the absolute difference between the predicted and actual value for each instance in the dataset. If this difference is below a certain threshold (delta), the function calculates the squared difference instead.
- Huber loss combines the useful parts of MSE and MAE with the addition of a delta hyperparameter. The value of the parameter determines which data points are outliers.

Classification Losses

- 1. Binary Cross Entropy/log loss Functions in machine learning models
- It is used in binary classification problems like two classes. example a person has covid or not
- Binary cross entropy compares each of the predicted probabilities to the actual class output which can be either 0 or 1. It then calculates the score that penalizes the probabilities based on the distance from the expected value. That means how close or far from the actual value.

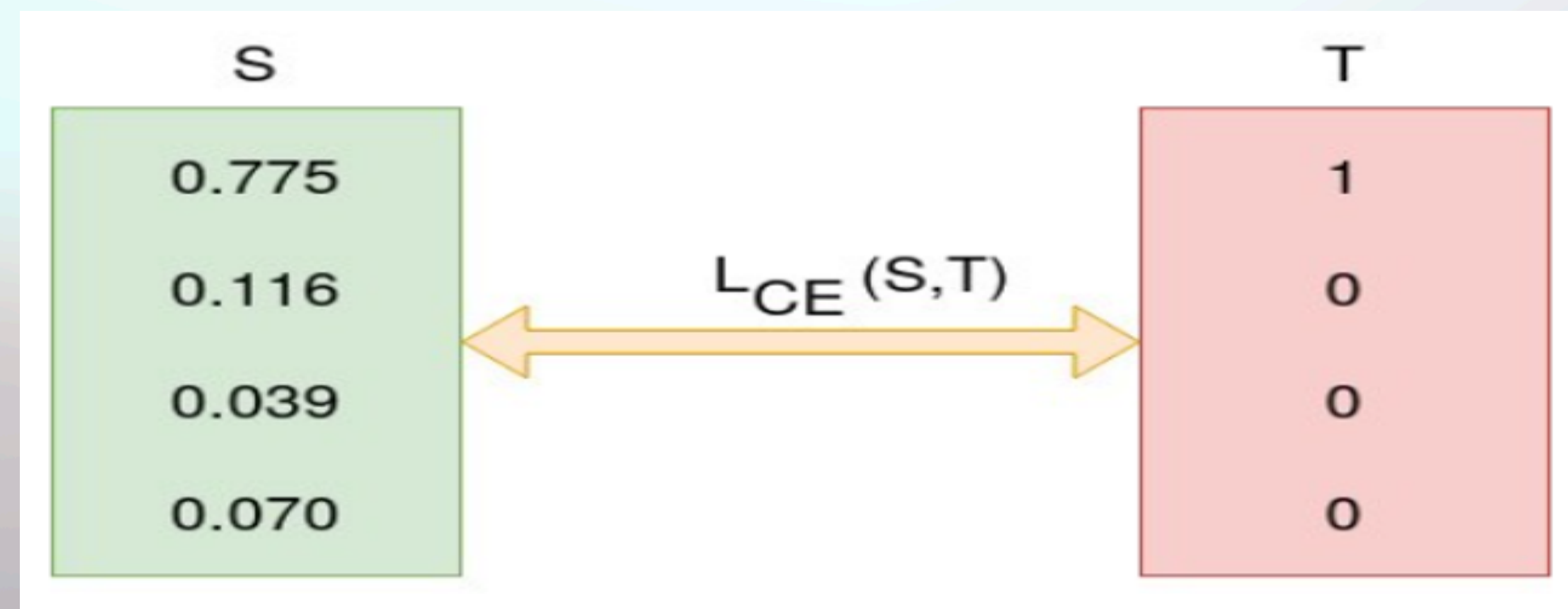
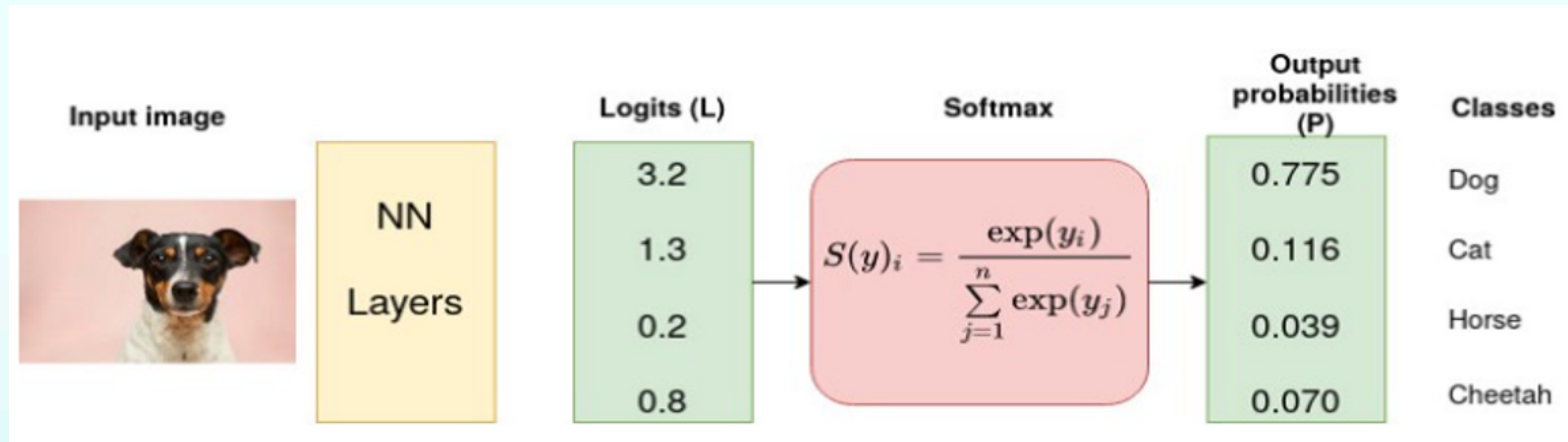
$$-(y \log(p) + (1 - y) \log(1 - p))$$

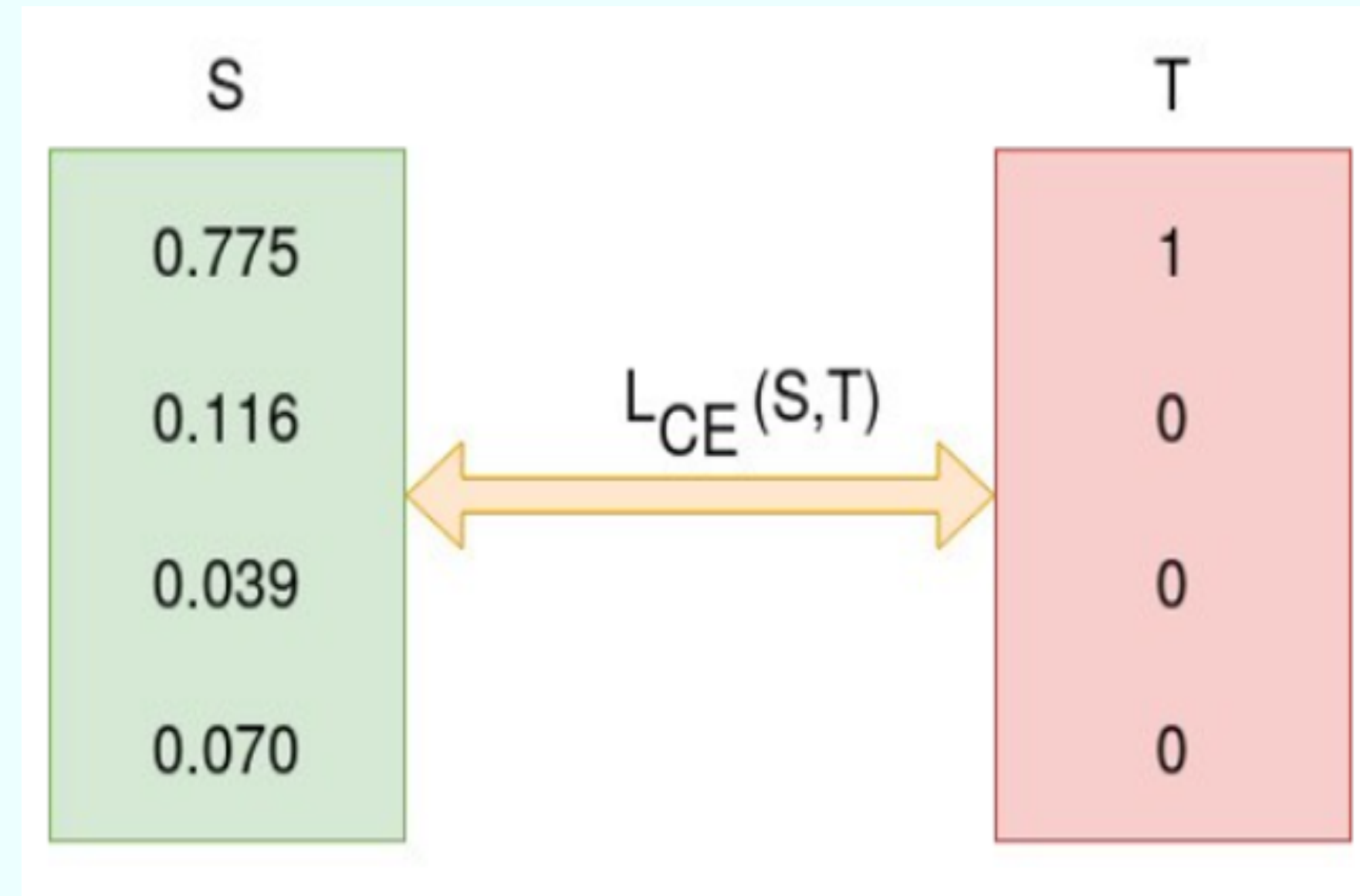
- •log - the natural log; •y – truth label/binary indicator (0 or 1); p - predicted probability

Cross-Entropy Loss Function

$$L_{\text{CE}} = - \sum_{i=1}^n t_i \log(p_i), \text{ for } n \text{ classes,}$$

where t_i is the truth label and p_i is the Softmax probability for the i^{th} class.





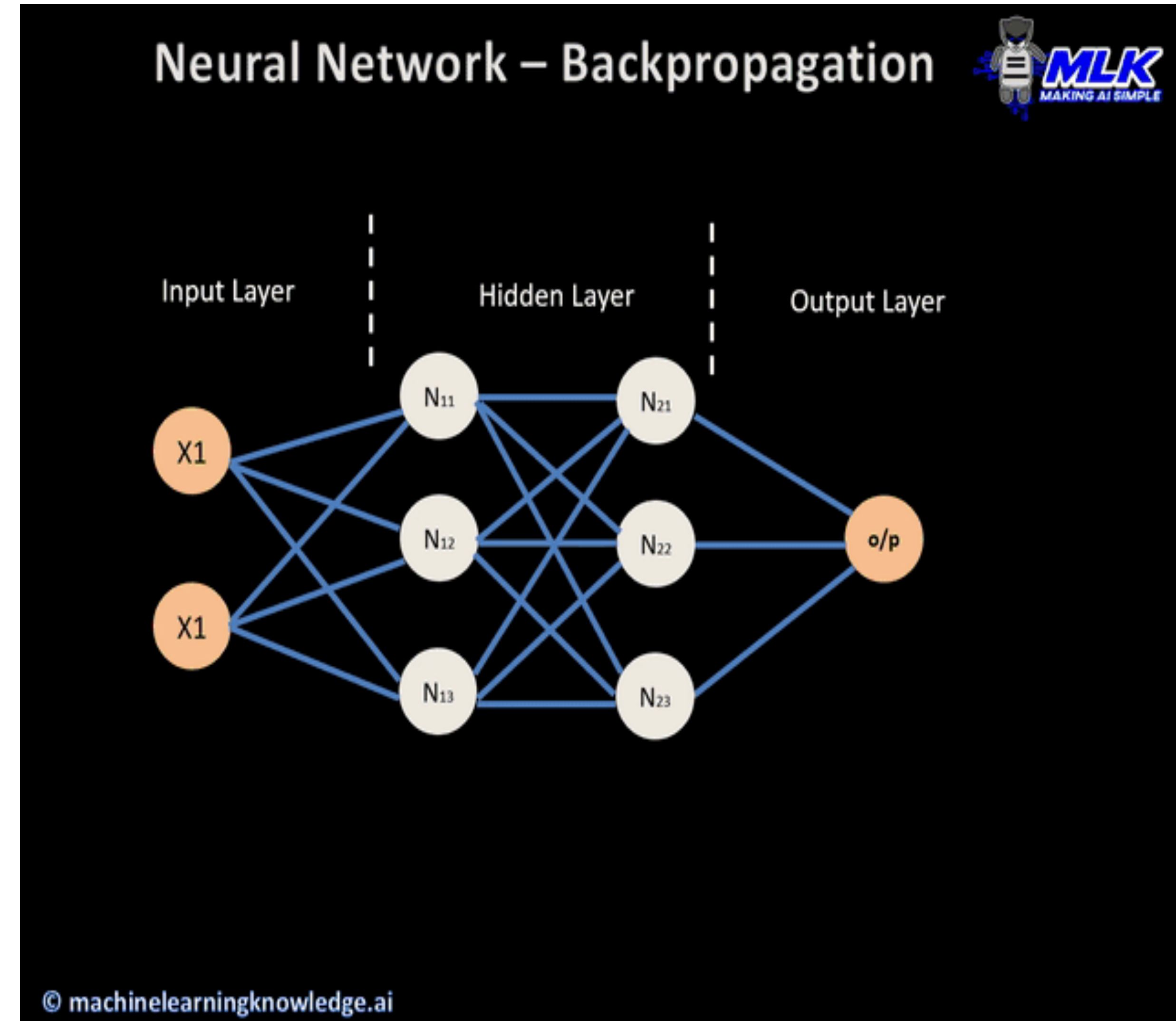
$$\begin{aligned} L_{CE} &= - \sum_{i=1} T_i \log(S_i) \\ &= - [1 \log_2(0.775) + 0 \log_2(0.126) + 0 \log_2(0.039) + 0 \log_2(0.070)] \\ &= - \log_2(0.775) \\ &= 0.3677 \end{aligned}$$

Backpropagation

- Backpropagation is used only to pass back the output error in hidden layers for adjusting their weights during training. But the word “feedforward” emphasizes that the network should not pass back any information while making the actual calculation of the output.
- If it passes back the information during calculation itself then it is termed as a recurrent neural network and not feed forward neural network.

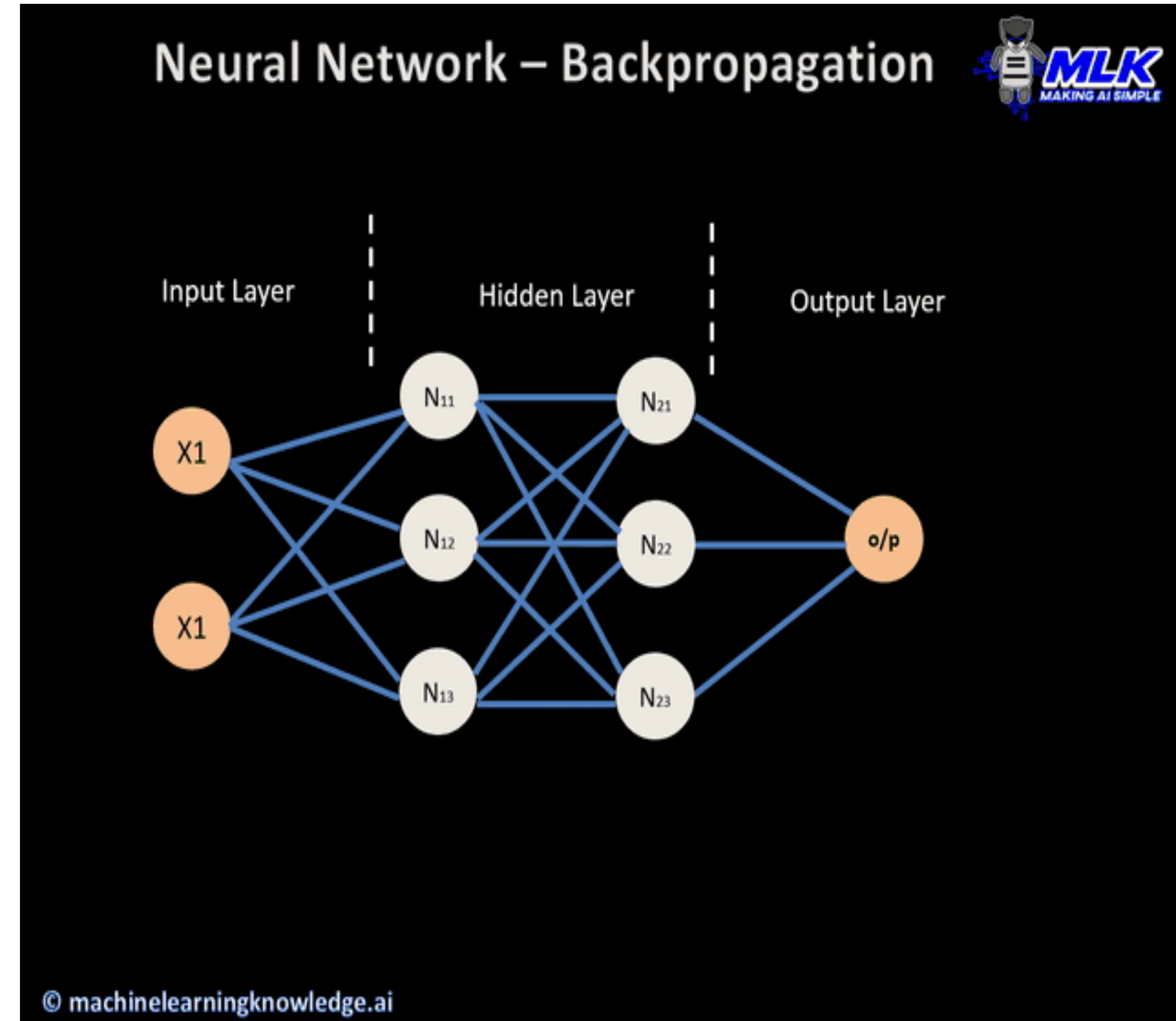
Backpropagation

- During the training phase, the neural network is initialized with random weight values.
- Training data is fed to the network and the network then calculates the output. This is known as a forward pass.
- The calculated output is then compared with the actual output with the help of loss/cost function and the error is determined.
- Now comes the backpropagation part where the network determines how to adjust all the weights in its network so that the loss can be minimized.
- This weight adjustment starts happening from the rear end of the network. The error is propagated in the backward direction to the front layers till the end and the neurons across the network start adjusting their weights. Hence the name backpropagation.

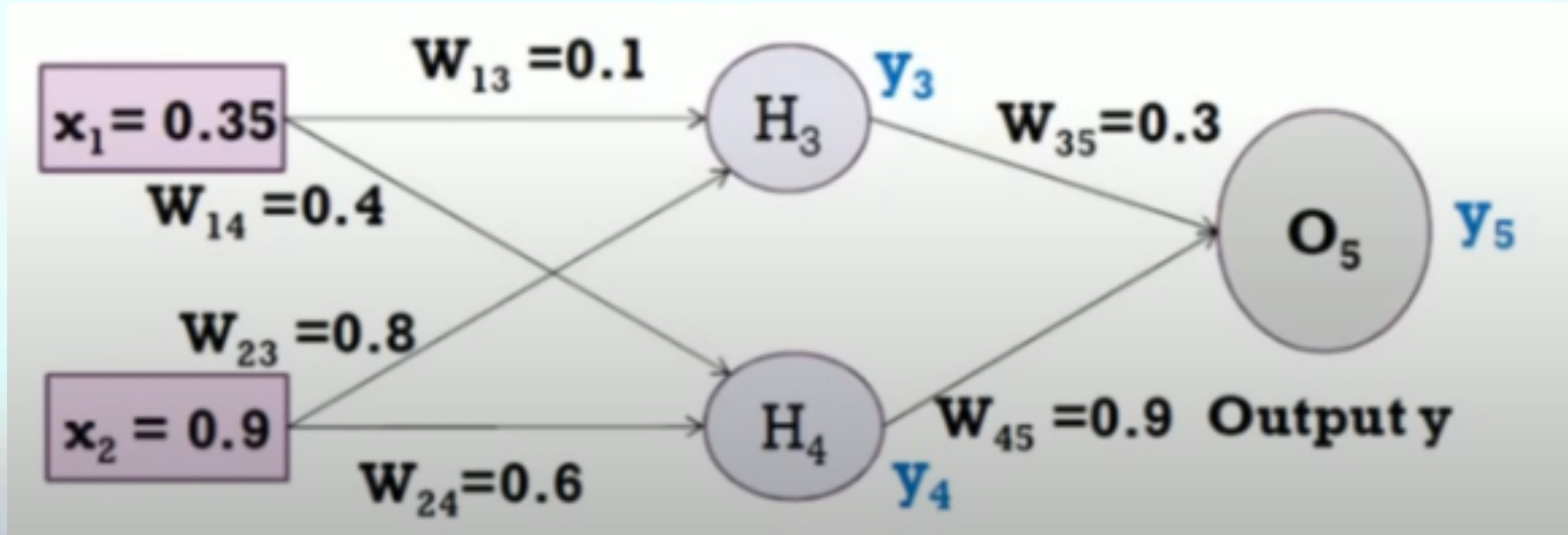


Backpropagation

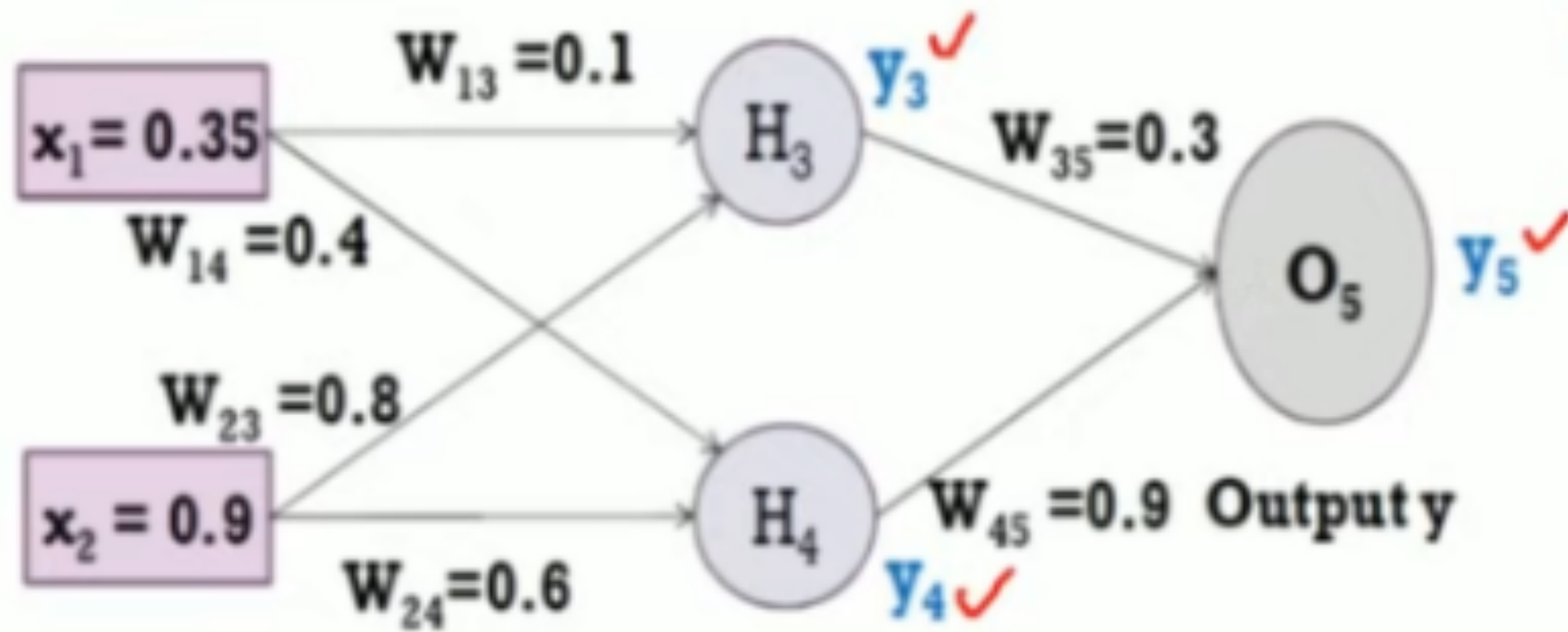
- During the training phase, the neural network is initialized with random weight values.
- Training data is fed to the network and the network then calculates the output. This is known as a forward pass.
- The calculated output is then compared with the actual output with the help of loss/cost function and the error is determined.
- Now comes the backpropagation part where the network determines how to adjust all the weights in its network so that the loss can be minimized.
- This weight adjustment starts happening from the rear end of the network. The error is propagated in the backward direction to the front layers till the end and the neurons across the network start adjusting their weights. Hence the name backpropagation.



Backpropagation - Example



Backpropagation - Example



- Forward Pass: Compute output for y_3 , y_4 and y_5 .

$$a_j = \sum_i (w_{i,j} * x_i) \quad y_j = F(a_j) = \frac{1}{1 + e^{-a_j}}$$

$$\begin{aligned} a_1 &= (w_{13} * x_1) + (w_{23} * x_2) \checkmark \\ &= (0.1 * 0.35) + (0.8 * 0.9) = 0.755 \\ y_3 &= f(a_1) = 1 / (1 + e^{-0.755}) = 0.68 \end{aligned}$$

$$\begin{aligned} a_2 &= (w_{14} * x_1) + (w_{24} * x_2) \checkmark \\ &= (0.4 * 0.35) + (0.6 * 0.9) = 0.68 \\ y_4 &= f(a_2) = 1 / (1 + e^{-0.68}) = 0.6637 \end{aligned}$$

$$\begin{aligned} a_3 &= (w_{35} * y_3) + (w_{45} * y_4) \checkmark \\ &= (0.3 * 0.68) + (0.9 * 0.6637) = 0.801 \checkmark \\ y_5 &= f(a_3) = 1 / (1 + e^{-0.801}) = 0.69 \text{ (Network Output)} \end{aligned}$$

Backpropagation - Example

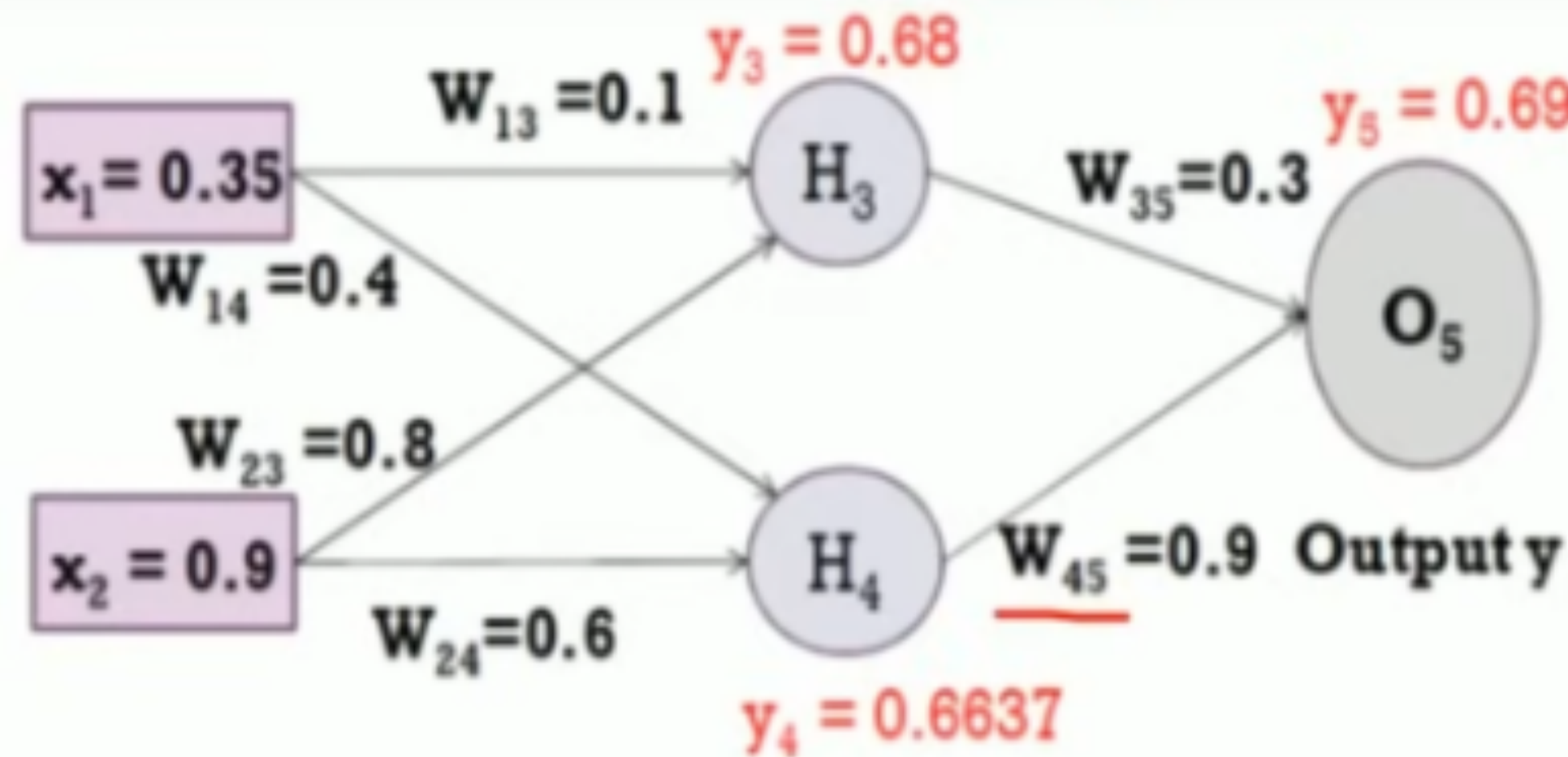
- Each weight changed by:

$$\Delta w_{ji} = \eta \delta_j o_i$$

$$\delta_j = o_j(1 - o_j)(t_j - o_j) \quad \text{if } j \text{ is an output unit}$$

$$\delta_j = o_j(1 - o_j) \sum_k \delta_k w_{kj} \quad \text{if } j \text{ is a hidden unit}$$

Backpropagation - Example



• Backward Pass: Compute δ_3 , δ_4 and δ_5 .

For output unit:

$$\delta_5 = y(1-y) (y_{\text{target}} - y) \\ = 0.69 * (1 - 0.69) * (0.5 - 0.69) = -0.0406$$

For hidden unit:

$$\delta_3 = y_3(1-y_3) w_{35} * \delta_5 \\ = 0.68 * (1 - 0.68) * (0.3 * -0.0406) = -0.00265$$

$$\delta_4 = y_4(1-y_4) w_{45} * \delta_5 \\ = 0.6637 * (1 - 0.6637) * (0.9 * -0.0406) = -0.0082$$

Compute new weights

$$\Delta w_{ji} = \eta \delta_j o_i$$

$$\Delta w_{45} = \eta \delta_5 y_4 = 1 * -0.0406 * 0.6637 = -0.0269 \\ w_{45}(\text{new}) = \Delta w_{45} + w_{45}(\text{old}) = -0.0269 + (0.9) = 0.8731$$

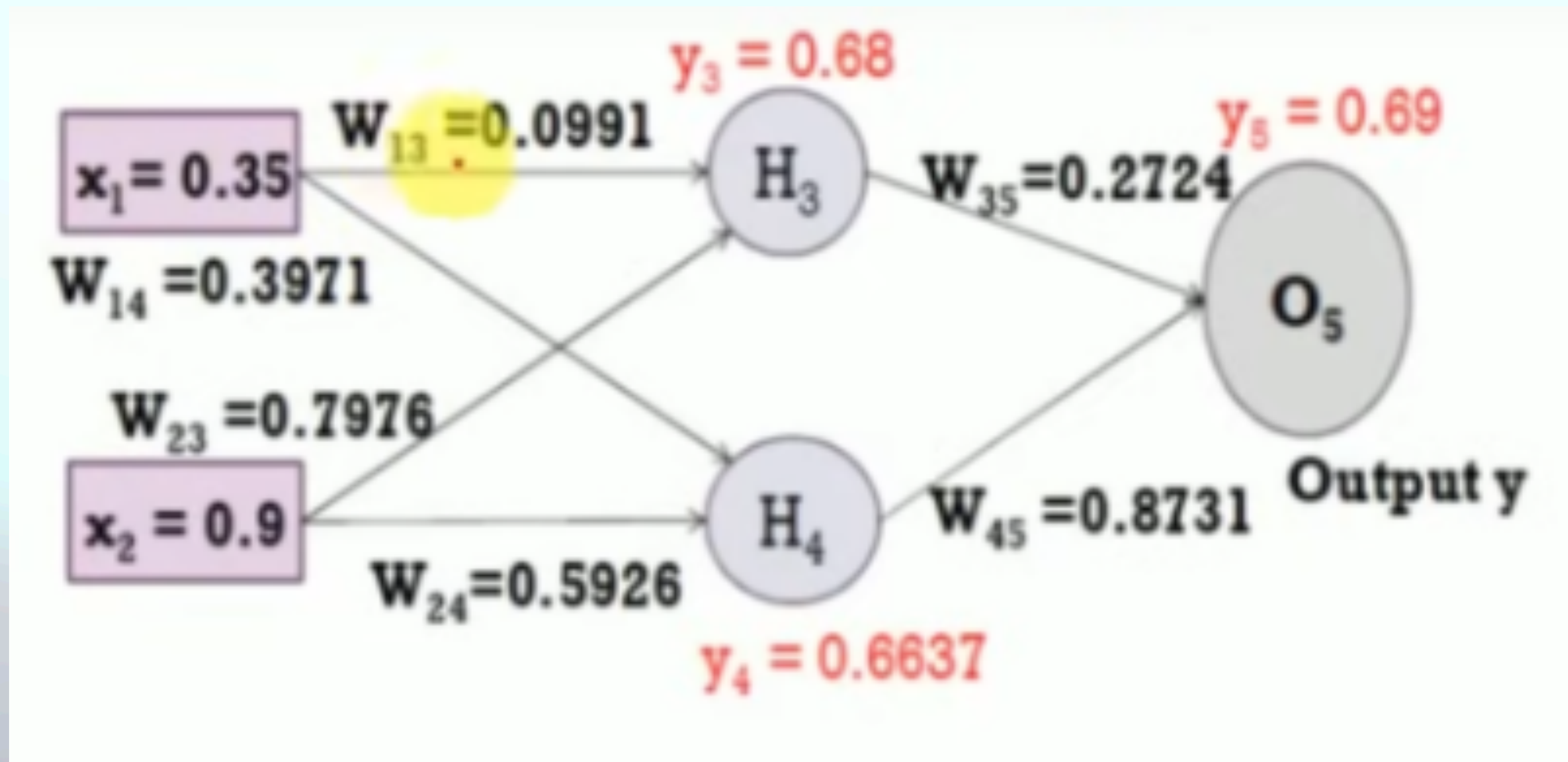
$$\Delta w_{14} = \eta \delta_4 x_1 = 1 * -0.0082 * 0.35 = -0.00287 \\ w_{14}(\text{new}) = \Delta w_{14} + w_{14}(\text{old}) = -0.00287 + 0.4 = 0.3971$$

Backpropagation - Example

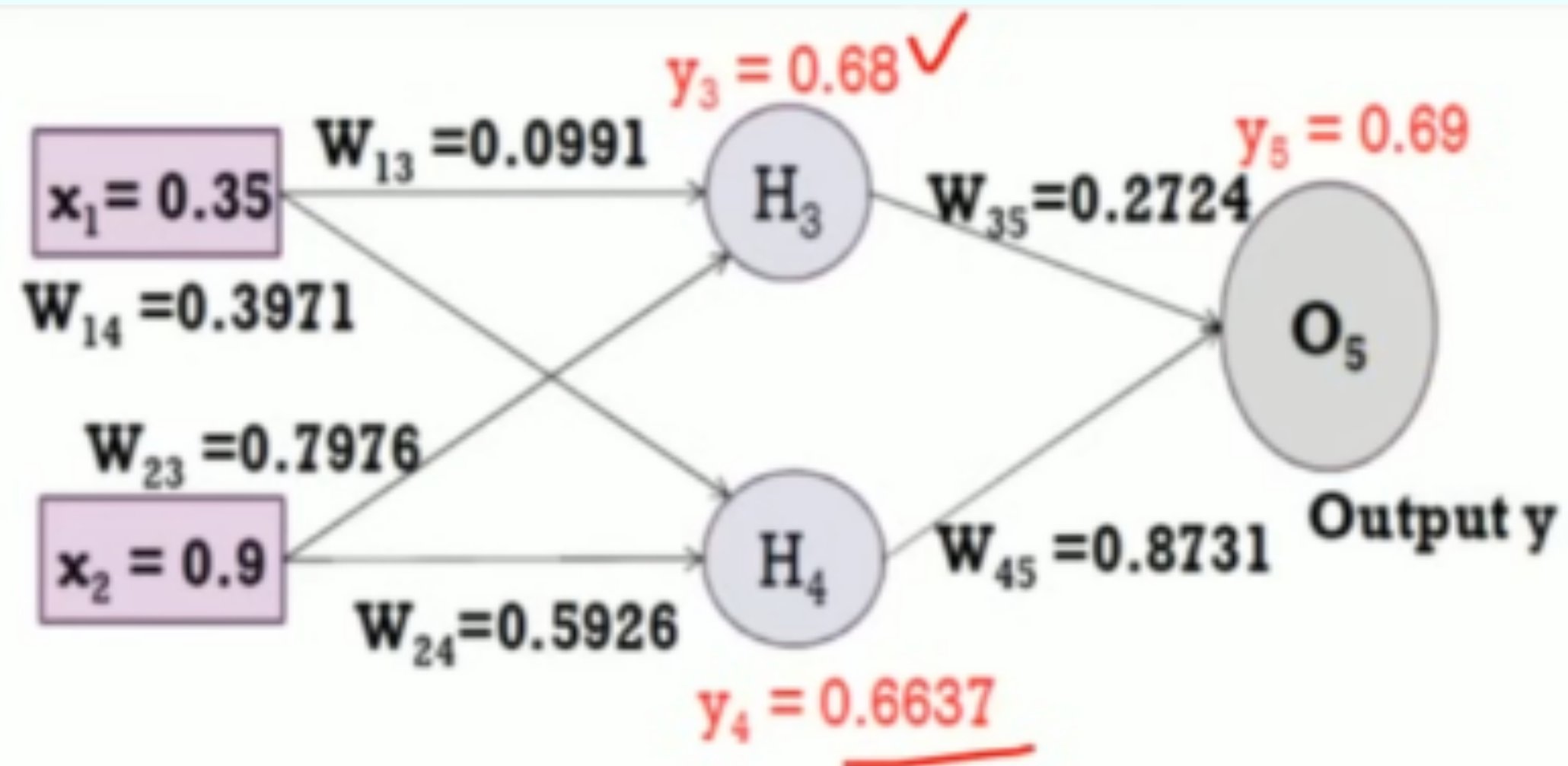
- Similarly, update all other weights

i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
1	3	0.1	-0.00265	0.35	1	0.0991
2	3	0.8	-0.00265	0.9	1	0.7976
1	4	0.4	-0.0082	0.35	1	0.3971
2	4	0.6	-0.0082	0.9	1	0.5926
3	5	0.3	-0.0406	0.68	1	0.2724
4	5	0.9	-0.0406	0.6637	1	0.8731

Backpropagation - Example



Backpropagation - Example



- Forward Pass: Compute output for y_3 , y_4 and y_5 .

$$a_j = \sum_i (w_{i,j} * x_i) \quad y_j = F(a_j) = \frac{1}{1 + e^{-a_j}}$$

$$a_1 = (w_{13} * x_1) + (w_{23} * x_2) = (0.0991 * 0.35) + (0.7976 * 0.9) = 0.7525$$

$$y_3 = f(a_1) = 1 / (1 + e^{-0.7525}) = \underline{0.6797}$$

$$a_2 = (w_{14} * x_1) + (w_{24} * x_2) = (0.3971 * 0.35) + (0.5926 * 0.9) = 0.6723$$

$$y_4 = f(a_2) = 1 / (1 + e^{-0.6723}) = \underline{0.6620}$$

$$a_3 = (w_{35} * y_3) + (w_{45} * y_4) = (0.2724 * 0.6797) + (0.8731 * 0.6620) = 0.7631$$

$$y_5 = f(a_3) = 1 / (1 + e^{-0.7631}) = \underline{0.6820} \text{ (Network Output)}$$

Optimizers for training Deep Learning Models

- While training any Deep learning (or machine Learning) model, the objective is to achieve minimum loss value. The optimization algorithm will try to find the local/global minima for the loss function for the given training data.
- Gradient Descent Algorithm
- Stochastic Gradient Descent
- Momentum
- Adagrad
- RmsProp
- Adadelta
- Adam

Optimizers for training Deep Learning Models



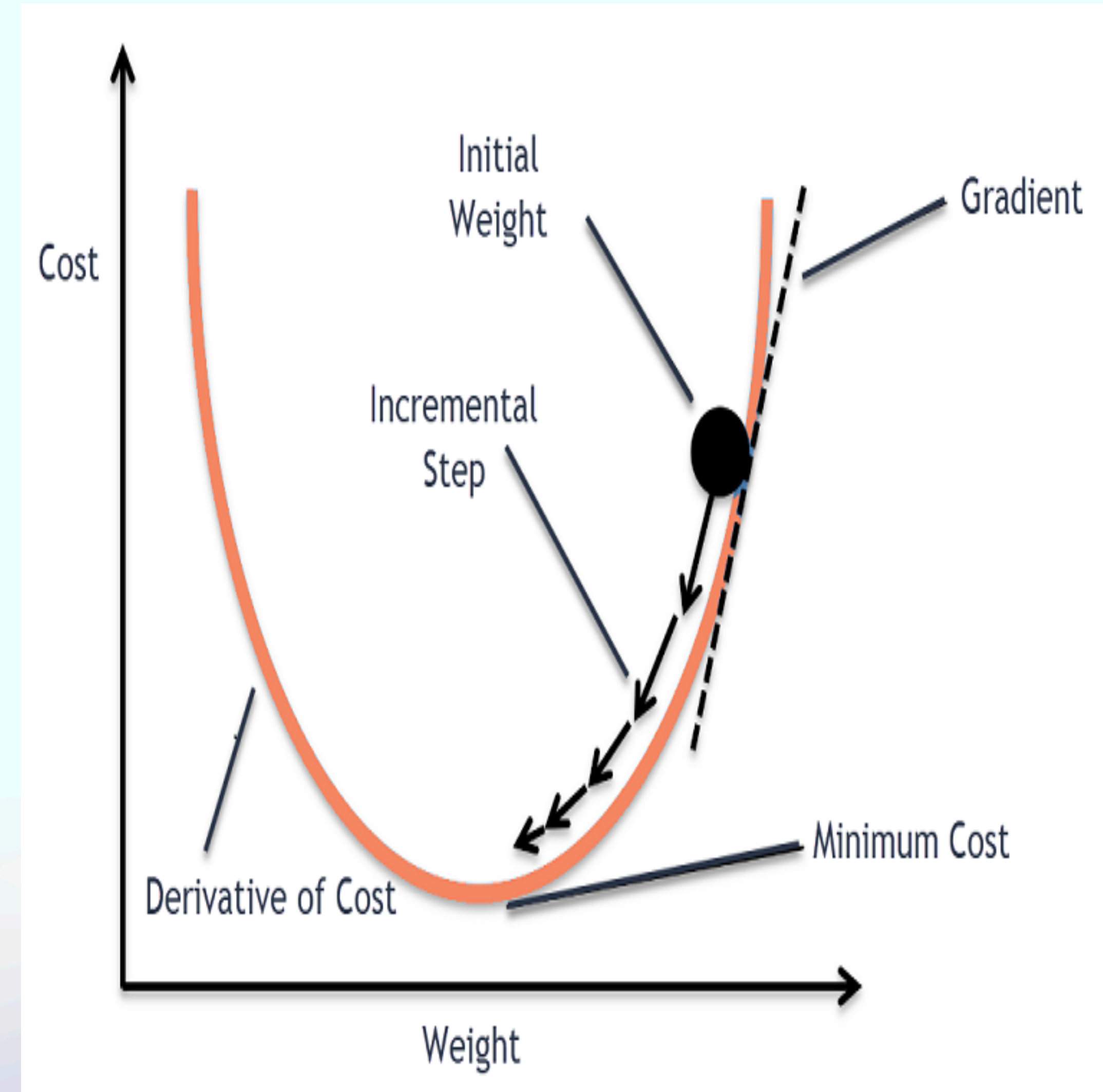
- While training any Deep learning (or machine Learning) model, the objective is to achieve minimum loss value. The optimization algorithm will try to find the local/global minima for the loss function for the given training data.
 - Optimizers utilize gradients computed by backpropagation to determine the direction and magnitude of parameter updates, aiming to navigate the model's high-dimensional parameter space efficiently.
- Gradient Descent Algorithm
 - Stochastic Gradient Descent
 - Momentum
 - Adagrad
 - RmsProp
 - Adadelta
 - Adam

Gradient Descent Algorithm

- Gradient Descent is a popular algorithm used to find the local minima of any given differentiable function.
- The basic intuition behind the working of gradient descent algorithms is, at each point in time, It tries to find the best direction to move and takes a small step such that the overall loss value decreases.
- To find the best direction at a given point, it calculates the gradient of the loss function at that point.

- Update rule:
$$w_{new} = w - \eta \cdot \frac{\partial L}{\partial w}$$

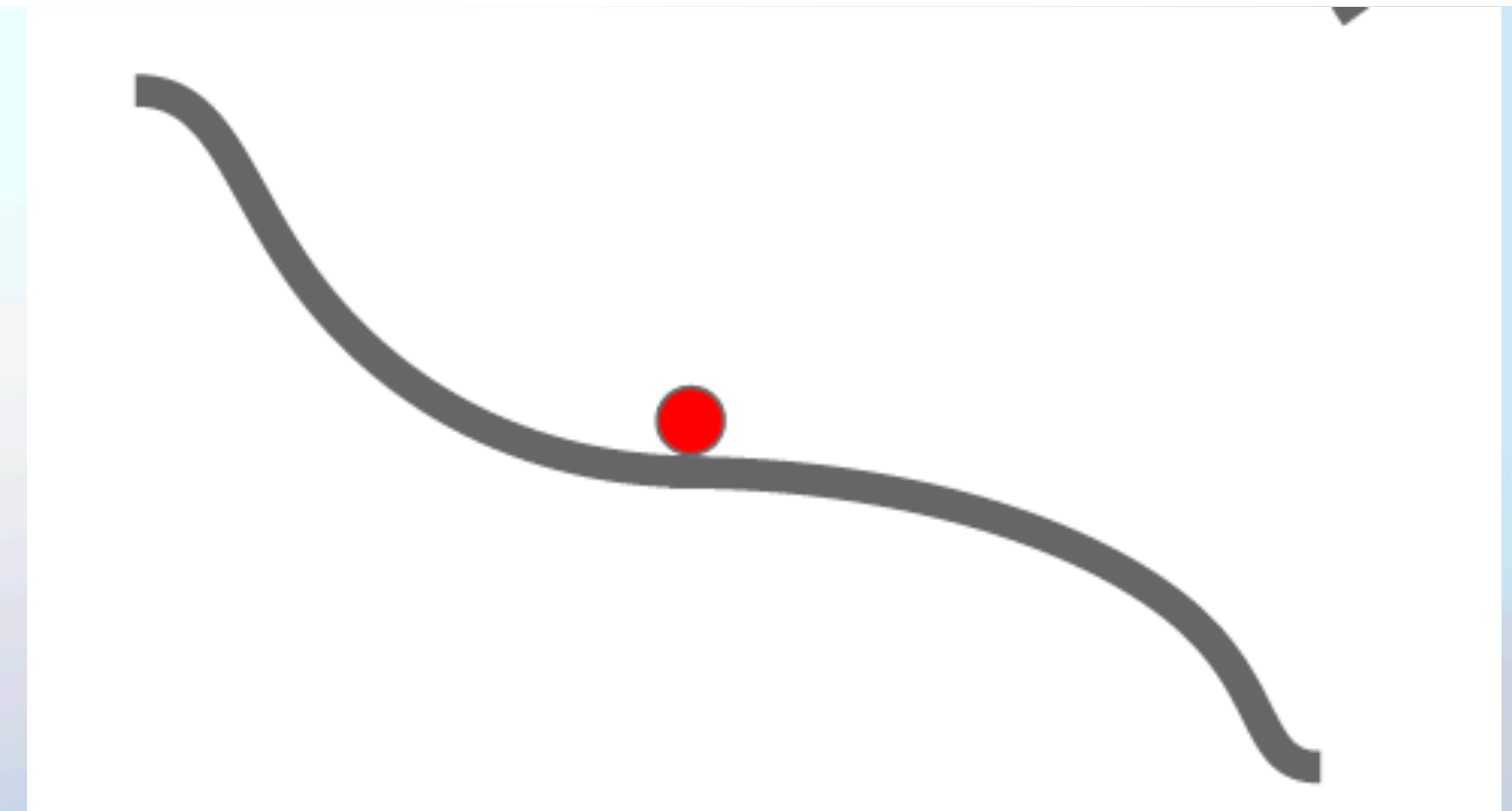
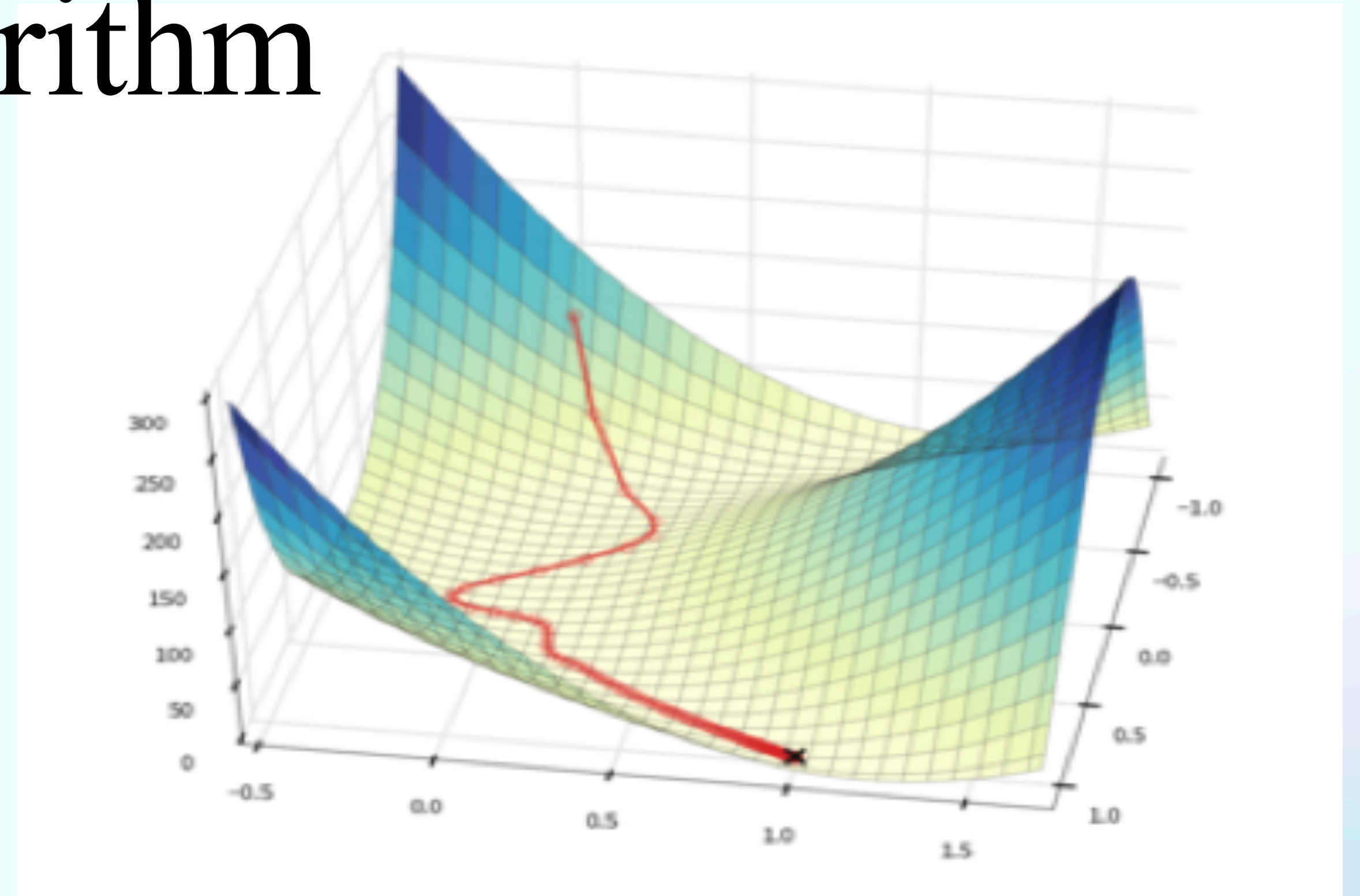
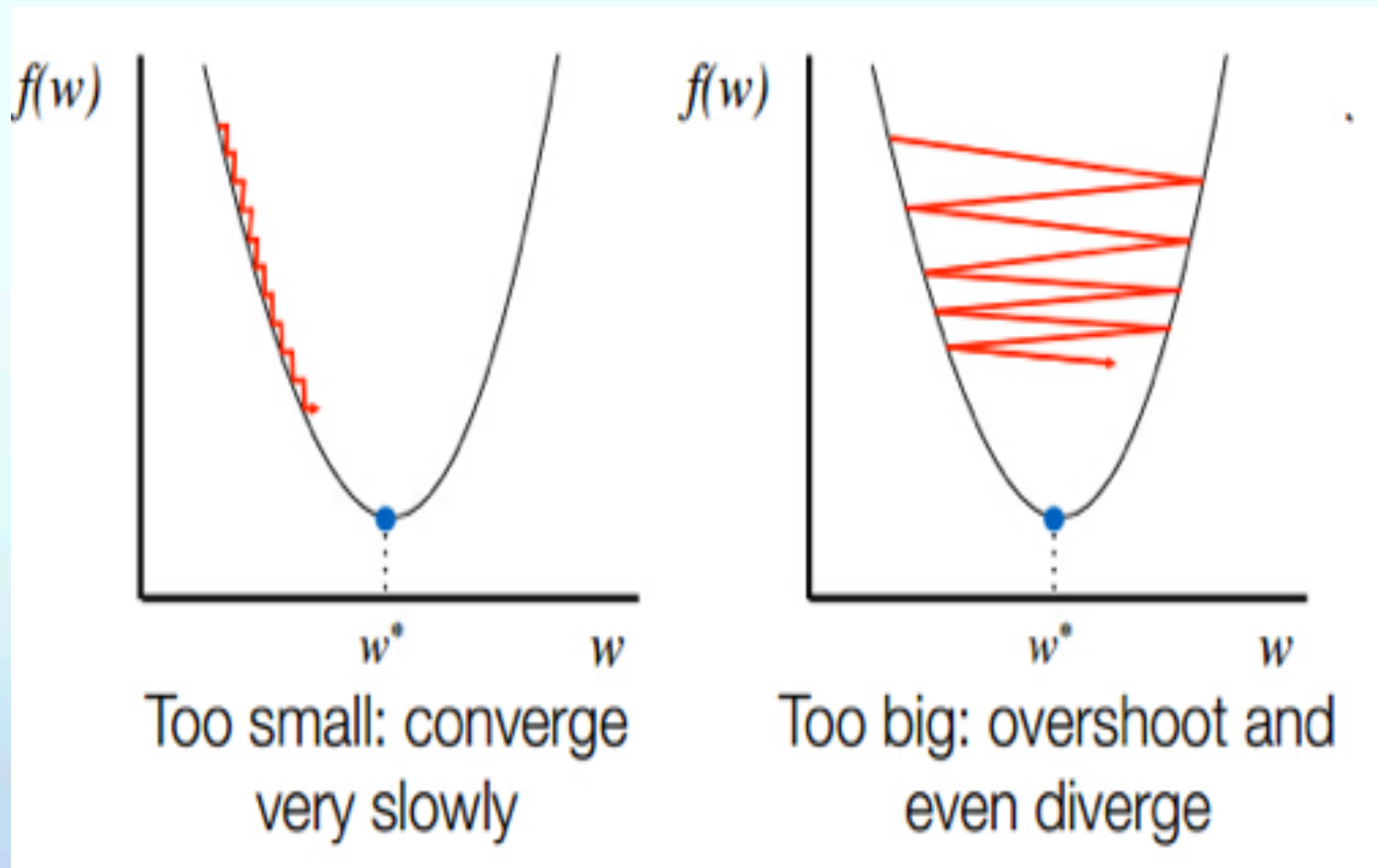
- w is the weight, η is the learning rate, and $\frac{\partial L}{\partial w}$ is the gradient of the loss with respect to the weight



Gradient Descent Algorithm

- Gradient descent works well for convex functions because the local minima become the global minima.
- Challenges:
 - Local minima and saddle points: In complex cost landscapes, Gradient Descent can get stuck in local minima or saddle points, especially in non-convex optimization problems common in deep learning.
 - Choosing the right learning rate: Finding an optimal learning rate requires experimentation and tuning. (Below, we'll see how adaptive learning rate algorithms can help alleviate this issue.)
 - It is expensive to calculate the gradients if the size of the data is huge.

Gradient Descent Algorithm



Stochastic Gradient Descent

- The stochastic gradient descent algorithm also works in a similar way but instead of calculating the gradient over the full dataset, it calculates the gradient value over each sample of the training dataset and updates the parameters(weights) accordingly
- So, if the dataset contains 1000 rows SGD will update the model parameters 1000 times in one cycle of dataset instead of one time as in Gradient Descent.

- Update rule:

- $$v = \gamma v + \eta \cdot \frac{\partial L}{\partial w}$$

- $$w_{new} = w - v$$

- γ is the momentum term, and v is the velocity.

Stochastic Gradient Descent

- Select an example from the dataset.
- Calculate its gradient, $\frac{\partial L}{\partial w}$
- Utilize the calculated gradient from step 2 to update the model weights, $w = w - \eta \cdot \frac{\partial L}{\partial w}$
- Repeat steps 1 to 3 for all examples in the training dataset.
- Completing a full pass through all the examples constitutes one epoch.
- Repeat this entire process for several epochs as specified during training.

Stochastic Gradient Descent

- Challenges:
 - Variance: The updates can be noisy due to the reliance on a single example, potentially causing the cost function to fluctuate. As a result, the algorithm does not converge to a minimum but jumps around the cost landscape.
 - Hyperparameter tuning: Correctly setting the learning rate requires experimentation.
- Advantages:
 - Efficiency: Using only one example at a time, SGD significantly reduces the computational requirements, making it faster and more scalable than Gradient Descent.
 - Escape local minima: The inherent noise in SGD can help the algorithm escape shallow local minima, potentially leading to better solutions in complex cost landscapes.
 - Online learning: SGD is well-suited for online learning scenarios where the model needs to update continuously as new data arrives.

Mini-batch Gradient Descent

- The mini-batch stochastic gradient descent algorithm calculates the gradient value over a mini-batch (a subset of training data) instead of every single sample. Now, this modification keeps the algorithm computationally feasible and as the gradient value is averaged over a few examples it also provides smooth convergence.
- With small learning rates and some other modifications, mini-batch SGD almost surely converges to the global minimum(or the local minimum in the case of pseudo-convex loss functions).

Mini-batch Gradient Descent

- Select a mini-batch, B
- Compute the mean gradient of the mini-batch, $\frac{\partial L}{\partial w} = \frac{1}{B} \sum_{k=1}^B \frac{\partial L_k}{\partial w}$
- Apply the mean gradient obtained in step 2 to update the model's weights, $w = w - \eta \cdot \frac{\partial L}{\partial w}$
- Repeat steps 1 to 2 for all the mini-batches that have been created.

Mini-batch Gradient Descent

- Challenges
 - Hyperparameter tuning: Like with the other variants, selecting the learning rate requires experimentation. Further, the choice of the batch size is crucial. If the batch size is too small, it faces the drawbacks of SGD, and if the batch size is too large, it is prone to the issues of basic Gradient Descent.
 - May get stuck at local minima
- Advantages:
 - Requires less memory space and Less time is required to converge when compared to SGD
 - Efficiency and stability: Mini-batch Gradient Descent offers a compromise between the computational efficiency of SGD and the stability of Gradient Descent.
 - Parallelization: Since mini-batches only contain a small, fixed number of samples, they can be computed in parallel, speeding up the training process.
 - Generalization: By not using the entire dataset for each update, Mini-batch Gradient Descent can help prevent overfitting, leading to models that generalize better on unseen data.

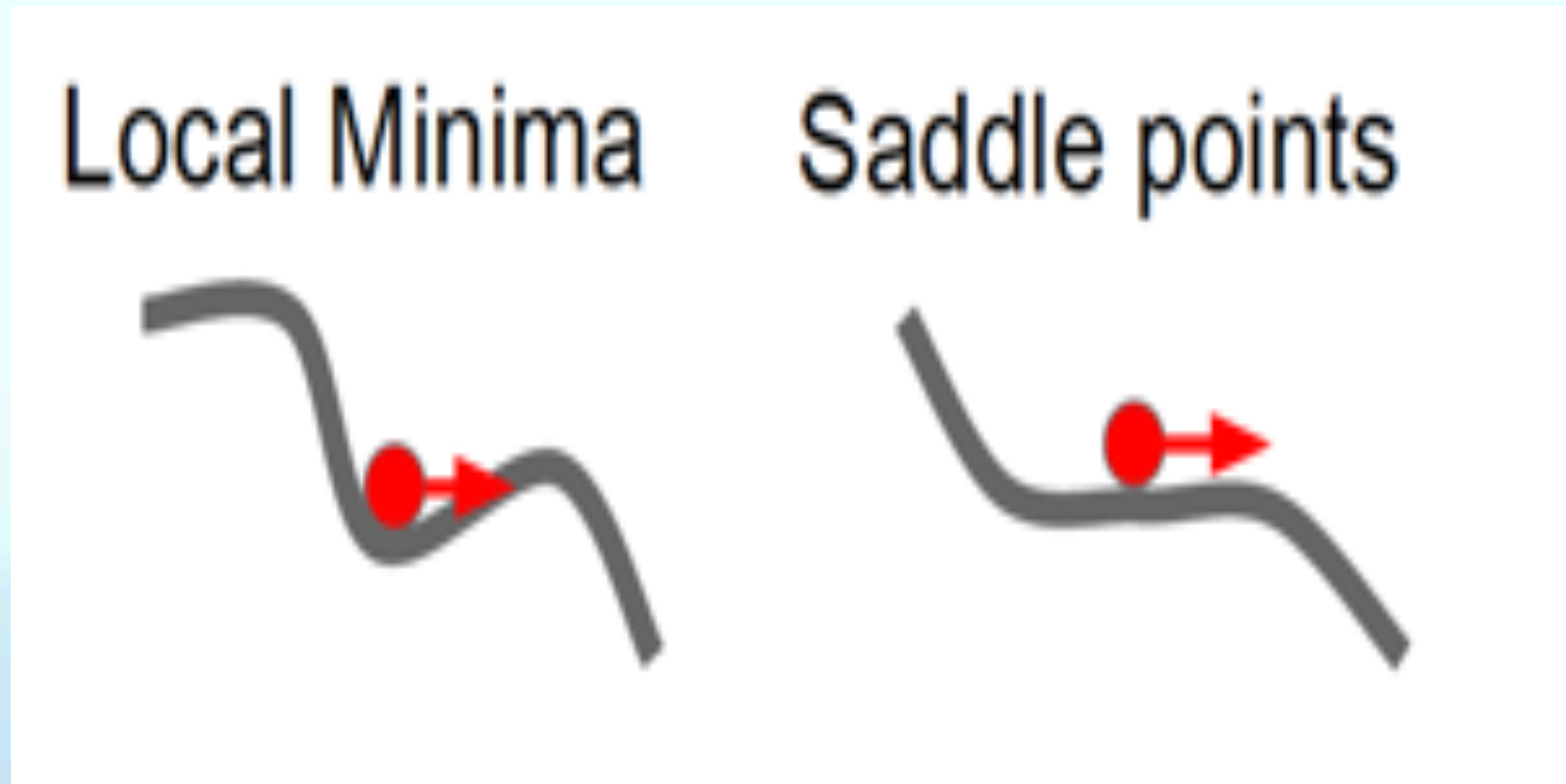
SGD with Momentum

- The momentum method of SGD optimizer, remembers the last few gradient values from the past training batches and does not just rely on the recently calculated gradient while making updates to the parameters.
- Instead, it takes inputs from the past gradient values to make sure that we are always moving towards the correct direction of convergence.
- The concept behind Momentum involves denoising the derivatives by employing an exponential weighting average by assigning more weight to recent updates compared to previous ones.
- In this way, the momentum method of the SGD optimizer achieves fast convergence without oscillations.

SGD with Momentum

- Select a mini-batch, B
- Compute the mean gradient of the mini-batch, $\frac{\partial L}{\partial w} = \frac{1}{B} \sum_{k=1}^B \frac{\partial L_k}{\partial w}$
- Compute velocity update, $v_{t+1} = \mu \cdot v_t - \alpha \frac{\partial L}{\partial w}$
- Update the parameters $w = w + v$

Escaping Saddle Points and Local Minima with Momentum



SGD with Momentum

- Advantages
 - Mitigates parameter oscillations and reduces parameter variance.
 - Achieves faster convergence compared to standard gradient descent.
- Disadvantage
 - Introduces an additional hyper-parameter that must be chosen manually and with precision

Adaptive Gradient (AdaGrad) Algorithm

- AdaGrad (Adaptive Gradient Algorithm) introduces dynamically adapting the learning rate to the conventional Gradient Descent, allowing for a more nuanced and effective optimization process.
- Square and accumulate the gradients in the gradient accumulation variable, which consequently tracks the sum of squares of the gradients for each parameter.
- Adjust learning rate: Modify the learning rate for each parameter inversely proportional to the square root of its accumulated gradient, ensuring parameters with smaller gradients have larger updates.
- This modification is highly beneficial because real-world datasets contain sparse as well as dense features.

- Update rule :
$$w := w - \frac{\alpha}{\sqrt{G_t + \epsilon}} \cdot \nabla_w J(w)$$

Adagrad Algorithm

- Select a mini-batch, B
- Compute the mean gradient of the mini-batch, $g = \frac{\partial L}{\partial w} = \frac{1}{B} \sum_{k=1}^B \frac{\partial L_k}{\partial w}$
- Accumulate squared gradient, $r = r + g \otimes g$
- Compute velocity update, $v = -\frac{\epsilon}{\delta + \sqrt{r}} \otimes g$
- Update the parameters $w = w + v$

Adaptive Gradient (AdaGrad) Algorithm

- Advantages:
 - Adaptive learning rates: By adjusting the learning rates based on past gradients, AdaGrad can effectively handle data with sparse features and different scales.
 - Simplicity and efficiency: AdaGrad simplifies the need for manual tuning of the learning rate, making the optimization process more straightforward.
- Challenges:
 - Diminishing learning rates: As training progresses, the accumulated squared gradients can grow very large, causing the learning rates to shrink and become infinitesimally small. This can prematurely halt the learning process.

Root Mean Square Propagation(RMSprop)

- RMSprop, like its predecessors, aims to optimize the model's parameters to minimize the cost function. by adjusting the learning rate for each parameter using a **moving average of recent squared gradients**, ensuring efficient and stable convergence.
- Calculate the gradient of the cost function for each parameter using a selected subset of the training data (mini-batch).
- Update the running average of squared gradients using a decay factor, γ , often set to 0.9. This moving average emphasizes more recent gradients, preventing the learning rate from diminishing too rapidly.
- Scale down the gradient by the square root of the updated running average, normalizing the updates and allowing for a consistent pace of learning across parameters.

RMSprop (contd)

- Select learning rate - ϵ ; decay rate - ρ ; small constant δ , usually 10^{-6}
- Select a mini-batch, B
- Compute the mean gradient of the mini-batch, $g = \frac{\partial L}{\partial w} = \frac{1}{B} \sum_{k=1}^B \frac{\partial L_k}{\partial w}$
- Accumulate squared gradient, $r = \rho r + (1 - \rho) g \otimes g$
- Compute velocity update, $v = -\frac{\epsilon}{\delta + \sqrt{r}} \otimes g$
- Update the parameters $w = w + v$

Root Mean Square Propagation(RMSprop)

- Advantages:
 - Adaptive learning rates: RMSprop dynamically adjusts learning rates, making it robust to the scale of gradients and well-suited for optimizing deep neural networks.
 - Overcoming AdaGrad's limitations: By focusing on recent gradients, RMSprop prevents the aggressive, monotonically decreasing learning rate problem seen in AdaGrad, ensuring sustained progress in training.

Adaptive Moment (Adam) Estimation Algorithm

- Adam (Adaptive Moment Estimation) combines the best properties of AdaGrad and RMSprop to provide an optimization algorithm that can handle sparse gradients on noisy problems.
- Adam seeks to optimize the model's parameters to minimize the cost function, utilizing adaptive learning rates for each parameter. It uniquely combines momentum (keeping track of past gradients) and scaling the learning rate based on the second moments of the gradients, making it effective for a wide range of problems.

$$w = w - \frac{\alpha \cdot \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

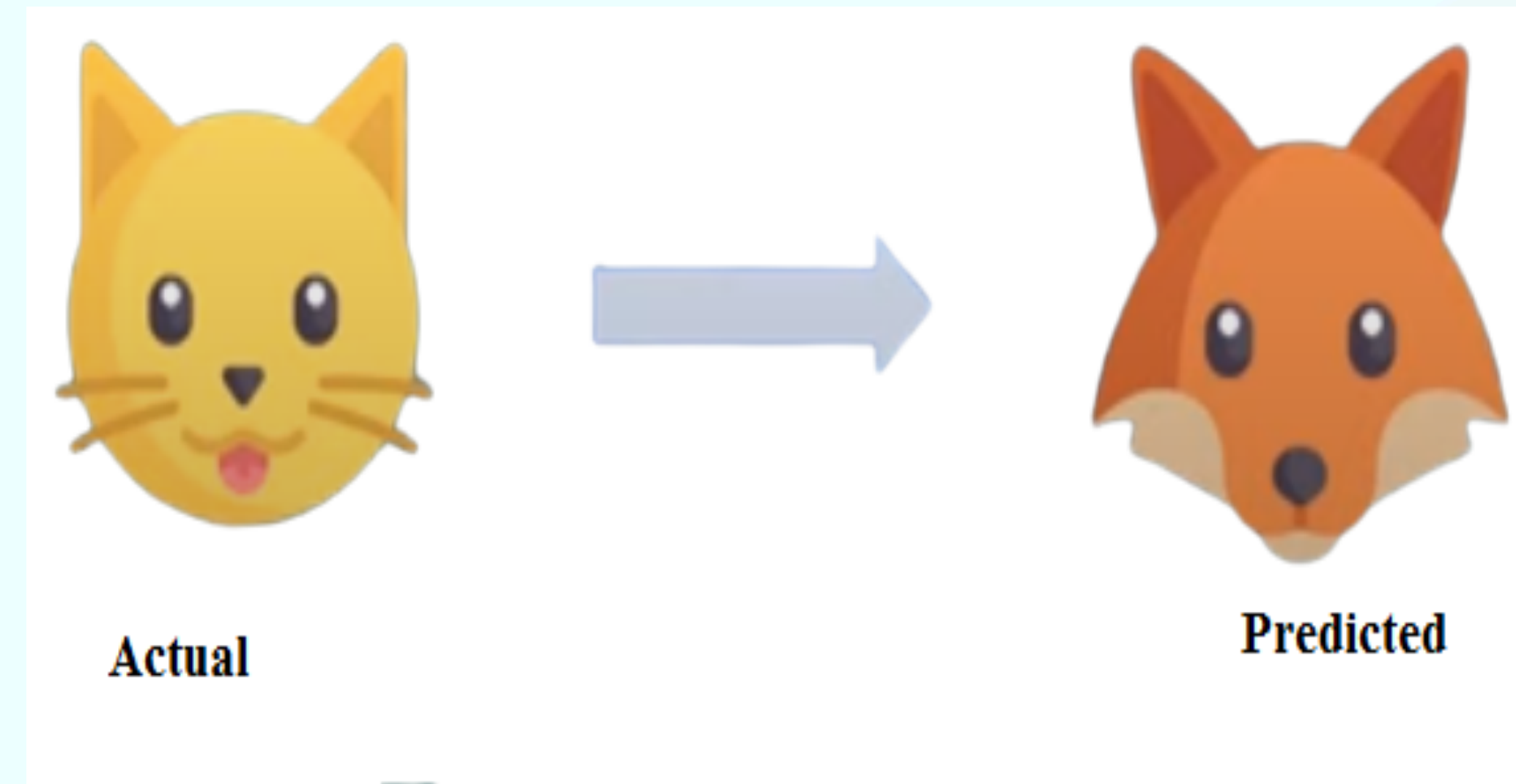
- Update rule:

Errors in Deep Learning

- An algorithm's ability to forecast values for a dataset that was unknown at the beginning is measured by an error. The deep learning model that performs the best on the specific dataset is chosen based on these errors. In deep learning, there are two primary categories of errors:
- Errors in Deep Learning:
 - Reducible Errors
 - Bias
 - Variance
 - Irreducible Errors

Bias

- What is a bias?
 - Bias is the error that calculates the difference between the average prediction of our model and the actual value that we are trying to predict.
- A model may have:
 - Low Bias: A low bias model will make fewer assumptions about the form of the target function.
 - High Bias: A model with a high bias makes more assumptions, and the model becomes unable to capture the important features of our dataset. A high bias model also cannot perform well on new data.



Bias (contd)

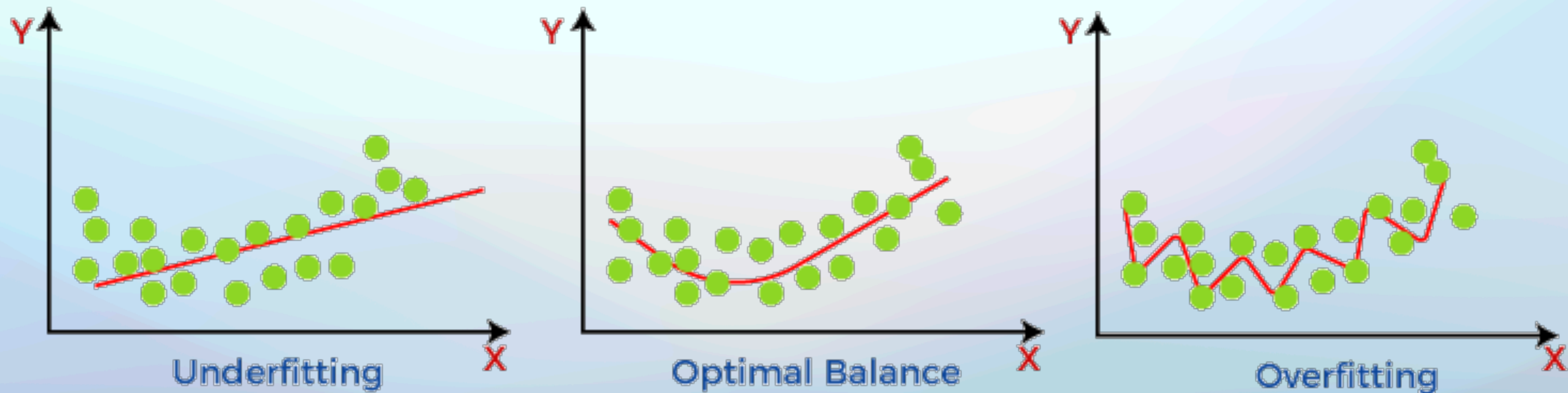
- Characteristics of a high bias model include:
 - Not able to capture proper data trends
 - Trained over noise also. So giving less accurate results
 - Suffers from underfitting
 - A more general or simple model
- High bias can be reduced by:
 - Increase the input features as the model is underfitted.
 - Decrease the regularization term.
 - Use more complex models, such as including some polynomial features.

Variance

- Variance is the amount by which the performance of a predictive model changes when it is trained on different subsets of the training data. More specifically, variance is the variability of the model that how much it is sensitive to another subset of the training dataset. i.e., how much can it adjust on the new subset of the training dataset?
- Variance errors are either of low variance or high variance.
 - Low variance: Low variance means that the model is less sensitive to changes in the training data and can produce consistent estimates of the target function with different subsets of data from the same distribution.
 - High variance: High variance means that the model is very sensitive to changes in the training data and can result in significant changes in the estimate of the target function when trained on different subsets of data from the same distribution.
- Usually, nonlinear algorithms have a lot of flexibility to fit the model, and have high variance.

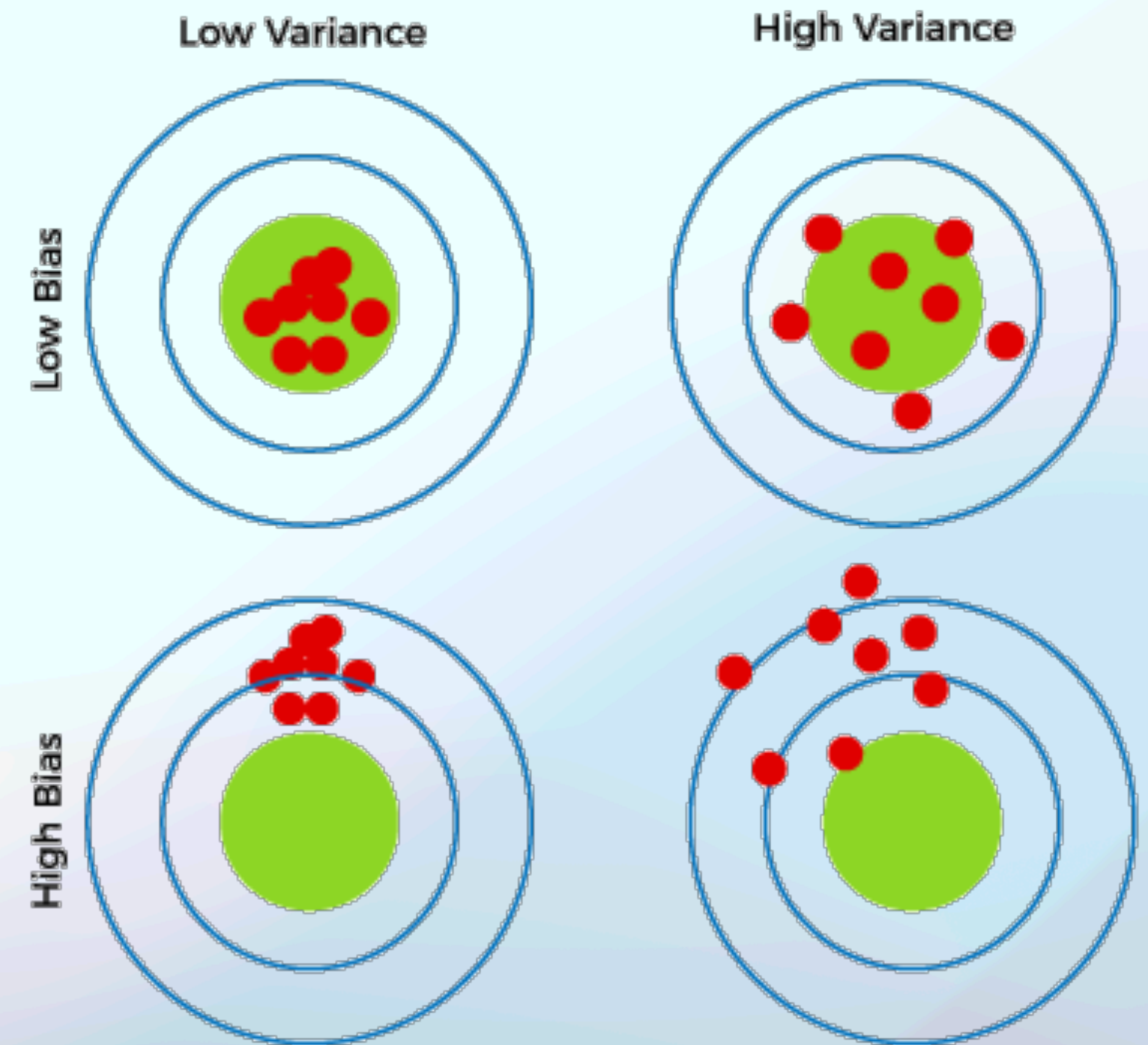
Variance (contd)

- Ways to Reduce High Variance:
 - Reduce the input features or number of parameters as a model is overfitted.
 - Do not use a much complex model.
 - Increase the training data.
 - Increase the Regularization term.

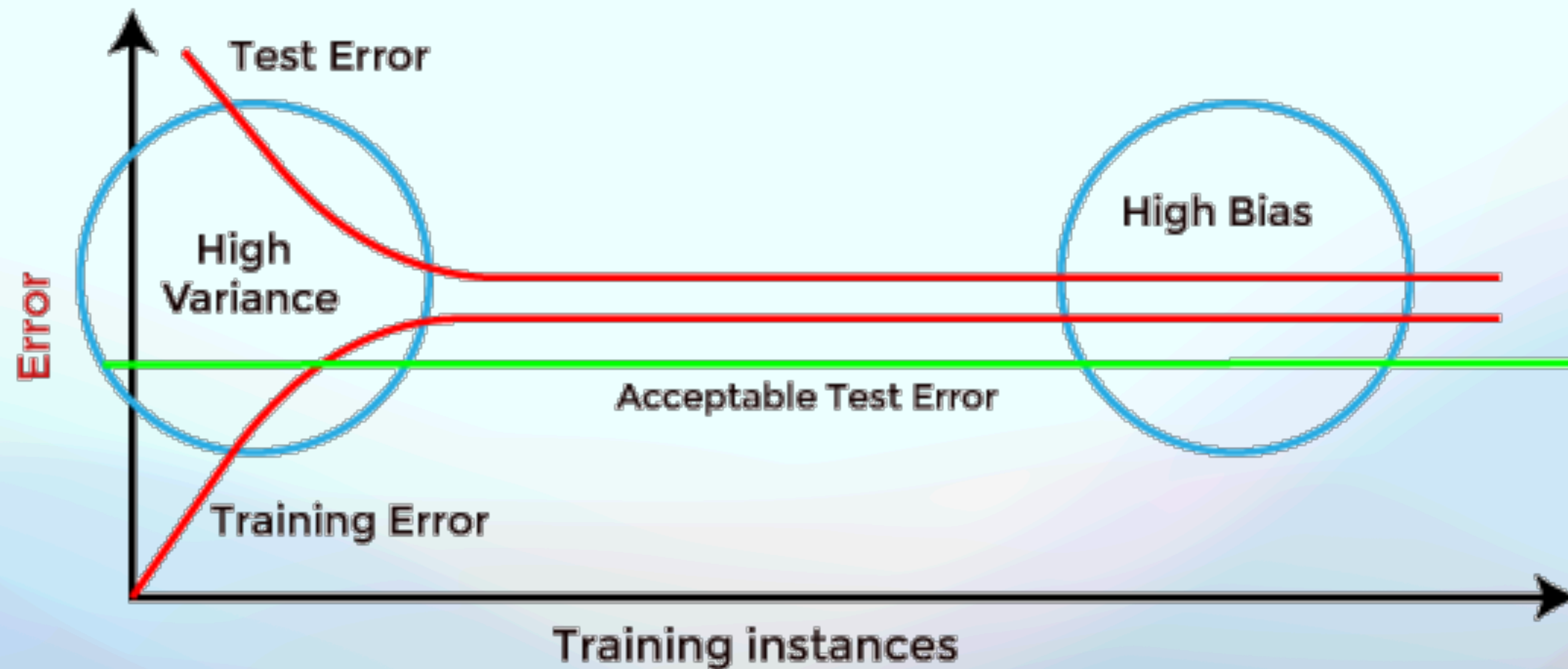


Combinations of Bias-Variance

- **Low-Bias, Low-Variance:** The combination of low bias and low variance shows an ideal machine learning model. However, it is not possible practically.
- **Low-Bias, High-Variance:** With low bias and high variance, model predictions are inconsistent and accurate on average. This case occurs when the model learns with a large number of parameters and hence leads to an overfitting
- **High-Bias, Low-Variance:** With High bias and low variance, predictions are consistent but inaccurate on average. This case occurs when a model does not learn well with the training dataset or uses few numbers of the parameter. It leads to underfitting problems in the model.
- **High-Bias, High-Variance:** With high bias and high variance, predictions are inconsistent and also inaccurate on average.



High variance or High Bias??



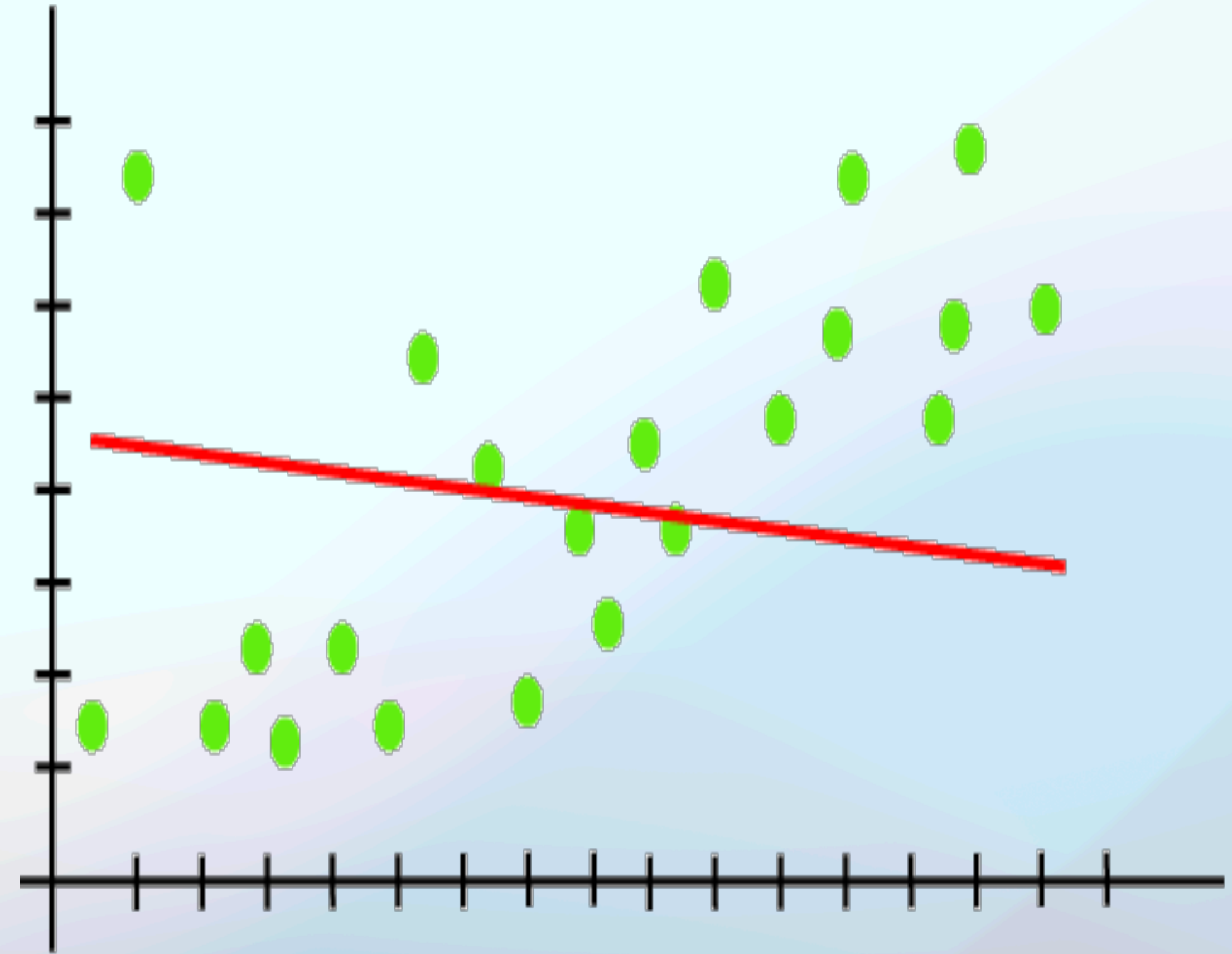
Bias-Variance Trade-Off

- It is required to make a balance between bias and variance errors, and this balance between the bias error and variance error is known as the bias variance trade-off.
- For an accurate prediction of the model, algorithms need a low variance and low bias. But this is not possible because bias and variance are related to each other:
 - If we decrease the variance, it will increase the bias.
 - If we decrease the bias, it will increase the variance.
- Hence, the Bias-Variance trade-off is about finding the sweet spot to make a balance between bias and variance errors.



Underfitting

- Underfitting occurs when the model is not able to capture the underlying trend of the data.
- To avoid the overfitting in the model, the fed of training data can be stopped at an early stage, due to which the model may not learn enough from the training data. As a result, it may fail to find the best fit of the dominant trend in the data.
- Hence it reduces the accuracy and produces unreliable predictions.
- An underfitted model has high bias and low variance.



Underfitting

- **Reasons for underfitting**

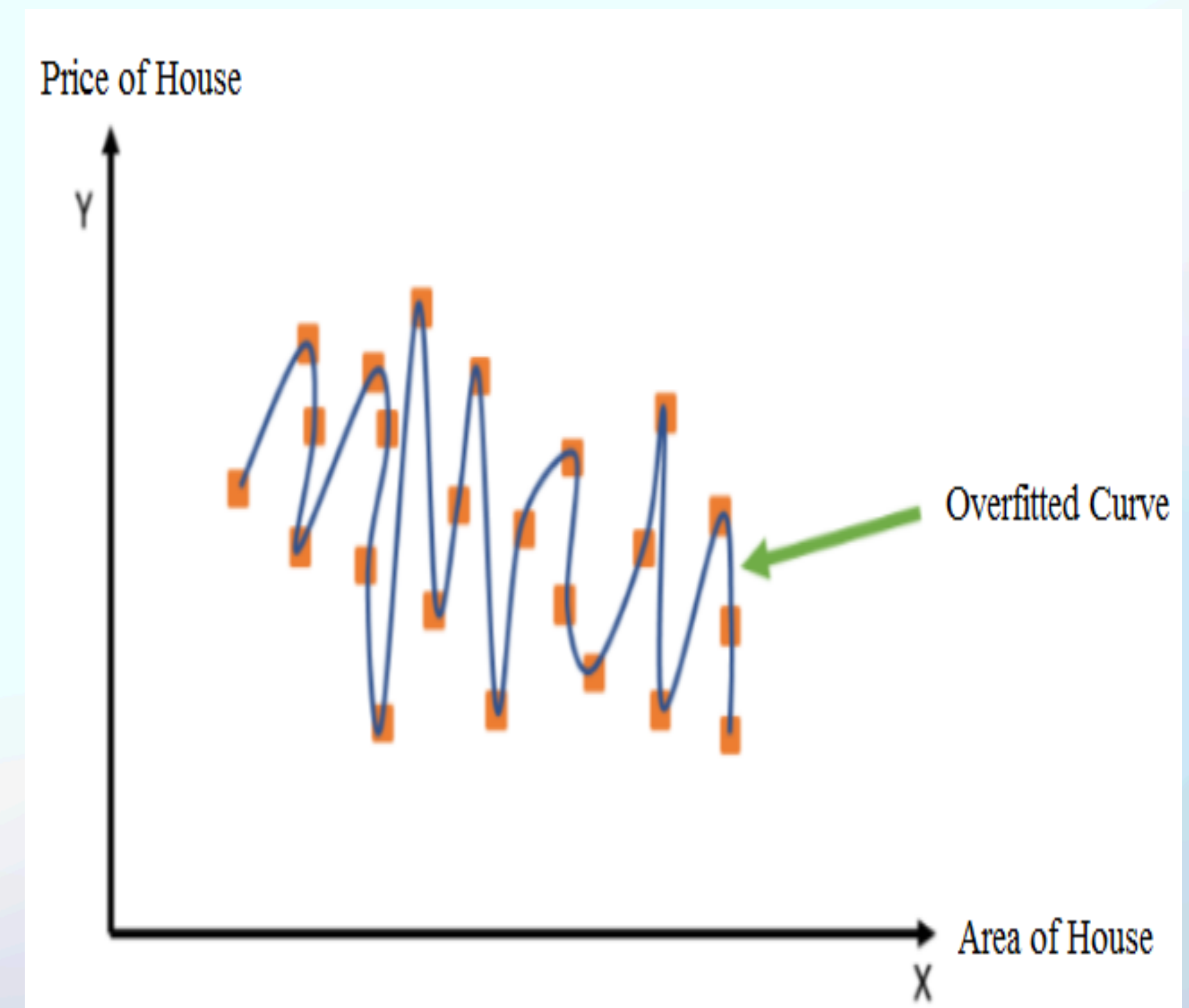
- Uncleaned data(contains noise) used for training
- High bias in the model.
- Less training data
- The model is too simple

- **How to avoid underfitting**

- Preprocessing the data to reduce noise in the data
- More training to the model
- Increasing the number of features in the dataset
- Increasing the model complexity
- Reduce noise in the data

Overfitting

- Overfitting occurs when the model tries to cover all the data points or more than the required data points present in the given dataset. Because of this, the model starts caching noise and inaccurate values present in the dataset, and all these factors reduce the efficiency and accuracy of the model.
- Chances of Overfitting are more with **non-parametric** and **non-linear** models that have more flexibility when learning a target function.
- For example, decision trees(nonparametric algorithms) are very flexible and are subject to overfitting training data.
- The overfitted model has low bias and high variance.



Overfitting

- **Reasons for Overfitting**

- Noisy training data
- High variance in the model
- Complex model
- Not using a resampling technique

- **How to avoid Overfitting**

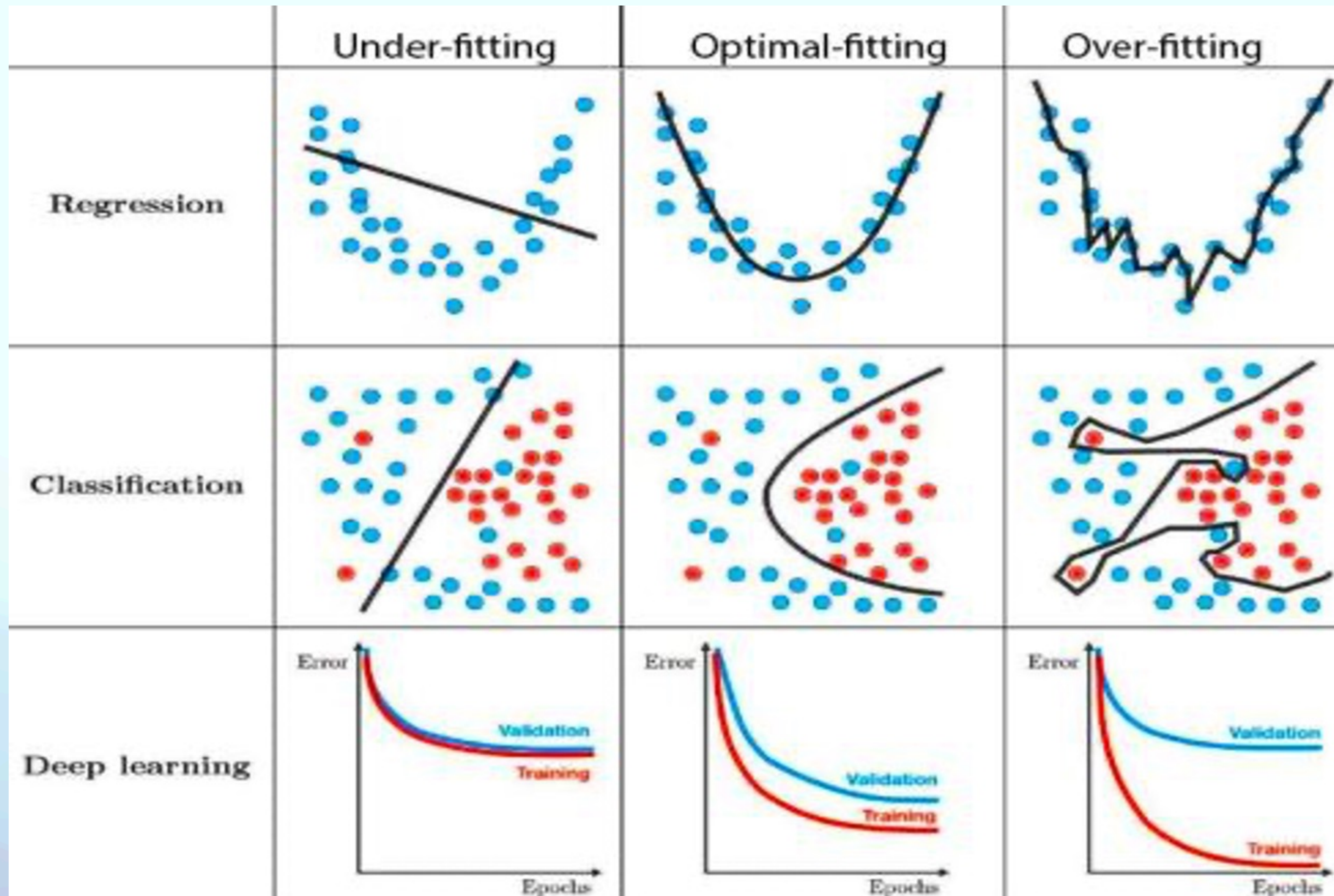
- Using K-fold cross-validation
- Using *Regularization techniques* such as Lasso and Ridge
- Implementing ensembling techniques.
- Picking a less parameterized/complex model
- Training the model with sufficient data

???

1. You are training a deep learning model on a small dataset of medical images to detect a specific type of tumor. After 100 epochs, you notice that the training accuracy is 99%, but the validation accuracy is only 70%.
2. A deep learning model trained for predicting patient readmission rates shows poor results on validation data.
3. You are building a simple neural network to predict house prices based on features like size, number of rooms, and location. Despite training for many epochs, the model's predictions are significantly off from the actual prices.

What signs indicate that your model is underfitting, and what changes would you make to improve its performance?

Model



Regularization techniques

- Regularization is a technique used in machine learning and deep learning to prevent overfitting and improve the generalization performance of a model on unseen data. It involves adding a penalty term to the loss function during training.
- In machine learning, regularization penalizes the coefficients. In deep learning, it penalizes the weight matrices of the nodes.
- Some regularization techniques are:
 - L1 and L2 regularization
 - Dropout
 - Data Augmentation
 - Early Stopping

L1 and L2 Regularization

- L1 and L2 are the most common types of regularization deep learning. These update the general cost function by adding another term known as the regularization term.
- Cost function = Loss (say, binary cross entropy) + Regularization term
- Due to the addition of this regularization term, the values of weight matrices decrease because it assumes that a neural network with smaller weight matrices leads to simpler models. Therefore, it will also reduce overfitting to quite an extent.
- L1 regularization : L1 regularization is also called lasso regression. L1 regularization adds the absolute values of the weights to the loss function. This encourages some weights to become exactly zero, effectively performing feature selection. Hence, it is very useful when we are trying to compress our model. Otherwise, we usually prefer L2 over it.
- For L1: $loss + \lambda * \sum w$

- L2 regularization, is known as ridge regression, on the other hand, adds the square of the weights to the loss function. This tends to evenly distribute the importance across all features, reducing the magnitude of weights and preventing them from growing too large.

$$\text{For L2: } \text{loss} + \lambda * \sum w^2$$

- where lambda is the regularization parameter. It is the hyperparameter whose value is optimized for better results. L2 regularization is also known as weight decay as it forces the weights to decay towards zero (but not exactly zero).

- L2 regularization, is known as ridge regression, on the other hand, adds the square of the weights to the loss function. This tends to evenly distribute the importance across all features, reducing the magnitude of weights and preventing them from growing too large.

$$\text{For L2: } \text{loss} + \lambda * \sum w^2$$

- where lambda is the regularization parameter. It is the hyperparameter whose value is optimized for better results. L2 regularization is also known as weight decay as it forces the weights to decay towards zero (but not exactly zero).

from keras import regularizers

model.add(Dense(64, input_dim=64, kernel_regularizer=regularizers.l2(0.01))

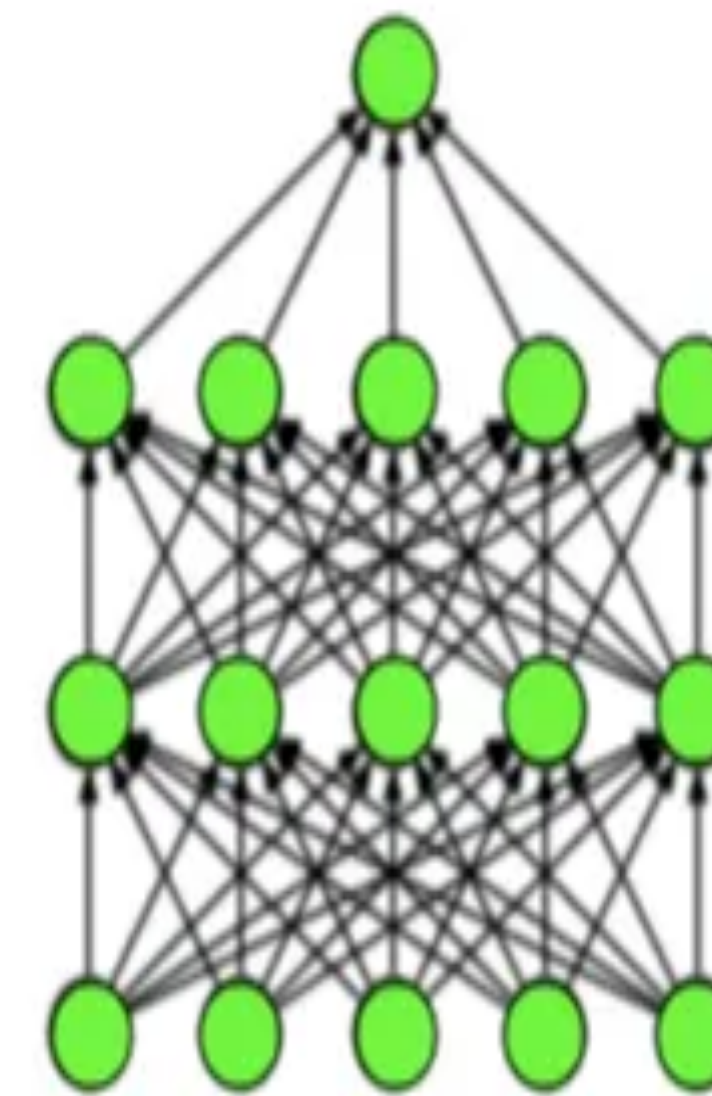
L1 & L2 regularization at the same time (Elastic Net)

- Regularization term: $loss + \lambda_1 \|w\| + \lambda_2 w^2$
- Or, if we number each individual weight w in the model $i = 1, 2, 3, \dots, n$, then regularization term is

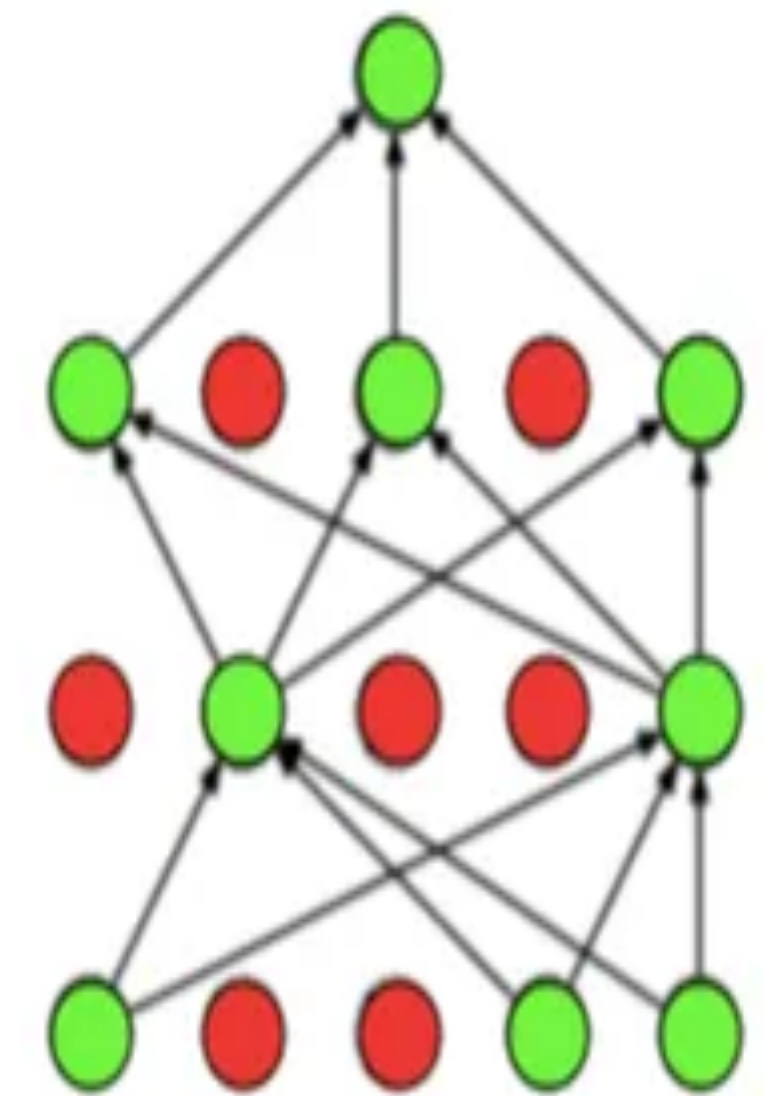
$$loss + \lambda_1 \sum \|w_i\| + \lambda_2 \sum w_i^2$$

Dropout

- During training, dropout randomly deactivates a fraction of neurons in each layer. This simulates training multiple neural networks in parallel, forcing the model to be more robust and preventing it from relying too heavily on any one neuron.
- At every iteration, it randomly selects some nodes and removes them along with all of their incoming and outgoing connections as shown below:
- So each iteration has a different set of nodes and this results in a different set of outputs. It can also be thought of as an ensemble technique in machine learning.

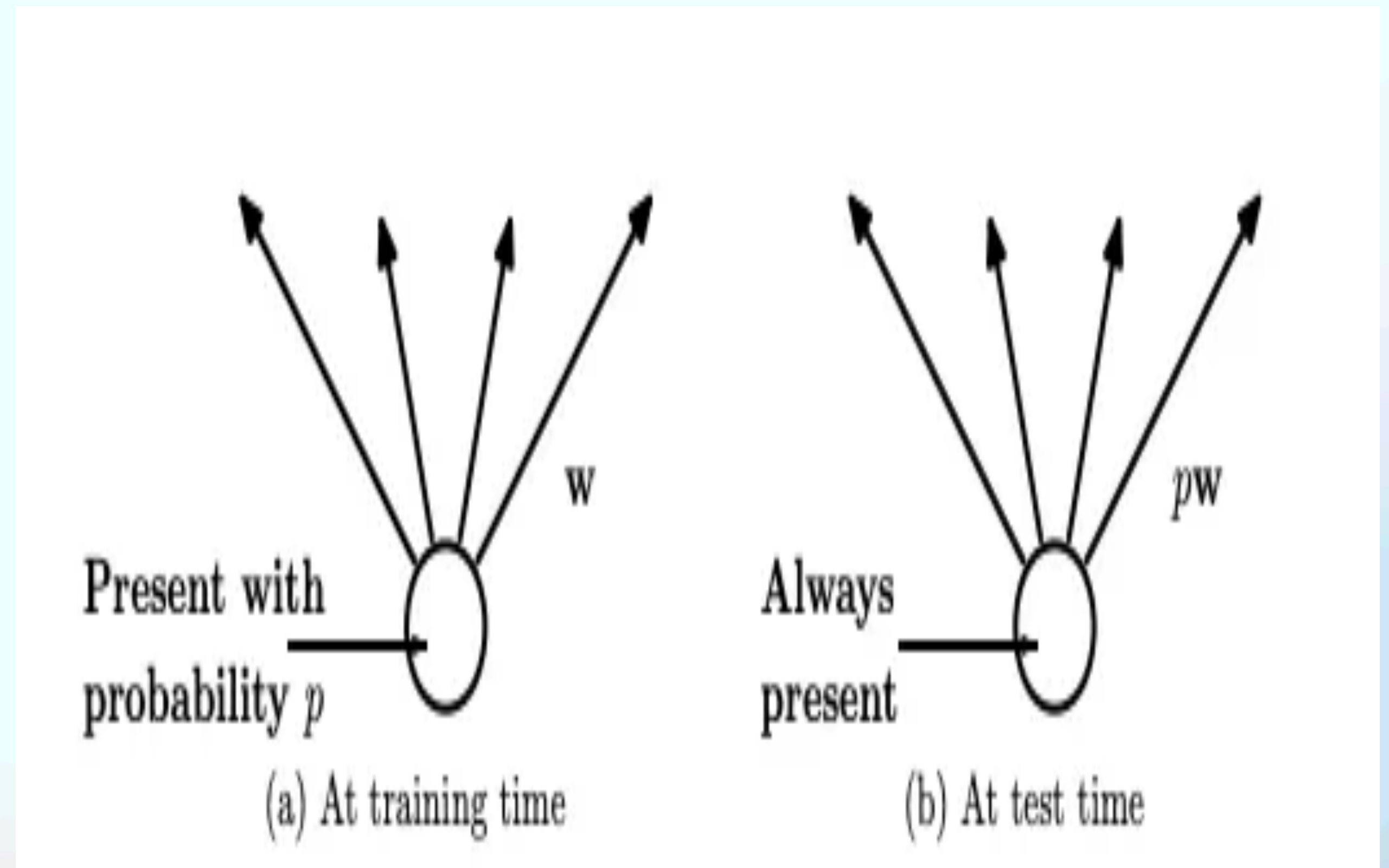
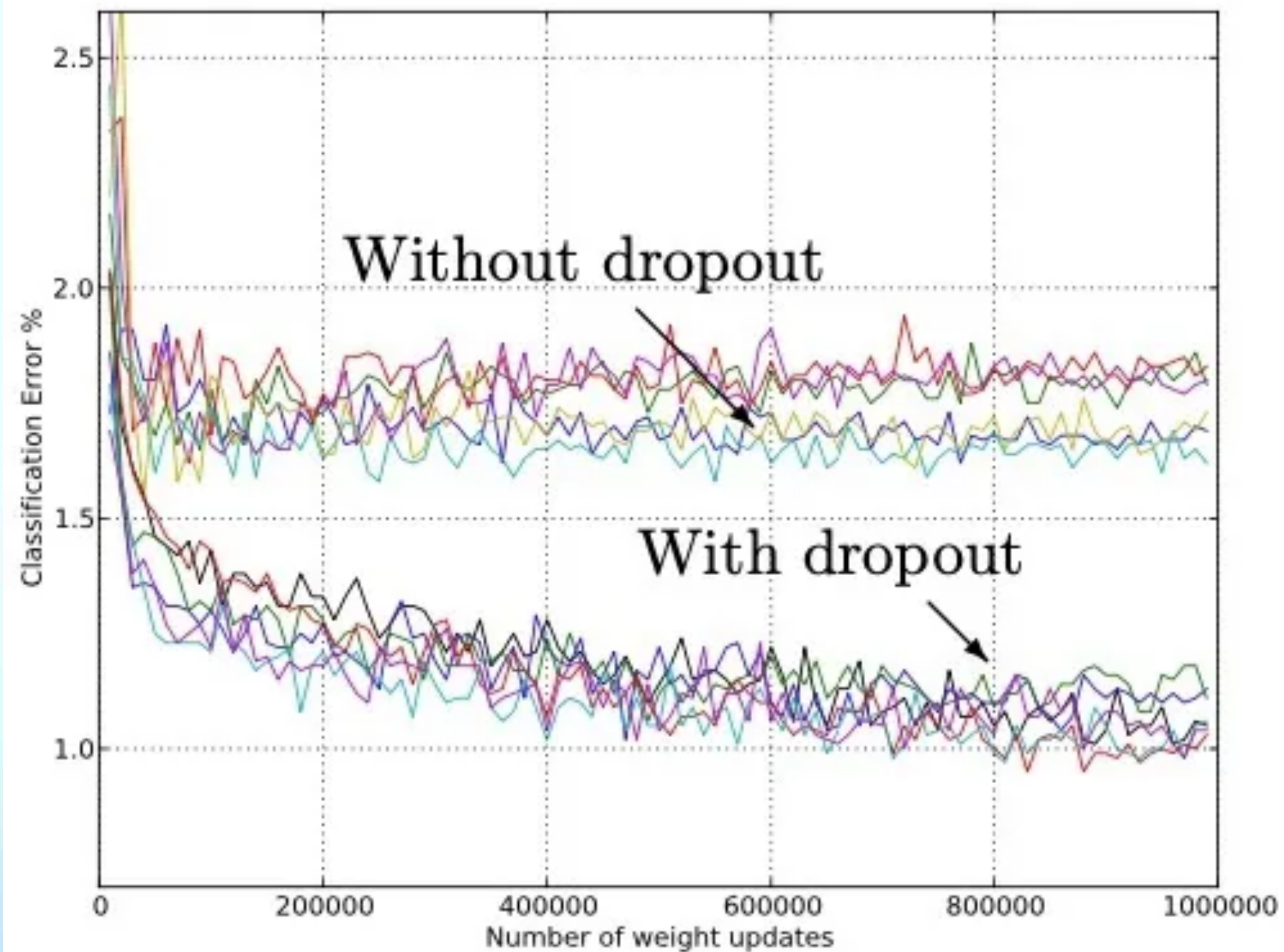


(a) Standard Neural Net



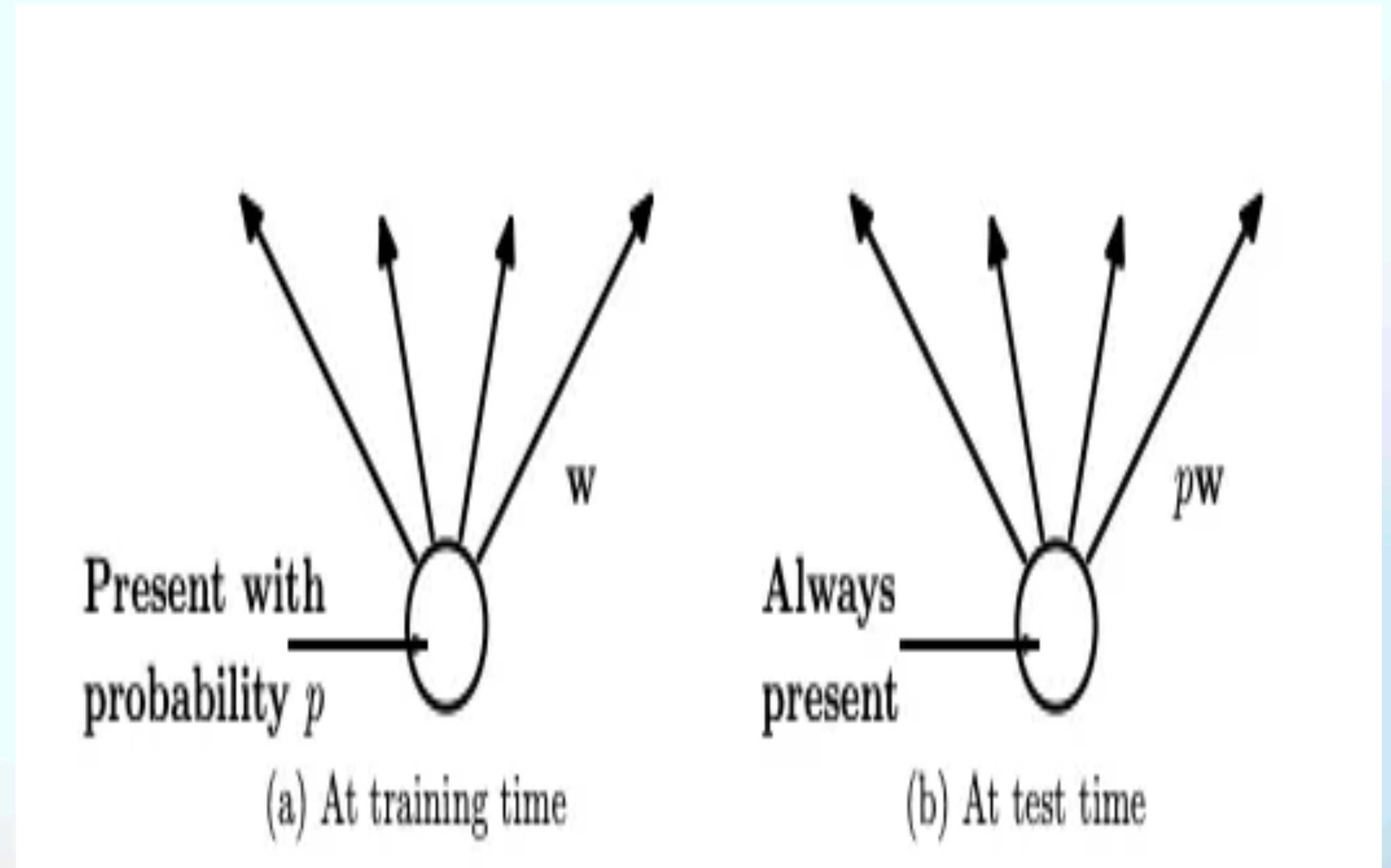
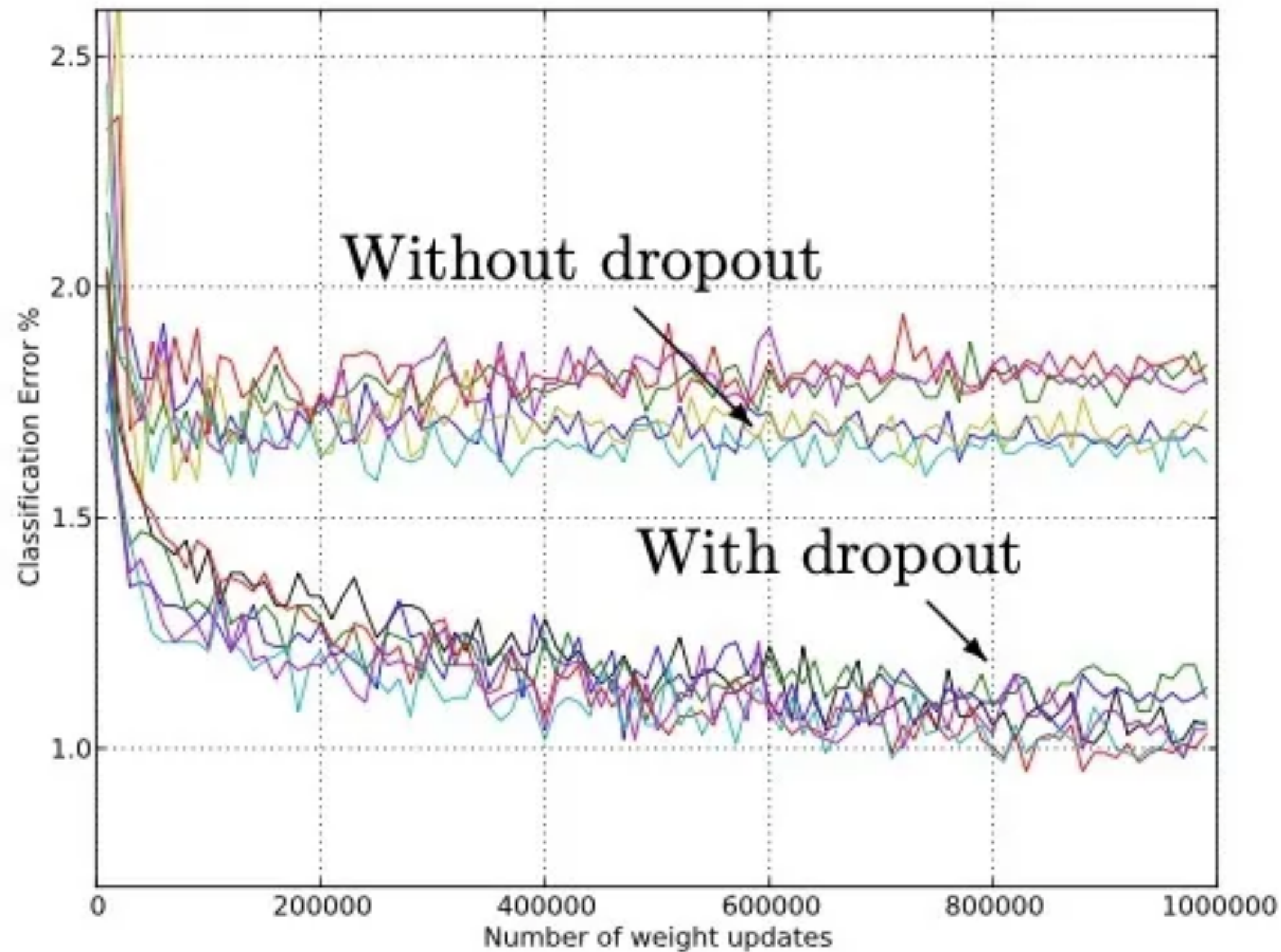
(b) After applying dropout.

Dropout (contd)



Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting", JMLR 2014

Dropout (contd)

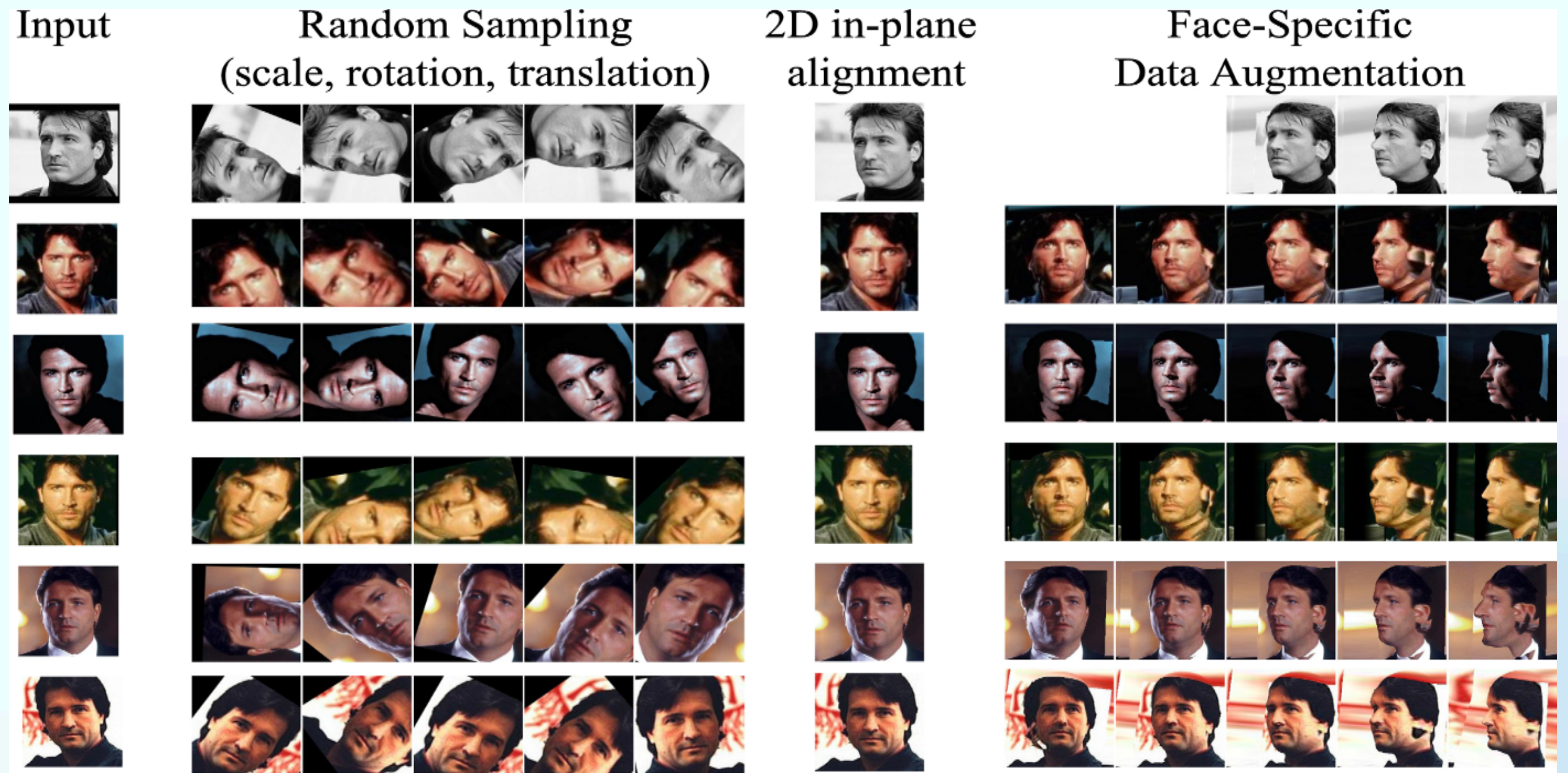


Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting", JMLR 2014

- `model = Sequential(Dense(output_dim=hidden1_num_units, input_dim=input_num_units, activation='relu'), Dropout(0.25))`

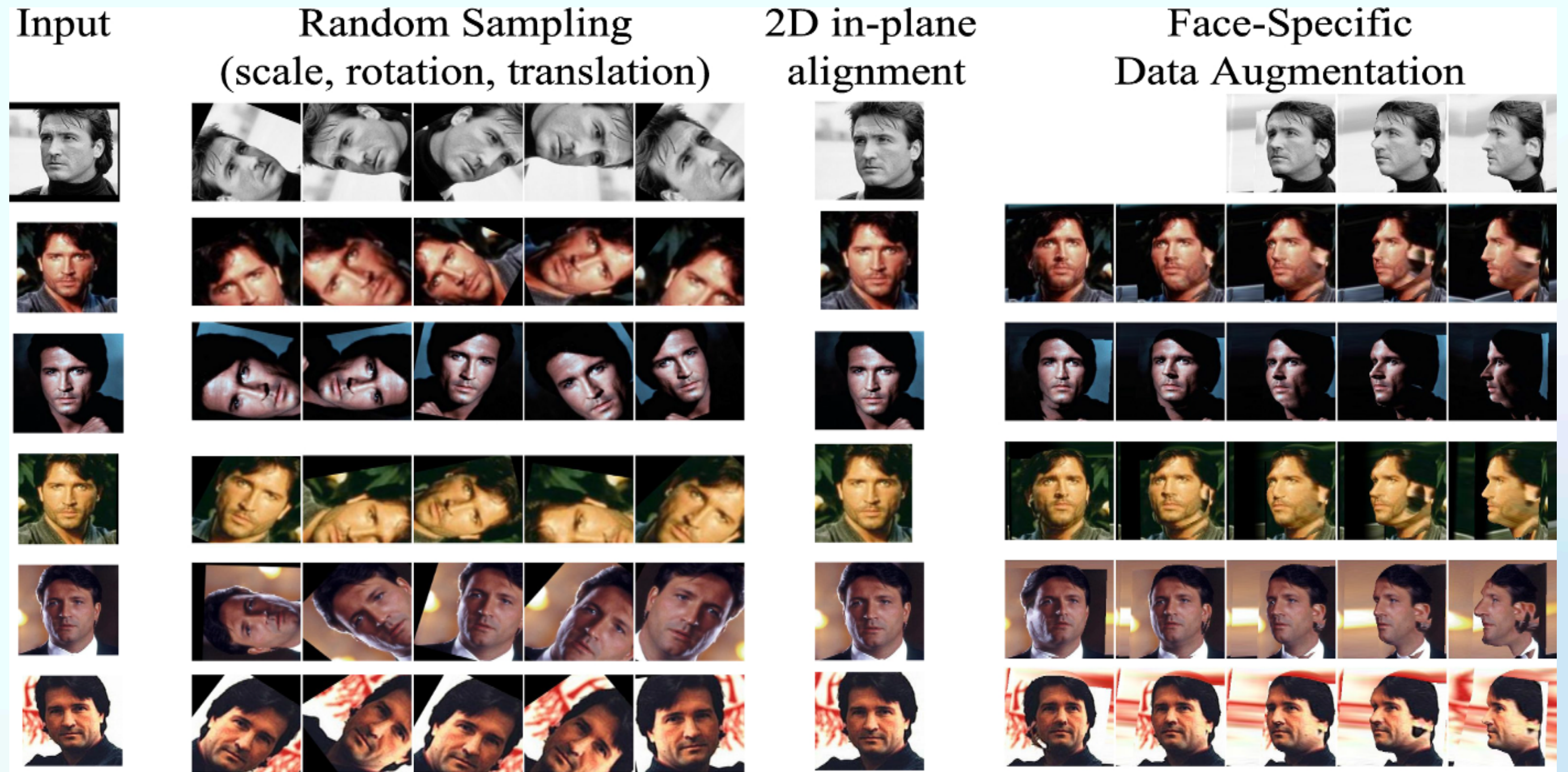
Data Augmentation

- Data augmentation involves randomly applying diverse augmentations, such as rotations, flips, crops, or color adjustments, to the input data. Then, we train on those images rather than the original image. Random mix/combinations can be performed like translation, shearing, stretching, distorting depending on your creativity.



Data Augmentation

- Data augmentation involves randomly applying diverse augmentations, such as rotations, flips, crops, or color adjustments, to the input data. Then, we train on those images rather than the original image. Random mix/combinations can be performed like translation, shearing, stretching, distorting depending on your creativity.



```
from keras.preprocessing.image import ImageDataGenerator
```

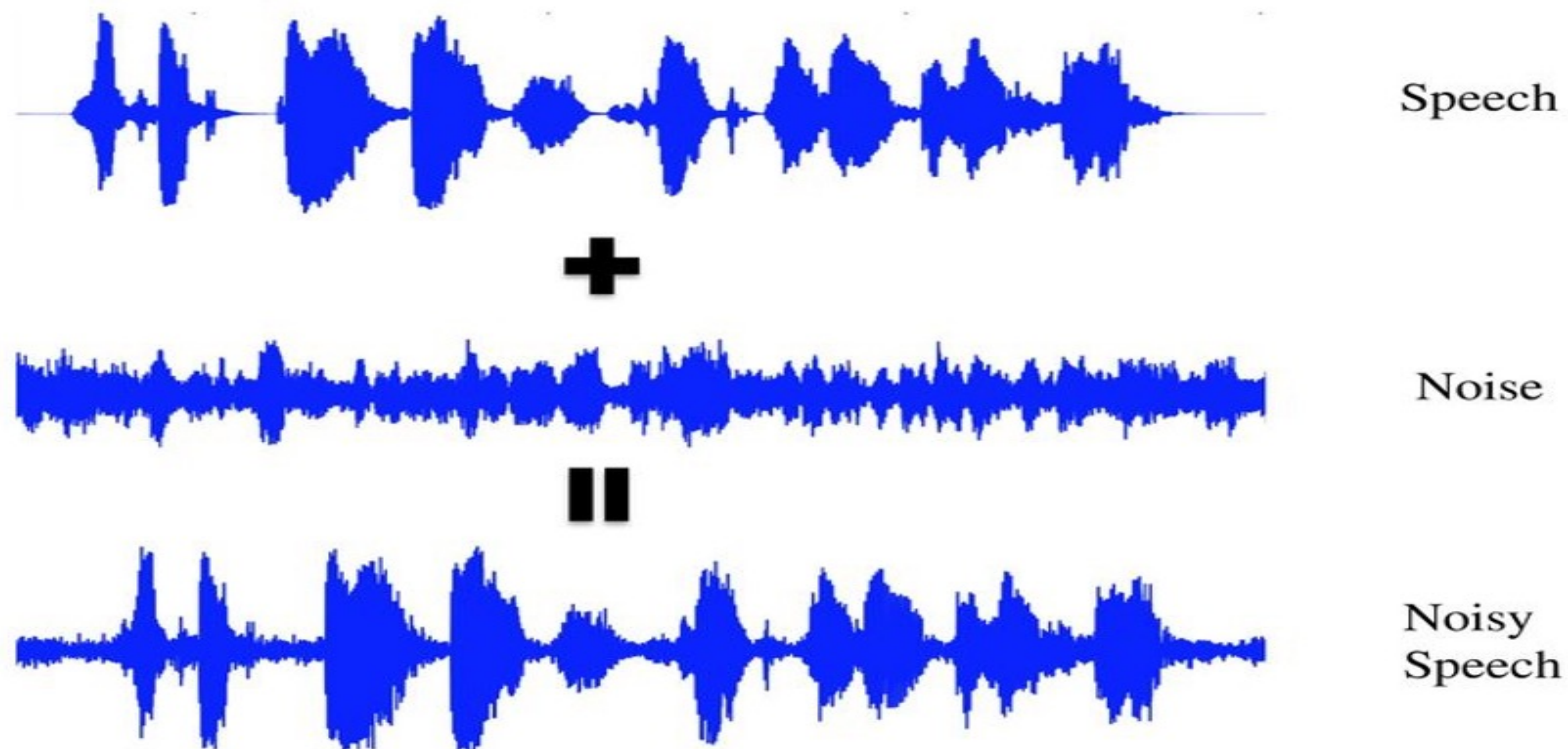
```
datagen = ImageDataGenerator(horizontal flip=True)
```

```
datagen.fit(train)
```


Data Augmentation

Deep Speech - Even more data!

Augmentation: noise synthesis, reverb, time-stretching, pitch-shifting,...



- `from keras.preprocessing.image import ImageDataGenerator`
- `datagen = ImageDataGenerator(
 • featurewise_center=False, # set input mean to 0 over the dataset
 • samplewise_center=False, # set each sample mean to 0
 • featurewise_std_normalization=False, # divide inputs by std of the dataset
 • samplewise_std_normalization=False, # divide each input by its std
 • zca_whitening=False, # apply ZCA whitening
 • rotation_range=10, # randomly rotate images in the range (degrees, 0 to 180)
 • zoom_range = 0.1, # Randomly zoom image
 • width_shift_range=0.1, # randomly shift images horizontally (fraction of total width)
 • height_shift_range=0.1, # randomly shift images vertically (fraction of total height)
 • horizontal_flip=False, # randomly flip images
 • vertical_flip=False) # randomly flip images`
- `datagen.fit(x_train)`

Early Stopping

- Instead of training the model for a fixed number of epochs, monitor its performance on a validation set. When performance stops improving or starts degrading, then stop training. It is known as early stopping.



```
from keras.callbacks import EarlyStopping
```

```
earlystop= EarlyStopping(monitor='val_acc', patience=3)
```


Try.....

- You are training a neural network using SGD with L1 regularization. The learning rate α is set to 0.01, and the regularization parameter λ is set to 0.1. You are at the 20th iteration, and the gradient of the loss with respect to a weight w is $g_{20} = 0.02$. The current weight is $w_{19} = 0.5$.

Calculate the updated weight w_{20} using the L1 regularization.

- You are training a neural network using SGD with L2 regularization. The learning rate α is set to 0.01, and the regularization parameter λ is set to 0.05. You are at the 30th iteration, and the gradient of the loss with respect to a weight w is $g_{30} = -0.03$. The current weight is $w_{29} = 0.4$.

Calculate the updated weight w_{30} using the L2 regularization.

Try.....

- You are training a neural network using SGD with combined L1 and L2 regularization (Elastic Net). The learning rate α is set to 0.01, the L1 regularization parameter λ_1 is 0.1, and the L2 regularization parameter λ_2 is 0.05. You are at the 40th iteration, and the gradient of the loss with respect to a weight w is $g_{40} = 0.04$. The current weight is $w_{\{39\}} = -0.3$